N-body Simulations.ipynb

MARKDOWN #1

```
# N-body Simulations by <span style="color: #0366d6;">**Dimo Dimov**</span>
```

MARKDOWN #2

```
<div style="background-color: #ffffff; border: 2px solid #0366d6; padding: 35px;
border-radius: 15px; font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Helvetica,
Arial, sans-serif; color: #24292e; max-width: 950px; margin: 20px auto; box-shadow: 0 10px
25px rgba(0,0,0,0.05);">

<h1 style="color: #0366d6; margin-top: 0; text-align: center; font-size: 2.5em;
 b o r d e r - b o t t o m :   2 p x   s o l i d   # f 1 f 8 f f ;   p a d d i n g - b o t t o m :   1
Framework</h1>

<h3 style="color: #586069; text-align: center; font-weight: 400; margin-top: 10px;">An
Experimental Journey from Planck Scales to Galactic Dynamics</h3>

<p style="font-size: 1.15em; line-height: 1.8; color: #444d56; text-align: justify; margin:
30px 0;">
    Welcome to a comprehensive computational exploration of <b>orbital mechanics and
gravitational chaos</b>.
    This project transitions from the fundamental laws of motion to the complex,
unpredictable "dance"
    of multiple celestial bodies interacting in 1D, 2D, and 3D space.
</p>

<table style="width: 100%; border-collapse: collapse; margin-bottom: 30px;">
    <tr>
        <td style="width: 50%; vertical-align: top; padding: 20px; background-color:
#f6f8fa; border-radius: 10px 0 0 10px; border-right: 2px solid #ffffff;">
                        < h 4   s t y l e = " c o l o r :   # 0 3 6 6 d 6 ;   m a r g i n - t o p :
            <ul style="padding-left: 20px; font-size: 0.95em; color: #24292e; line-height:
1.6;">
                <li>Simulate stable and chaotic N-body systems.</li>
                <li>Verify Keplerian laws and orbital stability.</li>
                <li>Optimize computation using Barnes-Hut.</li>
            </ul>
        </td>
        <td style="width: 50%; vertical-align: top; padding: 20px; background-color:
#f1f8ff; border-radius: 0 10px 10px 0;">
                        < h 4   s t y l e = " c o l o r :   # 0 0 5 c c 5 ;   m a r g i n - t o p :
            <ul style="padding-left: 20px; font-size: 0.95em; color: #24292e; line-height:
1.6;">
                <li>Information-Mass Equivalence Principle.</li>
                <li>Spatially Varying Higgs Field models.</li>
                <li>Quantum boundaries at the Planck Scale.</li>
            </ul>
        </td>
    </tr>
</table>

<p style="text-align: right; font-size: 0.9em; color: #6a737d; border-top: 1px solid
#eaecef; padding-top: 15px; margin-bottom: 0;">
    Created by <span style="color: #0366d6; font-weight: bold;">Dimo Dimov</span> |
```

Mathematical Concepts for Developers (2026)
</p>

MARKDOWN #3

```
<div class="alert alert-block alert-danger">
        < b >Ø=Ý4  I M P O R T A N T  D I S C L A I M E R :  S p e c u l a t i v e  C o n t e n t
    <br><br>
    This project is an experimental exploration of N-body systems and <b>blends established scientific facts with highly speculative models</b>. Please be advised that the content may contain errors in:
    <ul>
        <li><b>Mathematics & Physics:</b> Formulas and theoretical derivations.</li>
        <li><b>Code & Algorithms:</b> Implementation logic and numerical stability.</li>
        <li><b>Simulations & Visualizations:</b> Accuracy of orbital paths and data representation.</li>
    </ul>
    This notebook is intended for educational and creative purposes only. <b>It must not be taken as absolute scientific truth or used as a reference for professional astronomical calculations.</b>
</div>
```

MARKDOWN #4

```
# The N-body Problem: Mathematical and Algorithmic Challenges

The **N-body problem** involves predicting the individual motions of a group of celestial objects interacting with each other gravitationally. While the 2-body problem (like Earth and the Moon) has a closed-form analytical solution, the problem becomes chaotic and analytically unsolvable for $N \geq 3$.

### 1. The Mathematical Deadlock
For $N > 2$, there are no general closed-form solutions (integrals of motion) to describe the positions of the bodies over time. This necessitates the use of **numerical integration** to approximate the state of the system at discrete time steps.
```

MARKDOWN #5

```
### 2. Computational Complexity: The $O(N^2)$ Barrier
The primary algorithmic challenge is the calculation of gravitational forces. According to Newton's Law of Universal Gravitation:
$$F_{ij} = G \frac{m_i m_j}{r_{ij}^2}$$

In a system of $N$ particles, each particle interacts with every other particle.
- For $N=100$, we perform $\approx 10,000$ calculations per step.
- For $N=1,000,000$ (a small galaxy), we would need $1,000,000,000,000$ calculations per step.

This **$O(N^2)$ complexity** makes direct summation impossible for large-scale simulations, requiring more advanced algorithms like **Barnes-Hut** or **Fast Multipole Methods**.
```

MARKDOWN #6

```
### 3. Numerical Challenges
- **Singularities:** When two particles get extremely close, the distance $r \to 0$, causing the force to approach infinity. This leads to massive numerical errors. We mitigate this
```

using **softening parameters**.
- **Energy Drift:** Simple integration methods (like Euler) fail to conserve energy, causing orbits to artificially spiral outward or inward. This requires **Symplectic Integrators** like Velocity Verlet.
- **Sensitivity to Initial Conditions:** The system is inherently chaotic. Tiny differences in initial positions lead to vastly different outcomes over time (the "Butterfly Effect").

MARKDOWN #7

```
## Environment Configuration
This cell organizes all required libraries following PEP 8 standards.
Groups are divided into Standard Libraries, Numerical/Scientific tools, and Visualization
frameworks.
```

MARKDOWN #8

```
<div class="alert alert-block alert-warning">
        < b >& þ    I m p o r t a n t    N o t e :    S e q u e n t i a l    E x e c u t i o n    R e q u
    <br><br>
    To ensure the simulations and visualizations function correctly, please execute the code
cells <b>sequentially</b> from top to bottom. This project relies on physical constants,
helper functions, and data structures initialized in the early stages of the notebook.
Skipping cells may lead to <code>NameError</code> or unexpected simulation behavior.
    <br><br>
        < b >#ó    P e r f o r m a n c e    N o t e : < / b >    D u e    t o    t h e    c o m p l e x i t
1D/2D/3D rendering, it may take <b>several minutes</b> for all simulations and interactive
visualizations to fully process and display.
</div>
```

MARKDOWN #9

```
<div class="alert alert-block alert-info">
        < b >Ø=Ü¡    N o t e    o n    V i s u a l i z a t i o n    S t y l e s < / b >
    <br>
    As you execute the code cells, you may notice that the background color of the plots
switches between <b>white (light)</b> and <b>black (dark)</b> styles. This is intended to
match the specific aesthetic of each simulation (e.g., space environments vs. mathematical
trajectory analysis).
    <br><br>
    The visual theme is updated globally using <code>plt.style.use()</code> in specific
cells, affecting subsequent visualizations until the next style change.
</div>
```

CODE #10

```
# --- 1. Standard Library Imports ---
import os
import time

# --- 2. Third-Party Numerical & File Processing ---
import nbformat
import numpy as np
from scipy.spatial import cKDTree

# --- 3. Visualization: Matplotlib Core & Extensions ---
import matplotlib as mpl
import matplotlib.pyplot as plt
import matplotlib.patches as patches
```

```
from matplotlib.animation import FuncAnimation
from matplotlib.patches import Polygon, Circle
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.axes_grid1.inset_locator import zoomed_inset_axes, mark_inset

# --- 4. Interactive & Web-Based Rendering ---
import plotly.graph_objects as go
from IPython.display import HTML

print("All libraries imported and organized according to PEP 8.")
```

MARKDOWN #11

```
## The $O(N^2)$ Barrier

The **quadratic scaling ($O(N^2)$)** of gravitational force calculations renders direct
summation computationally prohibitive for large-scale systems. To maintain performance, we
must transition to more efficient, sub-quadratic approaches like the **Barnes-Hut
algorithm** or **Fast Multipole Methods (FMM)**.
```

CODE #12

```
# 1. Prepare Data
n_values = np.array([10, 100, 1000, 5000, 10000])
complexity_n2 = n_values**2

# 2. Setup Figure with 1 row and 2 columns
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# --- Plot 1: Linear Scale ---
ax1.plot(n_values, complexity_n2, 'r-o', label='O(N^2) - Direct Summation')
ax1.fill_between(n_values, complexity_n2, color='red', alpha=0.1)
ax1.set_title("Computational Scaling (Linear Scale)")
ax1.set_xlabel("Number of Bodies (N)")
ax1.set_ylabel("Number of Operations")
ax1.grid(True, linestyle='--', alpha=0.6)
ax1.legend()

# --- Plot 2: Logarithmic Scale ---
ax2.loglog(n_values, complexity_n2, 'b-o', label='O(N^2) - Direct Summation')
ax2.set_title("Computational Scaling (Log-Log Scale)")
ax2.set_xlabel("Number of Bodies (N)")
ax2.set_ylabel("Number of Operations (Log)")
ax2.grid(True, which="both", linestyle='--', alpha=0.6) # 'both' to show minor grid lines
ax2.legend()

# 3. Final Layout Adjustments
plt.tight_layout()
plt.show()
```

MARKDOWN #13

```
## The 1-Body Problem: Motion in a Vacuum
In the absence of external forces (like gravity from other masses), a single body follows
**Newton's First Law of Motion**:
"An object at rest stays at rest and an object in motion stays in motion with the same speed
and in the same direction unless acted upon by an unbalanced force."
```

Mathematical Representation:
The position $\mathbf{r}$ at time t can be described by:
$$\mathbf{r}(t) = \mathbf{r}_0 + \mathbf{v}_0 t$$
Where:
- $\mathbf{r}_0$ is the initial position.
- $\mathbf{v}_0$ is the initial velocity.
- Acceleration $\mathbf{a} = 0$, because there are no other bodies to exert gravitational force.

CODE #14

```
# 1. Define initial state (x, y)
initial_pos = np.array([0.0, 0.0])
initial_vel = np.array([1.0, 0.5])

# 2. Simulation parameters
dt = 0.1
total_steps = 50
# Create a time array (shape: 50,)
time_points = np.linspace(0, total_steps * dt, total_steps)

# 3. Calculate displacement for each time point
# Velocity (2, 1) * Time (1, 50) -> Displacement (2, 50)
displacement = initial_vel[:, np.newaxis] * time_points[np.newaxis, :]

# 4. Calculate trajectory (Initial position + Displacement)
# Transpose to get (50, 2) shape, which is easier for plotting
trajectory = (initial_pos[:, np.newaxis] + displacement).T

# 5. Visualization
plt.figure(figsize=(8, 5))
plt.plot(trajectory[:, 0], trajectory[:, 1], 'b--o', label='Path', markersize=4)
plt.scatter(trajectory[0, 0], trajectory[0, 1], color='green', s=100, label='Start')
plt.scatter(trajectory[-1, 0], trajectory[-1, 1], color='red', s=100, label='End')

plt.title("1-Body Problem: Constant Velocity (Fixed)")
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.legend()
plt.grid(True, linestyle=':')
plt.axis('equal')
plt.show()
```

MARKDOWN #15

Adding a Constant Force (The Gravity Concept)
To prepare for the N-body problem, we must introduce **acceleration**. Even with one body, we can simulate a "Central Force" (like a fixed sun that doesn't move) to see how the body reacts.

According to **Newton's Second Law**:
$$\mathbf{F} = m\mathbf{a} \implies \mathbf{a} = \frac{\mathbf{F}}{m}$$

In the next step, we will introduce a second body to see how the complexity jumps from a straight line to an orbital curve.

MARKDOWN #16

```markdown
## The 2-Body Problem: Orbital Mechanics
When two masses $m_1$ and $m_2$ interact, they exert equal and opposite gravitational forces on each other. According to Newton's law:

$$\mathbf{F}_{12} = G \frac{m_1 m_2}{r^2} \hat{\mathbf{r}}_{12}$$

### Equations of Motion:
For each body $i$, the acceleration $\mathbf{a}_i$ is:
$$\mathbf{a}_1 = \frac{\mathbf{F}_{21}}{m_1} = G \frac{m_2}{r^3} (\mathbf{r}_2 - \mathbf{r}_1)$$
$$\mathbf{a}_2 = \frac{\mathbf{F}_{12}}{m_2} = G \frac{m_1}{r^3} (\mathbf{r}_1 - \mathbf{r}_2)$$

Unlike the 1-body problem, the acceleration is **not zero** and changes at every time step because the distance $r$ changes. We must use a numerical integrator (like Velocity Verlet) to solve these coupled differential equations.
```

CODE #17

```python
# --- 1. Physics Engine Functions ---
def get_2body_accelerations(pos1, pos2, m1, m2, G=1.0, softening=0.1):
    """Calculates gravitational acceleration for two bodies."""
    r_vec = pos2 - pos1
    r_mag = np.sqrt(np.sum(r_vec**2) + softening**2)

    acc1 = G * m2 * r_vec / r_mag**3
    acc2 = -G * m1 * r_vec / r_mag**3
    return acc1, acc2

def simulate_2body(pos1, pos2, vel1, vel2, m1, m2, dt, steps):
    """Integrates motion using the Velocity Verlet algorithm."""
    hist1 = np.zeros((steps, 2))
    hist2 = np.zeros((steps, 2))

    a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)

    for i in range(steps):
        vel1 += a1 * dt / 2.0
        vel2 += a2 * dt / 2.0
        pos1 += vel1 * dt
        pos2 += vel2 * dt
        a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
        vel1 += a1 * dt / 2.0
        vel2 += a2 * dt / 2.0

        hist1[i] = pos1.copy()
        hist2[i] = pos2.copy()

    return hist1, hist2

# --- 2. Simulation Setup ---
m1, m2 = 100.0, 1.0  # Solar mass vs. Planet mass
p1 = np.array([0.0, 0.0])
p2 = np.array([10.0, 0.0])
v1 = np.array([0.0, 0.0])
v2 = np.array([0.0, 3.2]) # Circular orbit velocity
```

```
dt = 0.05
steps = 1000

# Run simulation to generate 'path1' and 'path2'
path1, path2 = simulate_2body(p1, p2, v1, v2, m1, m2, dt, steps)

# --- 3. Enhanced Visualization ---
plt.style.use('dark_background')
plt.figure(figsize=(10, 10))
ax = plt.gca()

# Plot Star with Glow Effect
ax.scatter(path1[-1, 0], path1[-1, 1], s=2500, color='orange', alpha=0.05) # Outer halo
ax.scatter(path1[-1, 0], path1[-1, 1], s=1000, color='yellow', alpha=0.1)  # Inner halo
ax.scatter(path1[-1, 0], path1[-1, 1], s=150, color='white', edgecolors='gold',
           linewidth=2, label=f'Star (m={m1})', zorder=5)

# Plot Orbital Path (History)
ax.plot(path2[:, 0], path2[:, 1], color='deepskyblue', linestyle='-', linewidth=1,
        alpha=0.5, label='Orbital Trajectory')

# Plot Current Planet Position
ax.scatter(path2[-1, 0], path2[-1, 1], color='cyan', s=60, edgecolors='white',
           label=f'Planet (m={m2})', zorder=6)

# Aesthetics
ax.set_title("2-Body Simulation: Stable Orbit with Glow Effect", fontsize=14, pad=20)
ax.set_aspect('equal')
ax.grid(True, linestyle=':', alpha=0.15)
ax.legend(frameon=False, loc='upper right')

plt.show()
```

MARKDOWN #18

```
## Orbital Classifications in the 2-Body Problem
The shape of an orbit depends on the **Total Mechanical Energy** ($E = KE + PE$):

1. **Circular Orbit ($E < 0$, $e=0$):** Perfect balance between gravity and velocity.
2. **Elliptical Orbit ($E < 0$, $0 < e < 1$):** Closed path, typical for planets.
3. **Parabolic Orbit ($E = 0$, $e = 1$):** The escape velocity; the body just barely
escapes.
4. **Hyperbolic Orbit ($E > 0$, $e > 1$):** High-speed flyby; the body escapes and never
returns.
```

CODE #19

```
# --- 1. Required Physics Functions ---
def get_2body_accelerations(pos1, pos2, m1, m2, G=1.0, softening=0.1):
    r_vec = pos2 - pos1
    r_mag = np.sqrt(np.sum(r_vec**2) + softening**2)
    acc1 = G * m2 * r_vec / r_mag**3
    acc2 = -G * m1 * r_vec / r_mag**3
    return acc1, acc2

def simulate_2body(pos1, pos2, vel1, vel2, m1, m2, dt, steps):
    hist1, hist2 = np.zeros((steps, 2)), np.zeros((steps, 2))
```

```
    a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
    for i in range(steps):
        vel1 += a1 * dt / 2.0; vel2 += a2 * dt / 2.0
        pos1 += vel1 * dt; pos2 += vel2 * dt
        a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
        vel1 += a1 * dt / 2.0; vel2 += a2 * dt / 2.0
        hist1[i], hist2[i] = pos1.copy(), pos2.copy()
    return hist1, hist2

# --- 2. Configuration & Initial Conditions ---
m1, m2 = 100.0, 1.0
pos1_init = np.array([0.0, 0.0])
pos2_init = np.array([10.0, 0.0])
vel1_init = np.array([0.0, 0.0])

velocities = {
    "Sub-circular (Fall)": 2.0,
    "Circular": 3.16,
    "Elliptical": 4.0,
    "Escape (Parabolic)": 4.47,
    "Hyperbolic": 5.5
}

# --- 3. Visualization ---
plt.style.use('dark_background')
plt.figure(figsize=(12, 10))
ax = plt.gca()

# Plot Star with Glow
ax.scatter(0, 0, s=3000, color='orange', alpha=0.03)
ax.scatter(0, 0, s=1000, color='yellow', alpha=0.07)
ax.scatter(0, 0, s=180, color='white', edgecolors='gold', label='Central Star', zorder=10)

# Colors for different orbits
colors = ['#ff4b4b', '#4bff4b', '#4b96ff', '#e14bff', '#ffff4b']

for i, (label, v_y) in enumerate(velocities.items()):
    p1, p2 = pos1_init.copy(), pos2_init.copy()
    v1, v2 = vel1_init.copy(), np.array([0.0, v_y])

    # Run simulation
    _, path2 = simulate_2body(p1, p2, v1, v2, m1, m2, dt=0.05, steps=1200)

    # Plot trajectory with a subtle shadow/glow
    ax.plot(path2[:, 0], path2[:, 1], color=colors[i], lw=2.5, alpha=0.9,
            label=f"{label} (v={v_y})", zorder=5-i)

    # Highlight the current position of the body
    ax.scatter(path2[-1, 0], path2[-1, 1], color=colors[i], s=60, edgecolors='white',
zorder=6)

# Aesthetics and Polish
ax.set_title("Comparative Orbital Mechanics: Velocity vs. Trajectory", fontsize=16, pad=25)
ax.set_xlabel("X distance", fontsize=12, alpha=0.7)
ax.set_ylabel("Y distance", fontsize=12, alpha=0.7)

# Reference Axes
```

```
ax.axhline(0, color='white', lw=0.5, alpha=0.2)
ax.axvline(0, color='white', lw=0.5, alpha=0.2)

# Set limits and aspect
ax.set_xlim(-45, 25)
ax.set_ylim(-35, 35)
ax.set_aspect('equal')
ax.grid(True, linestyle=':', alpha=0.1)

# Custom Legend
ax.legend(loc='upper left', frameon=True, facecolor='#111111', edgecolor='#444444',
fontsize=10)

plt.tight_layout()
plt.show()
```

MARKDOWN #20

```
## Visualizing Kepler's Three Laws
Through our N-body simulation, we can empirically demonstrate Kepler's laws:

1.  **The Law of Orbits:** All planets move in elliptical orbits, with the sun at one focus.
2.  **The Law of Areas:** A line that connects a planet to the sun sweeps out equal areas in
equal times (Conservation of Angular Momentum).
3.  **The Law of Periods:** The square of the orbital period ($T$) of a planet is directly
proportional to the cube of the semi-major axis ($a$) of its orbit: $T^2 \propto a^3$.
```

MARKDOWN #21

```
<div class="alert alert-block alert-success">
        < b >Ø<ß¬   N o t e   o n   I n t e r a c t i v e   A n i m a t i o n s < / b >
    <br>
    Several visualizations in this project are <b>interactive animations</b> rendered via
<code>FuncAnimation</code> or <code>Plotly</code>.
    <br><br>
    Upon execution, these cells will display a media player interface. You can use the
<b>Play/Pause</b> buttons, seek bars, and speed controls to observe the dynamical evolution
of the N-body systems over time. If an animation does not appear immediately, please allow a
few seconds for the simulation frames to be processed and embedded.
</div>
```

CODE #22

```
# --- 1. Physics Engine ---
def get_2body_accelerations(pos1, pos2, m1, m2, G=1.0, softening=0.1):
    r_vec = pos2 - pos1
    r_mag = np.sqrt(np.sum(r_vec**2) + softening**2)
    acc1 = G * m2 * r_vec / r_mag**3
    acc2 = -G * m1 * r_vec / r_mag**3
    return acc1, acc2

def simulate_2body_step(pos1, pos2, vel1, vel2, m1, m2, dt):
    # Velocity Verlet integration step
    a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
    vel1 += a1 * dt / 2.0; vel2 += a2 * dt / 2.0
    pos1 += vel1 * dt; pos2 += vel2 * dt
    a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
    vel1 += a1 * dt / 2.0; vel2 += a2 * dt / 2.0
```

```
    return pos1, pos2, vel1, vel2

# --- 2. Setup Conditions ---
m_sun, m_planet = 500.0, 1.0
p_sun, p_planet = np.array([0.0, 0.0]), np.array([15.0, 0.0])
v_sun, v_planet = np.array([0.0, 0.0]), np.array([0.0, 3.8])
dt = 0.04

# --- 3. Animation Setup ---
plt.style.use('dark_background')
fig, ax = plt.subplots(figsize=(10, 10))
ax.set_aspect('equal')
ax.set_xlim(-20, 20)
ax.set_ylim(-20, 20)

# Visual elements
orbit_line, = ax.plot([], [], color='cyan', lw=1, alpha=0.3)
planet_dot, = ax.plot([], [], 'o', color='skyblue', markersize=10, label='Planet')
sun_dot = ax.scatter(0, 0, s=200, color='orange', edgecolors='white', label='Sun')
velocity_arrow = ax.quiver(0, 0, 0, 0, color='red', scale=35, width=0.005, label='Velocity
Vector')

# Data containers
x_hist, y_hist = [], []

def init():
    orbit_line.set_data([], [])
    planet_dot.set_data([], [])
    return orbit_line, planet_dot, velocity_arrow

def update(frame):
    global p_sun, p_planet, v_sun, v_planet

    # Run 5 physics steps per frame for smoother motion
    for _ in range(5):
        p_sun, p_planet, v_sun, v_planet = simulate_2body_step(p_sun, p_planet, v_sun,
v_planet, m_sun, m_planet, dt)

    x_hist.append(p_planet[0])
    y_hist.append(p_planet[1])

    # Update orbit path
    orbit_line.set_data(x_hist, y_hist)

    # Update planet position
    planet_dot.set_data([p_planet[0]], [p_planet[1]])

    # Update velocity vector visualization
    velocity_arrow.set_offsets(p_planet)
    velocity_arrow.set_UVC(v_planet[0], v_planet[1])

    return orbit_line, planet_dot, velocity_arrow

# Create Animation
ani = FuncAnimation(fig, update, frames=200, init_func=init, blit=True, interval=30)
plt.close()
```

```
# Display as HTML5 Video
HTML(ani.to_jshtml())
```

CODE #23

```
# --- 1. Physics Engine (2-Body Problem) ---
def get_2body_accelerations(pos1, pos2, m1, m2, G=1.0, softening=0.1):
    r_vec = pos2 - pos1
    r_mag = np.sqrt(np.sum(r_vec**2) + softening**2)
    # Both bodies exert force on each other (Newton's 3rd Law)
    acc1 = G * m2 * r_vec / r_mag**3
    acc2 = -G * m1 * r_vec / r_mag**3
    return acc1, acc2

def simulate_2body_barycentric(pos1, pos2, vel1, vel2, m1, m2, dt, steps):
    hist1 = np.zeros((steps, 2))
    hist2 = np.zeros((steps, 2))

    # Initial accelerations
    a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)

    for i in range(steps):
        # Velocity Verlet Integration
        vel1 += a1 * dt / 2.0
        vel2 += a2 * dt / 2.0
        pos1 += vel1 * dt
        pos2 += vel2 * dt
        a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
        vel1 += a1 * dt / 2.0
        vel2 += a2 * dt / 2.0

        hist1[i] = pos1.copy()
        hist2[i] = pos2.copy()
    return hist1, hist2

# --- 2. Setup: Comparable Masses (Binary System) ---
m1, m2 = 300.0, 100.0  # Mass ratio 3:1 to see both ellipses clearly
G_CONST = 1.0

# Initial positions (placed relative to Barycenter at 0,0)
# r1*m1 = r2*m2
pos1 = np.array([-5.0, 0.0])
pos2 = np.array([15.0, 0.0])

# Initial velocities for elliptical orbits
vel1 = np.array([0.0, -1.2])
vel2 = np.array([0.0, 3.6])

dt, steps = 0.02, 4000
path1, path2 = simulate_2body_barycentric(pos1, pos2, vel1, vel2, m1, m2, dt, steps)

# --- 3. Visualization ---
plt.style.use('dark_background')
fig, ax = plt.subplots(figsize=(12, 12))

# Plot full orbital paths
ax.plot(path1[:, 0], path1[:, 1], color='orange', lw=1, alpha=0.4, label='Star Path
```

```
(m=300)')
ax.plot(path2[:, 0], path2[:, 1], color='cyan', lw=1, alpha=0.4, label='Planet Path
(m=100)')

# Barycenter marker
ax.scatter(0, 0, marker='+', color='white', s=100, label='Barycenter (Focus)', alpha=0.5)

# --- Kepler's 2nd Law: Equal Area Slices ---
# We use a larger interval and "slower" step skip to make it visual
interval = 400
for i in range(0, steps - interval, interval * 2):
    # Slice for Body 1 (Star)
    seg1 = path1[i : i + interval]
    verts1 = np.vstack([ [0, 0], seg1, [0, 0] ])
    ax.add_patch(Polygon(verts1, facecolor='orange', alpha=0.2, edgecolor='gold'))

    # Slice for Body 2 (Planet)
    seg2 = path2[i : i + interval]
    verts2 = np.vstack([ [0, 0], seg2, [0, 0] ])
    ax.add_patch(Polygon(verts2, facecolor='cyan', alpha=0.15, edgecolor='deepskyblue'))

# Final positions (dots)
ax.scatter(path1[-1, 0], path1[-1, 1], color='orange', s=150, edgecolors='white', zorder=5)
ax.scatter(path2[-1, 0], path2[-1, 1], color='cyan', s=80, edgecolors='white', zorder=5)

# --- Styling ---
ax.set_title("Binary System: Two Ellipses around a Common Center\nKepler's 2nd Law (Equal
Areas)", fontsize=16, pad=20)
ax.set_xlabel("X distance", color='gray')
ax.set_ylabel("Y distance", color='gray')
ax.grid(True, linestyle=':', alpha=0.2)
ax.set_aspect('equal')
ax.legend(frameon=False)

# Clean spines
for spine in ax.spines.values():
    spine.set_visible(False)

plt.tight_layout()
plt.show()
```

MARKDOWN #24

```
## Kepler's Third Law: The Harmonic Law
The square of the orbital period ($T$) is proportional to the cube of the semi-major axis
($a$):
$$\frac{T_1^2}{a_1^3} = \frac{T_2^2}{a_2^3} = \text{constant}$$

In this simulation, we compare two planets:
- **Planet A** at distance $r=10$
- **Planet B** at distance $r=20$
We calculate their orbital periods and verify if the ratio remains constant.
```

CODE #25

```
# --- 1. Physics & Helper Functions ---
def get_2body_accelerations(pos1, pos2, m1, m2, G=1.0, softening=0.1):
    r_vec = pos2 - pos1
```

```
    r_mag = np.sqrt(np.sum(r_vec**2) + softening**2)
    acc1 = G * m2 * r_vec / r_mag**3
    acc2 = -G * m1 * r_vec / r_mag**3
    return acc1, acc2

def simulate_2body(pos1, pos2, vel1, vel2, m1, m2, dt, steps):
    hist1, hist2 = np.zeros((steps, 2)), np.zeros((steps, 2))
    a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
    for i in range(steps):
        vel1 += a1 * dt / 2.0; vel2 += a2 * dt / 2.0
        pos1 += vel1 * dt; pos2 += vel2 * dt
        a1, a2 = get_2body_accelerations(pos1, pos2, m1, m2)
        vel1 += a1 * dt / 2.0; vel2 += a2 * dt / 2.0
        hist1[i], hist2[i] = pos1.copy(), pos2.copy()
    return hist1, hist2

def estimate_period(trajectory, dt):
    """Finds the orbital period by detecting when the body returns to its starting angular
position."""
    for i in range(200, len(trajectory)):
        # Check if the planet has crossed the positive x-axis again
        if trajectory[i, 1] * trajectory[i-1, 1] < 0 and trajectory[i, 0] > 0:
            return i * dt
    return None

def get_circular_velocity(r, M, G=1.0):
    return np.sqrt(G * M / r)

# --- 2. Simulation Setup ---
M_sun, G_const, dt, max_steps = 1000.0, 1.0, 0.05, 12000

# Planet A (Inner)
r_a = 10.0
v_a = get_circular_velocity(r_a, M_sun)
_, path_a = simulate_2body(np.zeros(2), np.array([r_a, 0.0]), np.zeros(2), np.array([0.0,
v_a]), M_sun, 1.0, dt, max_steps)

# Planet B (Outer)
r_b = 20.0
v_b = get_circular_velocity(r_b, M_sun)
_, path_b = simulate_2body(np.zeros(2), np.array([r_b, 0.0]), np.zeros(2), np.array([0.0,
v_b]), M_sun, 1.0, dt, max_steps)

# Calculate Periods and Ratios
T_a, T_b = estimate_period(path_a, dt), estimate_period(path_b, dt)
ratio_a, ratio_b = (T_a**2)/(r_a**3), (T_b**2)/(r_b**3)

# --- 3. Refined Visualization ---
plt.style.use('dark_background')
fig, ax = plt.subplots(figsize=(12, 12))

# Plot the Sun with Glow
ax.scatter(0, 0, s=4000, color='orange', alpha=0.03)
ax.scatter(0, 0, s=1200, color='yellow', alpha=0.08)
ax.scatter(0, 0, s=200, color='white', edgecolors='gold', label='Sun (Massive Center)',
zorder=10)
```

```
# Plot Orbital Paths (Dashed)
ax.plot(path_a[:, 0], path_a[:, 1], 'deepskyblue', lw=1, ls='--', alpha=0.4)
ax.plot(path_b[:, 0], path_b[:, 1], 'tomato', lw=1, ls='--', alpha=0.4)

# Synchronized State: Where are the planets when A finishes 1 orbit?
idx_sync = int(T_a / dt)
ax.scatter(path_a[idx_sync, 0], path_a[idx_sync, 1], color='cyan', s=100,
edgecolors='white', label=f'Planet A (T={T_a:.1f}s)', zorder=11)
ax.scatter(path_b[idx_sync, 0], path_b[idx_sync, 1], color='red', s=100, edgecolors='white',
label=f'Planet B (at t=T_a)', zorder=11)

# Annotate Synchronization
ax.annotate('Planet A completes 1 full orbit', xy=(path_a[idx_sync, 0], path_a[idx_sync,
1]),
            xytext=(20, 30), textcoords='offset points', color='cyan',
arrowprops=dict(arrowstyle='->', color='cyan'))
ax.annotate('Planet B is still far behind', xy=(path_b[idx_sync, 0], path_b[idx_sync, 1]),
            xytext=(20, -40), textcoords='offset points', color='tomato',
arrowprops=dict(arrowstyle='->', color='tomato'))

# Verification Text Box
results_text = (f"Kepler's 3rd Law Verification:\n"
                f"-----------------------------\n"
                f"Planet A Ratio (T²/r³): {ratio_a:.6f}\n"
                f"Planet B Ratio (T²/r³): {ratio_b:.6f}\n"
                f"Difference: {abs(ratio_a - ratio_b):.6f}")
ax.text(0.05, 0.05, results_text, transform=ax.transAxes, color='white',
        bbox=dict(facecolor='black', edgecolor='gray', alpha=0.7, boxstyle='round,pad=1'))

# Coordinate System & Grid
ax.axhline(0, color='white', lw=0.5, alpha=0.2)
ax.axvline(0, color='white', lw=0.5, alpha=0.2)
ax.grid(True, linestyle=':', alpha=0.1)
ax.set_title("Kepler's 3rd Law: The Harmonic Relationship ($T^2 \propto r^3$)", fontsize=16,
pad=20)
ax.set_aspect('equal')
ax.set_xlim(-25, 25); ax.set_ylim(-25, 25)
ax.legend(loc='upper right', frameon=False)

plt.tight_layout()
plt.show()
```

MARKDOWN #26

```
## The 3-Body Problem in One Dimension
In a 1D universe, particles are constrained to move along a single axis ($x$).
The gravitational force between two bodies $i$ and $j$ is still governed by the
inverse-square law, but the vector direction is simplified to a sign:
$$F_{ij} = G \frac{m_i m_j}{(x_j - x_i)^2} \cdot \text{sgn}(x_j - x_i)$$

**Numerical Challenge:** In 1D, particles will inevitably "collide" (pass through $x_j - x_i
= 0$). To prevent infinite forces and numerical collapse, the **softening parameter**
($\epsilon$) is crucial:
$$a_i = \sum_{j \neq i} G \frac{m_j \cdot (x_j - x_i)}{((x_j - x_i)^2 + \epsilon^2)^{1.5}}$$
```

MARKDOWN #27

```
<div class="alert alert-block alert-info">
        < b >Ø<ß    M u l t i - D i m e n s i o n a l   A p p r o a c h :   F r o m   1 D   t o   3 D
    <br>
    While most <b>N-body simulations</b> are traditionally performed in <b>2D</b> or
<b>3D</b> space to mirror physical reality, this project takes a broader mathematical
approach.
    <br><br>
    In addition to standard spatial models, we include <b>1D (One-Dimensional)
simulations</b>. These constrained environments allow for a clearer analysis of
gravitational interactions, "collisions," and numerical softening without the complexity of
orbital planes. This progression is designed to demonstrate how the fundamental laws of
motion scale across different dimensional constraints.
</div>
```

CODE #28

```
def get_1d_accelerations(pos, masses, G=1.0, softening=0.1):
    """
    Calculate 1D accelerations for N bodies.
    pos: array of shape (N,)
    masses: array of shape (N,)
    """
    n = len(pos)
    acc = np.zeros(n)
    for i in range(n):
        for j in range(n):
            if i == j: continue
            dx = pos[j] - pos[i]
            inv_r3 = (dx**2 + softening**2)**(-1.5)
            acc[i] += G * masses[j] * dx * inv_r3
    return acc

def simulate_1d_3body(pos, vel, masses, dt, steps, G=1.0, softening=0.1):
    """1D Velocity Verlet integrator."""
    history = np.zeros((steps, len(pos)))
    curr_pos = pos.copy()
    curr_vel = vel.copy()

    acc = get_1d_accelerations(curr_pos, masses, G, softening)

    for i in range(steps):
        # Velocity Verlet: First half-step velocity
        curr_vel += acc * dt / 2.0
        # Position update
        curr_pos += curr_vel * dt
        # Re-calculate acceleration
        acc = get_1d_accelerations(curr_pos, masses, G, softening)
        # Second half-step velocity
        curr_vel += acc * dt / 2.0
        history[i] = curr_pos

    return history

# --- Simulation Execution ---

# Initial Setup: 3 bodies on a line
```

```
pos_init = np.array([-5.0, 0.0, 5.0])
vel_init = np.array([0.5, 0.0, -0.5])
masses = np.array([10.0, 20.0, 10.0])

dt = 0.01
steps = 2000

# Run Simulation
history_1d = simulate_1d_3body(pos_init, vel_init, masses, dt, steps, softening=0.5)

# --- Visualization: White Background ---

# Set overall style to white
plt.style.use('default')

fig, ax = plt.subplots(figsize=(12, 6))
# Set background color to pure white
fig.patch.set_facecolor('white')
ax.set_facecolor('white')

time_axis = np.linspace(0, steps*dt, steps)
colors = ['red', 'green', 'blue']

for i in range(3):
    ax.plot(time_axis, history_1d[:, i], color=colors[i],
            label=f'Body {i} (m={masses[i]})', linewidth=2)

ax.set_title("1D 3-Body Simulation: Space-Time Trajectories", fontsize=14)
ax.set_xlabel("Time (s)", fontsize=12)
ax.set_ylabel("Position (X)", fontsize=12)

# Grid and Legend on white background
ax.grid(True, linestyle='--', alpha=0.6, color='gray')
ax.legend(facecolor='white', framealpha=1)

plt.show()
```

MARKDOWN #29

```
# The Mathematical Complexity of the 3-Body Problem in 2D

The **3-Body Problem** is a classic challenge in celestial mechanics. While the motion of
two bodies is perfectly predictable (Keplerian orbits), adding a third body introduces
**deterministic chaos**. This means that while the laws governing the motion are simple, the
long-term behavior of the system is impossible to predict with a general formula.
```

MARKDOWN #30

```
## 1. Defining the System
Consider three point masses $m_1, m_2, m_3$ in a 2D plane. Their positions are represented
by vectors $\mathbf{r}_i = (x_i, y_i)$.

The interaction is governed by **Newton's Law of Universal Gravitation**. For each body $i$,
the total force $\mathbf{F}_i$ is the vector sum of the gravitational pulls from the other
two bodies:

$$\mathbf{F}_i = \sum_{j \neq i} G \frac{m_j m_i}{|\mathbf{r}_j - \mathbf{r}_i|^3}
(\mathbf{r}_j - \mathbf{r}_i)$$
```

According to **Newton's Second Law** ($\mathbf{F} = m\mathbf{a}$), the acceleration of body i is:
$$\frac{d^2\mathbf{r}_i}{dt^2} = \sum_{j \neq i} G \frac{m_j}{|\mathbf{r}_j - \mathbf{r}_i|^3} (\mathbf{r}_j - \mathbf{r}_i)$$

MARKDOWN #31

2. The Coupled Equations of Motion
In a 2D environment, this leads to a system of **6 coupled second-order ordinary differential equations (ODEs)**. For example, the equations for the first body (m_1) are:

$$\ddot{x}_1 = G m_2 \frac{x_2 - x_1}{r_{12}^3} + G m_3 \frac{x_3 - x_1}{r_{13}^3}$$
$$\ddot{y}_1 = G m_2 \frac{y_2 - y_1}{r_{12}^3} + G m_3 \frac{y_3 - y_1}{r_{13}^3}$$

Where r_{ij} is the Euclidean distance:
$$r_{ij} = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

Because each equation depends on the positions of the other two bodies, the system is **non-linear** and highly interdependent.

MARKDOWN #32

3. The Lack of Analytical Solutions
In 1887, **Henri Poincaré** proved a groundbreaking theorem: for $N > 2$, there are no general closed-form analytical solutions (integrals of motion) that can describe the system's state using standard algebraic functions.

Key Obstacles:
* **Integrals of Motion:** We only have 10 classical constants (Energy, Linear Momentum, Angular Momentum, and Center of Mass motion).
* **Degrees of Freedom:** A 2D 3-body system has 12 degrees of freedom (position and velocity for each body in x and y). We don't have enough constraints to "solve" the equations algebraically.
* **Sensitivity to Initial Conditions:** Small changes in the initial positions or velocities lead to radically different outcomes over time, a phenomenon known as the **Butterfly Effect**.

MARKDOWN #33

4. Stability and Chaos
Most 3-body configurations are unstable. Eventually, one of two things usually happens:
1. One body is ejected from the system at high speed (Gravity Assist/Slingshot).
2. The remaining two bodies form a stable binary pair.

However, specific periodic solutions exist, such as the famous **"Figure-8" orbit**, where three equal masses follow each other along a single path in a stable, repeating pattern.

MARKDOWN #34

The Extraordinary "Figure-Eight" Orbit
While most 3-body systems are chaotic, a few periodic solutions exist. The **Figure-Eight** is a remarkable stable orbit where three equal masses ($m_1 = m_2 = m_3$) follow each other along a single figure-eight-shaped path.

To achieve this, the initial conditions must be extremely precise. Even a 0.1% error in velocity will cause the system to diverge into chaos over time, demonstrating the sensitivity of the N-body problem.

CODE #35

```
def compute_accelerations_clean(pos, mass, G, softening=0.0):
    """
    Calculates pairwise acceleration without RuntimeWarnings.
    Uses masking to avoid division by zero on the diagonal.
    """
    # 1. Relative position vectors (N, N, 3)
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]

    # 2. Distance squared (N, N)
    dist_sq = np.sum(diff**2, axis=-1) + softening**2

    # 3. Mask for off-diagonal elements
    mask = ~np.eye(pos.shape[0], dtype=bool)

    # 4. Calculate 1/r^3 only where distance > 0
    dist_inv3 = np.zeros_like(dist_sq)
    dist_inv3[mask] = dist_sq[mask]**(-1.5)

    # 5. Final acceleration (N, 3)
    acc = G * np.sum(diff * dist_inv3[:, :, np.newaxis] * mass.T[:, :, np.newaxis], axis=1)
    return acc

def run_stable_simulation(pos, vel, mass, steps, dt, G, softening=0.0):
    """Velocity Verlet Integrator for periodic orbits."""
    history = np.zeros((steps, pos.shape[0], pos.shape[1]))
    acc = compute_accelerations_clean(pos, mass, G, softening)

    for i in range(steps):
        vel += acc * dt / 2.0
        pos += vel * dt
        acc = compute_accelerations_clean(pos, mass, G, softening)
        vel += acc * dt / 2.0
        history[i] = pos.copy()
    return history

# --- Initial Conditions for Figure-Eight ---
G_const = 1.0
m_3body = np.array([[1.0], [1.0], [1.0]])

# Precise values for stability
p1 = np.array([0.97000436, -0.24308753, 0.0])
v1 = np.array([0.466203685, 0.43236573, 0.0])

# Position and Velocity arrays
pos_init = np.array([p1, -p1, [0.0, 0.0, 0.0]])
vel_init = np.array([v1, v1, -2*v1])

# Simulation Setup
dt_8 = 0.005
steps_8 = 5000

# Execute
traj_8 = run_stable_simulation(pos_init.copy(), vel_init.copy(), m_3body, steps_8, dt_8,
G_const)
```

```
# --- Visualization ---
plt.figure(figsize=(10, 7))
colors = ['crimson', 'forestgreen', 'royalblue']

for i in range(3):
    plt.plot(traj_8[:, i, 0], traj_8[:, i, 1], color=colors[i], lw=1.5, alpha=0.8)
    # Current position dot
    plt.scatter(traj_8[-1, i, 0], traj_8[-1, i, 1], color=colors[i], s=100,
edgecolors='black', label=f'Body {i+1}')

plt.title("Periodic 3-Body Solution: The Figure-Eight Orbit", fontsize=14)
plt.xlabel("X Position", fontsize=12)
plt.ylabel("Y Position", fontsize=12)
plt.legend(loc='upper right')
plt.axis('equal')
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```

MARKDOWN #36

```
## The Butterfly Effect in the 3-Body Problem
A system is **chaotic** if a microscopic change in the initial conditions leads to a
macroscopic difference in the outcome over time.

In this simulation, we run two "Figure-8" scenarios simultaneously:
1. **Reference System:** Uses the precise stable coordinates.
2. **Perturbed System:** One body's X-position is shifted by only $10^{-4}$ (0.0001).

We will visualize the "Error Distance" between the same body in both systems to see how the
divergence grows exponentially.
```

CODE #37

```
# 1. Base Setup (Reuse the clean functions from the previous commit)
G_const = 1.0
mass_3b = np.ones((3, 1))
dt = 0.005
steps = 6000

# Precise Initial Conditions
p1 = np.array([0.97000436, -0.24308753, 0.0])
v1 = np.array([0.466203685, 0.43236573, 0.0])

pos_ref = np.array([p1.copy(), -p1.copy(), [0.0, 0.0, 0.0]])
vel_ref = np.array([v1.copy(), v1.copy(), -2*v1.copy()])

# 2. Perturbed System (Microscopic change)
perturbation = 0.0001
pos_per = pos_ref.copy()
pos_per[0, 0] += perturbation  # Shift Body 1's X-position slightly
vel_per = vel_ref.copy()

# 3. Run both simulations
traj_ref = run_stable_simulation(pos_ref, vel_ref, mass_3b, steps, dt, G_const)
traj_per = run_stable_simulation(pos_per, vel_per, mass_3b, steps, dt, G_const)

# 4. Calculate Euclidean Error for Body 1 over time
error = np.sqrt(np.sum((traj_ref[:, 0, :] - traj_per[:, 0, :])**2, axis=1))
```

```
# --- Visualization ---
plt.figure(figsize=(12, 5))

# Plot 1: Trajectory Comparison (Final states)
plt.subplot(1, 2, 1)
plt.plot(traj_ref[:, 0, 0], traj_ref[:, 0, 1], 'b-', alpha=0.5, label='Reference (Stable)')
plt.plot(traj_per[:, 0, 0], traj_per[:, 0, 1], 'r--', alpha=0.5, label='Perturbed (Chaos)')
plt.title("Trajectories of Body 1")
plt.legend()
plt.axis('equal')

# Plot 2: Exponential Growth of Error
plt.subplot(1, 2, 2)
plt.semilogy(np.linspace(0, steps*dt, steps), error, color='darkred')
plt.title("Growth of Error (Log Scale)")
plt.xlabel("Time")
plt.ylabel("Distance between identical bodies")
plt.grid(True, which="both", ls="-", alpha=0.2)

plt.tight_layout()
plt.show()
```

MARKDOWN #38

```
## Beyond the Figure-Eight: Hierarchical vs. Chaotic Systems
In 2D space, 3-body systems generally fall into two categories:

1. **Hierarchical Systems:** Two bodies form a tight binary pair, and the third body orbits
them from a much larger distance. This configuration is relatively stable and common in
nature (e.g., a star, a planet, and a moon).
2. **Chaotic Systems:** All three bodies are at comparable distances. Their mutual
gravitational pulls lead to a "complex dance" that usually ends with one body being ejected
from the system at high speed.
```

CODE #39

```
# --- 1. Physics Engine (Vectorized) ---
def compute_accelerations_3d(pos, mass, G, softening=0.01):
    """Pairwise gravitational acceleration."""
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
    dist_sq = np.sum(diff**2, axis=-1) + softening**2
    dist_inv3 = dist_sq**(-1.5)
    np.fill_diagonal(dist_inv3, 0)
    acc = G * np.sum(diff * dist_inv3[:, :, np.newaxis] * mass.T[:, :, np.newaxis], axis=1)
    return acc

def run_stable_simulation(pos, vel, mass, steps, dt, G):
    """Velocity Verlet integrator."""
    # Correcting history shape initialization
    history = np.zeros((steps, pos.shape[0], pos.shape[1]))
    acc = compute_accelerations_3d(pos, mass, G)
    for i in range(steps):
        vel += acc * dt / 2.0
        pos += vel * dt
        acc = compute_accelerations_3d(pos, mass, G)
        vel += acc * dt / 2.0
        history[i] = pos.copy()
```

```
    return history

# --- 2. Hierarchical Setup ---
m_hier = np.array([[1000.0], [10.0], [0.1]]) # Sun, Planet, Moon
pos_hier = np.array([
    [0.0, 0.0, 0.0],    # Sun
    [18.0, 0.0, 0.0],   # Planet
    [19.5, 0.0, 0.0]    # Moon
])
vel_hier = np.array([
    [0.0, 0.0, 0.0],
    [0.0, 7.5, 0.0],    # Planet Velocity
    [0.0, 10.2, 0.0]    # Moon Velocity
])

# Execute Simulation
# Decreasing dt for higher precision in the moon's orbit
traj_hier = run_stable_simulation(pos_hier, vel_hier, m_hier, steps=12000, dt=0.005, G=1.0)

# --- 3. Refined Visualization ---
plt.style.use('dark_background')
fig, ax = plt.subplots(figsize=(12, 10))

# Plot Sun with Glow
ax.scatter(0, 0, s=4000, color='orange', alpha=0.05)
ax.scatter(0, 0, s=1000, color='yellow', alpha=0.1)
ax.scatter(0, 0, s=200, color='white', marker='*', edgecolors='gold', label='Sun (Focus)',
zorder=10)

# Plot Orbits
ax.plot(traj_hier[:, 1, 0], traj_hier[:, 1, 1], color='dodgerblue', lw=1.5, alpha=0.8,
label='Planet Orbit')
ax.plot(traj_hier[:, 2, 0], traj_hier[:, 2, 1], color='crimson', lw=0.6, alpha=0.6,
label='Moon Trajectory')

# Current positions
ax.scatter(traj_hier[-1, 1, 0], traj_hier[-1, 1, 1], color='dodgerblue', s=80,
edgecolors='white', zorder=11)
ax.scatter(traj_hier[-1, 2, 0], traj_hier[-1, 2, 1], color='white', s=20, label='Moon
Current Pos', zorder=12)

# --- Zoom-in Inset ---
# Zoom factor and location (upper right)
ax_ins = zoomed_inset_axes(ax, zoom=3.0, loc='upper right', borderpad=3)

ax_ins.plot(traj_hier[:, 1, 0], traj_hier[:, 1, 1], color='dodgerblue', lw=2)
ax_ins.plot(traj_hier[:, 2, 0], traj_hier[:, 2, 1], color='crimson', lw=1, alpha=0.8)
ax_ins.scatter(traj_hier[-1, 1, 0], traj_hier[-1, 1, 1], color='dodgerblue', s=150)
ax_ins.scatter(traj_hier[-1, 2, 0], traj_hier[-1, 2, 1], color='white', s=40)

# Zoom focus area
x_last, y_last = traj_hier[-1, 1, 0], traj_hier[-1, 1, 1]
ax_ins.set_xlim(x_last - 3.5, x_last + 3.5)
ax_ins.set_ylim(y_last - 3.5, y_last + 3.5)
ax_ins.set_xticks([]); ax_ins.set_yticks([])
mark_inset(ax, ax_ins, loc1=2, loc2=4, fc="none", ec="white", alpha=0.2)
```

```
# Legend at Bottom Left
ax.legend(loc='lower left', frameon=True, facecolor='#111111', edgecolor='gray',
fontsize=10)

# Aesthetics
ax.set_title("Hierarchical 3-Body System: Moon orbiting Planet orbiting Sun", fontsize=14,
pad=20)
ax.set_aspect('equal')
ax.grid(True, linestyle=':', alpha=0.1)
ax.set_xlabel("X distance [AU]", color='gray')
ax.set_ylabel("Y distance [AU]", color='gray')

plt.show()
```

MARKDOWN #40

```
### Mathematical Explanation: Chaotic 3-Body System Setup

This code block initializes a **3-Body Problem** in 2D/3D space, specifically designed to
demonstrate **chaotic behavior**. Unlike the 2-body problem, the 3-body problem has no
general closed-form analytical solution and is highly sensitive to initial conditions.

#### 1. Mass Matrix ($m$)
The code defines a column vector for masses:
$$m = \begin{bmatrix} 10.0 \\ 10.0 \\ 10.0 \end{bmatrix}$$
All three bodies are assigned equal mass, which often leads to complex gravitational
"dances" and exchanges of energy.

#### 2. Phase Space Initialization
The system's state is defined by the initial positions ($\mathbf{r}_i$) and velocities
($\mathbf{v}_i$):

*   **Initial Positions ($\mathbf{R}$):** The bodies are placed at specific coordinates in
the XY plane:
    $$\mathbf{r}_1 = (1, 1, 0), \quad \mathbf{r}_2 = (-1, -1, 0), \quad \mathbf{r}_3 = (0,
0.5, 0)$$
*   **Initial Velocities ($\mathbf{V}$):** Small velocity vectors are assigned to trigger
interaction:
    $$\mathbf{v}_1 = (0.2, -0.1, 0), \quad \mathbf{v}_2 = (-0.2, 0.1, 0), \quad \mathbf{v}_3
= (0.1, 0.3, 0)$$

#### 3. Gravitational Dynamics
The simulation relies on **Newton's Law of Universal Gravitation**. The acceleration
$\mathbf{a}_i$ for each body $i$ is calculated by summing the forces exerted by the other
bodies $j$:
$$\mathbf{a}_i = G \sum_{j \neq i} \frac{m_j (\mathbf{r}_j - \mathbf{r}_i)}{(|\mathbf{r}_j -
\mathbf{r}_i|^2 + \epsilon^2)^{3/2}}$$
*   **$G=1.0$:** Gravitational constant (normalized).
*   **$\epsilon=0.1$ (Softening):** A small constant added to prevent the force from
becoming infinite during close encounters (singularities).

#### 4. Trajectory Plotting
The code iterates through time steps ($6000$ steps with $dt=0.02$) and plots the resulting
trajectories:
*   **Lines:** Show the history of motion (pathways).
*   **Points (Scatter):** Indicate the final position of each body at the end of the
simulation.
```

MARKDOWN #41

```markdown
# The Butterfly Effect in N-Body Systems

The **3-Body Problem** is a quintessential example of a **deterministic chaotic system**. Although the laws of motion (Newton's Law of Universal Gravitation) are simple and fixed, the long-term behavior of the system is unpredictable.

### Key characteristics observed in these simulations:

*   **Sensitivity to Initial Conditions:** Even a microscopic change in the initial velocity (e.g., $\delta v \approx 10^{-4}$) leads to a completely different trajectory after a few orbital periods. This is often referred to as the **Lyapunov time**, the timescale over which the system's state becomes uncomputable.
*   **Ejection Events:** In most chaotic scenarios, one body eventually gains enough kinetic energy from a close gravitational encounter to be "slingshot" out of the system ($E_{total} > 0$ for that body), leaving behind a stable binary pair.
*   **Irreversibility:** Due to numerical rounding errors in floating-point math and the accumulation of errors in the integrator (e.g., Velocity Verlet), simulating the system backward in time rarely returns to the exact starting point. This mirrors the physical reality of **entropy** in complex dynamical systems.
```

CODE #42

```
# 1. Base Physical Constants and Initial Conditions
G = 1.0
m_chaos = np.ones((3, 1)) * 10.0
pos_init = np.array([
    [1.0, 1.0, 0.0],
    [-1.0, -1.0, 0.0],
    [0.0, 0.5, 0.0]
])

# Base velocity vector (from your previous setup)
vel_base = np.array([
    [0.2, -0.1, 0.0],
    [-0.2, 0.1, 0.0],
    [0.4, 0.5, 0.0]
])

# 2. Setup Visualization Grid (2 rows x 3 columns)
plt.style.use('default')
fig, axes = plt.subplots(2, 3, figsize=(18, 11), dpi=100)
fig.patch.set_facecolor('white')
axes = axes.flatten()

# High noise level: 20% variation to trigger immediate divergence
noise_level = 0.20
colors = ['magenta', 'cyan', 'lime']
np.random.seed(42) # For reproducibility

# 3. Iterative Simulations with Significant Perturbations
for i in range(5):
    ax = axes[i]

    # Apply significant random noise to the initial velocity
    # Each simulation will have a unique 'random_push'
    random_push = 1 + np.random.uniform(-noise_level, noise_level, vel_base.shape)
```

```
    vel_perturbed = vel_base * random_push

    # Run the simulation (Using your Velocity Verlet engine)
    # Reducing dt for better stability during close encounters with high noise
    traj = run_stable_simulation(pos_init.copy(), vel_perturbed, m_chaos,
                                 steps=6000, dt=0.015, G=G)

    # Plot Trajectories
    for j in range(3):
        # Main path with transparency
        ax.plot(traj[:, j, 0], traj[:, j, 1], color=colors[j],
                alpha=0.6, lw=1.2, label=f'Body {j+1}')
        # Final position marker
        ax.scatter(traj[-1, j, 0], traj[-1, j, 1], color=colors[j],
                   s=60, edgecolors='black', zorder=5)

    # Subplot Styling
    ax.set_title(f"Variant #{i+1}\nNoise: ±{noise_level*100:.0f}%",
                 fontsize=12, fontweight='bold', pad=10)
    ax.set_facecolor('#FAFAFA') # Slightly grey background for contrast
    ax.grid(True, linestyle=':', alpha=0.4)
    ax.set_aspect('equal')

    # Dynamic limit adjustment to keep all bodies in view
    limit = np.max(np.abs(traj[:, :, :2])) * 1.1
    ax.set_xlim(-limit, limit)
    ax.set_ylim(-limit, limit)

# 4. Final Formatting and Information Box
axes[5].axis('off')
info_text = (
    "CHAOS ANALYSIS:\n"
    "-------------------\n"
    "Significant velocity noise\n"
    "triggers immediate phase\n"
    "space divergence.\n\n"
    "Notice how Body 2 (Cyan)\n"
    "is ejected in some plots\n"
    "but captured in others."
)
axes[5].text(0.5, 0.5, info_text, ha='center', va='center',
             fontsize=13, color='#333333', bbox=dict(facecolor='white', alpha=0.5))

plt.suptitle("High-Perturbation Sensitivity: The 3-Body Butterfly Effect",
             fontsize=20, fontweight='bold', y=0.98)
plt.tight_layout()
plt.show()
```

MARKDOWN #43

```
# The 3-Body Problem in 3D Space

Moving from a 2D plane to **3D space** increases the complexity of the system by adding a
third degree of freedom for each body. In 3D, each mass $m_i$ has a position vector
$\mathbf{r}_i = (x_i, y_i, z_i)$ and a velocity vector $\mathbf{v}_i = (v_{xi}, v_{yi},
v_{zi})$.
```

The Vector Formulation
The gravitational force exerted on body i by body j in 3D is defined as:
$$\mathbf{F}_{ij} = -G \frac{m_i m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

Where:
- $\mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j$ is the relative position vector.
- $|\mathbf{r}_{ij}| = \sqrt{(x_i-x_j)^2 + (y_i-y_j)^2 + (z_i-z_j)^2}$ is the Euclidean distance in 3D.

MARKDOWN #44

1. Governing Equations
For a system of 3 bodies, we must solve **9 coupled second-order ordinary differential equations (ODEs)** simultaneously (3 equations for each body: x, y, and z components).

For body m_1, the equations of motion are:
$$\ddot{x}_1 = -G m_2 \frac{x_1 - x_2}{r_{12}^3} - G m_3 \frac{x_1 - x_3}{r_{13}^3}$$
$$\ddot{y}_1 = -G m_2 \frac{y_1 - y_2}{r_{12}^3} - G m_3 \frac{y_1 - y_3}{r_{13}^3}$$
$$\ddot{z}_1 = -G m_2 \frac{z_1 - z_2}{r_{12}^3} - G m_3 \frac{z_1 - z_3}{r_{13}^3}$$

Similar equations apply to m_2 and m_3. This system lacks a general analytical solution, requiring numerical integration methods like **Velocity Verlet**.

MARKDOWN #45

2. Constraints and Constants
Despite the chaos, the system still obeys fundamental conservation laws:
1. **Conservation of Energy (E):** The sum of Kinetic (T) and Potential (V) energy remains constant.
   $$E = \sum \frac{1}{2} m_i v_i^2 - \sum_{i < j} \frac{G m_i m_j}{r_{ij}}$$
2. **Conservation of Linear Momentum ($\mathbf{P}$):** The center of mass of the system moves at a constant velocity.
3. **Conservation of Angular Momentum ($\mathbf{L}$):** Unlike 2D (where L is a scalar), in 3D $\mathbf{L}$ is a **3D vector** $\mathbf{L} = \sum m_i (\mathbf{r}_i \times \mathbf{v}_i)$, representing the plane of rotation of the system.

MARKDOWN #46

3. The "Softening" Technique
In 3D simulations, close encounters between bodies can cause the distance r_{ij} to approach zero, leading to infinite forces and numerical "explosions" (particles flying away at unrealistic speeds).
To prevent this, we use a **softening parameter** ϵ:
$$|\mathbf{r}_{ij}|_{soft} = \sqrt{(x_i-x_j)^2 + (y_i-y_j)^2 + (z_i-z_j)^2 + \epsilon^2}$$
This ensures the denominator never becomes zero and stabilizes the integration.

MARKDOWN #47

    < b >Ø=Ý±þ I n t e r a c t i v e 3 D E n v i r o n m e n t s < / b >

    This project includes Interactive 3D Visualizations powered by <code>Plotly</code>. Unlike static plots, these environments allow for deep exploration of chaotic trajectories:
    <ul>
        <li>Rotate: Click and drag your mouse to view the 3-body system from any angle.</li>
        <li>Zoom: Use the scroll wheel to get closer to orbital encounters.</li>

```
        <li><b>Inspect Data:</b> Hover your cursor over the paths or bodies to see real-time
<b>X, Y, Z coordinates</b> and particle labels.</li>
        <li><b>Pan:</b> Right-click and drag to move the camera across the 3D space.</li>
    </ul>
    If the interactive 3D plot does not load, ensure that <code>ipywidgets</code> is
installed and your browser supports WebGL.
</div>
```

CODE #48

```
def compute_accelerations_3d(pos, mass, G, softening=0.1):
    """3D Vectorized acceleration with softening to prevent singularities."""
    # diff shape: (N, N, 3)
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]

    # distance squared: (N, N)
    dist_sq = np.sum(diff**2, axis=-1) + softening**2
    dist_inv3 = dist_sq**(-1.5)

    # Mask diagonal to avoid self-interaction
    np.fill_diagonal(dist_inv3, 0)

    # Acceleration: (N, 3)
    acc = G * np.sum(diff * dist_inv3[:, :, np.newaxis] * mass.T[:, :, np.newaxis], axis=1)
    return acc

def simulate_3d_3body(pos, vel, mass, steps, dt, G, softening=0.1):
    """3D Velocity Verlet Integrator."""
    history = np.zeros((steps, 3, 3))
    acc = compute_accelerations_3d(pos, mass, G, softening)

    for i in range(steps):
        vel += acc * dt / 2.0
        pos += vel * dt
        acc = compute_accelerations_3d(pos, mass, G, softening)
        vel += acc * dt / 2.0
        history[i] = pos.copy()
    return history

# --- 3D Chaotic Initialization ---
m_3d = np.array([[10.0], [10.0], [10.0]])
p_3d = np.array([
    [1.0, 0.0, 0.0],
    [-1.0, 0.0, 0.0],
    [0.0, 1.0, 1.0] # Placed in 3D space
])
v_3d = np.array([
    [0.0, 0.5, 0.1],
    [0.0, -0.5, -0.1],
    [0.2, 0.0, 0.0]
])

# Run for 2000 steps
traj_3d = simulate_3d_3body(p_3d, v_3d, m_3d, steps=2000, dt=0.02, G=1.0)

# Create the Plotly Figure
fig = go.Figure()
```

```
colors = ['magenta', 'cyan', 'lime']
names = ['Body 1', 'Body 2', 'Body 3']

for i in range(3):
    # Add trajectory lines
    fig.add_trace(go.Scatter3d(
        x=traj_3d[:, i, 0], y=traj_3d[:, i, 1], z=traj_3d[:, i, 2],
        mode='lines',
        line=dict(color=colors[i], width=3),
        name=f'{names[i]} Path'
    ))
    # Add current position (the sphere)
    fig.add_trace(go.Scatter3d(
        x=[traj_3d[-1, i, 0]], y=[traj_3d[-1, i, 1]], z=[traj_3d[-1, i, 2]],
        mode='markers',
        marker=dict(color=colors[i], size=8, symbol='circle'),
        name=f'Current {names[i]}'
    ))

# Layout settings
fig.update_layout(
    title="Interactive 3D Chaotic 3-Body Simulation",
    scene=dict(
        xaxis_title='X',
        yaxis_title='Y',
        zaxis_title='Z',
        bgcolor="black"
    ),
    width=900,
    height=700,
    template="plotly_dark"
)

fig.show()
```

MARKDOWN #49

```
## 3D Orbital Geometry
In a three-dimensional system, the concept of an "orbit" becomes more fluid. Unlike the 2nd law of Kepler, which implies motion in a fixed plane, 3D interactions often lead to **orbital plane precession**.

### Key Observations:
1. **Out-of-Plane Motion:** Bodies can gain velocity in the $Z$-axis through gravitational assists (slingshots), causing their orbital plane to tilt or rotate.
2. **Close Encounters:** In 3D, bodies rarely "collide" mathematically ($r \to 0$), as they usually have a small offset in the third dimension. However, the $1/r^2$ force still causes extreme velocity changes.
3. **Complex Envelopes:** The resulting trajectories often form a "cloud" or a "torus" rather than a simple ellipse, showing how the gravitational potential field changes dynamically.
```

CODE #50

```
# Use the traj_3d data from your previous simulation
fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')
```

```
colors = ['crimson', 'limegreen', 'dodgerblue']

# Get the limits of the axes to place shadows on the "walls"
x_lim = ax.get_xlim()
y_lim = ax.get_ylim()
z_lim = ax.get_zlim()

for i in range(3):
    x = traj_3d[:, i, 0]
    y = traj_3d[:, i, 1]
    z = traj_3d[:, i, 2]

    # 1. Main 3D Trajectory
    ax.plot(x, y, z, color=colors[i], lw=2, label=f'Body {i+1}', zorder=10)

    # 2. Floor Projection (XY plane at the bottom of Z)
    # We use z_lim[0] to pick the minimum Z value
    ax.plot(x, y, np.full_like(z, z_lim[0]), color=colors[i], alpha=0.2, lw=1)

    # 3. Wall Projection (XZ plane at the back of Y)
    # We use y_lim[1] to pick the maximum Y value
    ax.plot(x, np.full_like(y, y_lim[1]), z, color=colors[i], alpha=0.2, lw=1)

# Formatting
plt.style.use('default')
ax.set_title("3D Orbital Paths with Fixed Plane Projections", fontsize=14)
ax.set_xlabel("X Axis")
ax.set_ylabel("Y Axis")
ax.set_zlabel("Z Axis")

# Set consistent limits to keep shadows on the "walls"
ax.set_zlim(z_lim)
ax.set_ylim(y_lim)

ax.view_init(elev=20, azim=30)
ax.legend()
plt.show()
```

CODE #51

```
# 1. Increase the embedding limit to 50MB
mpl.rcParams['animation.embed_limit'] = 50.0

# --- Physics Engine (Vectorized 3D) ---
def compute_accelerations_3d(pos, mass, G, softening=0.1):
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
    dist_sq = np.sum(diff**2, axis=-1) + softening**2
    dist_inv3 = dist_sq**(-1.5)
    np.fill_diagonal(dist_inv3, 0)
    acc = G * np.sum(diff * dist_inv3[:, :, np.newaxis] * mass.T[:, :, np.newaxis], axis=1)
    return acc

# --- Initial Conditions (3D Chaotic) ---
m_3d = np.array([[10.0], [10.0], [10.0]])
pos_3d = np.array([[1.0, 0.0, 0.0], [-1.0, 0.0, 0.0], [0.0, 1.0, 1.0]])
vel_3d = np.array([[0.0, 0.5, 0.1], [0.0, -0.5, -0.1], [0.2, 0.0, 0.0]])
```

```
G_const, dt = 1.0, 0.015 # Slightly larger dt for faster visual evolution

# --- Animation Setup ---
plt.style.use('default')
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

colors = ['crimson', 'limegreen', 'dodgerblue']
# Pre-create plot elements
lines = [ax.plot([], [], [], color=colors[i], lw=1.5, alpha=0.7)[0] for i in range(3)]
dots = [ax.plot([], [], [], 'o', color=colors[i], markersize=8)[0] for i in range(3)]
floor_projections = [ax.plot([], [], [], color=colors[i], alpha=0.15, lw=1)[0] for i in
range(3)]
wall_projections = [ax.plot([], [], [], color=colors[i], alpha=0.15, lw=1)[0] for i in
range(3)]

# Fixed limits for the 3D space
limit = 4.0
ax.set_xlim(-limit, limit); ax.set_ylim(-limit, limit); ax.set_zlim(-limit, limit)
z_floor = -limit
y_wall = limit

ax.set_title("3D Chaotic N-Body Animation", fontsize=14)
ax.view_init(elev=20, azim=45)

# Trajectory buffers
x_hist = [[] for _ in range(3)]
y_hist = [[] for _ in range(3)]
z_hist = [[] for _ in range(3)]

def update(frame):
    global pos_3d, vel_3d

    # Run 6 physics sub-steps per frame to make it faster
    for _ in range(6):
        acc = compute_accelerations_3d(pos_3d, m_3d, G_const)
        vel_3d += acc * dt / 2.0
        pos_3d += vel_3d * dt
        acc = compute_accelerations_3d(pos_3d, m_3d, G_const)
        vel_3d += acc * dt / 2.0

    for i in range(3):
        x_hist[i].append(pos_3d[i, 0])
        y_hist[i].append(pos_3d[i, 1])
        z_hist[i].append(pos_3d[i, 2])

        # Performance optimization: show only the last 150 points for the tail
        # This reduces data size significantly
        tail = -150
        curr_x = x_hist[i][tail:]
        curr_y = y_hist[i][tail:]
        curr_z = z_hist[i][tail:]

        lines[i].set_data(curr_x, curr_y)
        lines[i].set_3d_properties(curr_z)
```

```
        dots[i].set_data([pos_3d[i, 0]], [pos_3d[i, 1]])
        dots[i].set_3d_properties([pos_3d[i, 2]])

        # Floor Shadows (XY plane)
        floor_projections[i].set_data(curr_x, curr_y)
        floor_projections[i].set_3d_properties(np.full_like(curr_z, z_floor))

        # Wall Shadows (XZ plane)
        wall_projections[i].set_data(curr_x, np.full_like(curr_y, y_wall))
        wall_projections[i].set_3d_properties(curr_z)

    return lines + dots + floor_projections + wall_projections

# Create animation with fewer frames (200) to keep size manageable
ani = FuncAnimation(fig, update, frames=50, interval=40, blit=True)
plt.close()

HTML(ani.to_jshtml())
```

MARKDOWN #52

```
## Achieving Stability: Hierarchical Triple Systems
To prevent the immediate chaotic ejection of a body, we can arrange the system
hierarchically.
This configuration treats the inner binary pair as a single gravitational source for the
distant third body.

**Mathematical Condition for Stability:**
The ratio of the semi-major axis of the outer orbit ($R$) to the inner orbit ($r$) must be
sufficiently large ($R/r > 3$ typically).
This minimizes the tidal forces that cause chaotic disruptions.
```

CODE #53

```
# 1. Setup Hierarchical System
# m1 & m2: Inner binary pair (like two stars)
# m3: Outer planet or third star
m_stable = np.array([[50.0], [50.0], [1.0]])
G_const = 1.0

# Initial Positions
# m1 and m2 are close (-1, 0) and (1, 0)
# m3 is far away (0, 15)
pos_st = np.array([
    [-1.0, 0.0, 0.0],
    [1.0, 0.0, 0.0],
    [0.0, 15.0, 0.0]
])

# Initial Velocities
# v1 and v2: High velocity to orbit each other
# v3: Velocity calculated for a wide orbit around the pair
v_inner = 4.0
v_outer = 2.6
vel_st = np.array([
    [0.0, -v_inner, 0.0],
    [0.0, v_inner, 0.0],
    [v_outer, 0.0, 0.0]
```

```
])

# 2. Simulation Parameters
dt_st = 0.01
steps_st = 10000 # Long simulation to prove stability

# Use the 3D engine
traj_st = simulate_3d_3body(pos_st, vel_st, m_stable, steps_st, dt_st, G_const,
softening=0.1)
```

CODE #54

```
# Set dark background for a space aesthetic
plt.style.use('dark_background')

# 1. Setup Figure
plt.figure(figsize=(12, 12))
ax = plt.gca()

# 2. Define Colors and Labels
# Using bright neon-like colors for better contrast on black
colors = ['#FFD700', '#FF4500', '#00BFFF'] # Gold, OrangeRed, DeepSkyBlue
labels = ['Star A (Mass: 50)', 'Star B (Mass: 50)', 'Outer Planet (Mass: 1)']

# 3. Plot Trajectories
for i in range(3):
    # Plot the line with a slight alpha for a "glow" effect
    plt.plot(traj_st[:, i, 0], traj_st[:, i, 1],
             color=colors[i], lw=1.2, alpha=0.8, label=labels[i])

    # Mark the final position with a larger glowing point
    plt.scatter(traj_st[-1, i, 0], traj_st[-1, i, 1],
                color=colors[i], s=80, edgecolors='white', linewidth=0.5, zorder=5)

    # Optional: Mark the starting position with a small dot
    plt.scatter(traj_st[0, i, 0], traj_st[0, i, 1],
                color=colors[i], s=20, marker='x', alpha=0.5)

# 4. Styling the Plot
plt.title("Stable Hierarchical 3-Body System\nStatic Orbital Path Visualization",
          fontsize=16, pad=20, color='white')
plt.xlabel("X Coordinate", fontsize=12, alpha=0.7)
plt.ylabel("Y Coordinate", fontsize=12, alpha=0.7)

# Ensure the orbits aren't distorted
plt.axis('equal')

# Add a subtle grid
plt.grid(True, linestyle='--', alpha=0.15)

# Legend with transparent background
plt.legend(loc='upper right', frameon=True, facecolor='black', edgecolor='gray')

# Remove top and right spines for a cleaner look
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
```

```
plt.tight_layout()
plt.show()
```

MARKDOWN #55

```
# Mathematical Foundation: The Direct Summation Method

The **Direct Summation** (or Brute Force) approach is the most accurate way to simulate
$N$-body systems. It calculates the exact gravitational interaction between every possible
pair of particles in the system.

### 1. Newton's Gravitational Law
For any two particles $i$ and $j$ with masses $m_i$ and $m_j$, the gravitational force
$\mathbf{F}_{ij}$ exerted on particle $i$ by particle $j$ is:
$$\mathbf{F}_{ij} = G \frac{m_i m_j}{|\mathbf{r}_{ij}|^2} \hat{\mathbf{r}}_{ij} = G
\frac{m_i m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

Where:
- $G$ is the Gravitational Constant.
- $\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i$ is the relative position vector.
- $|\mathbf{r}_{ij}|$ is the Euclidean distance between the particles.
```

MARKDOWN #56

```
### 2. Superposition Principle
According to the principle of superposition, the total force acting on a particle $i$ is the
vector sum of all forces exerted by all other $N-1$ particles:
$$\mathbf{F}_i = \sum_{j=1, j \neq i}^{N} \mathbf{F}_{ij}$$

Using Newton's Second Law ($\mathbf{F} = m\mathbf{a}$), we derive the **acceleration
vector** $\mathbf{a}_i$:
$$\mathbf{a}_i = \frac{\mathbf{F}_i}{m_i} = \sum_{j=1, j \neq i}^{N} G
\frac{m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

### 3. The "Softening" Term
In numerical simulations, if two particles get extremely close ($|\mathbf{r}_{ij}| \to 0$),
the force approaches infinity, causing numerical instability (particles "exploding" away).
To fix this, we introduce a **softening parameter** $\epsilon$:
$$\mathbf{a}_i \approx \sum_{j=1, j \neq i}^{N} G \frac{m_j}{(|\mathbf{r}_{ij}|^2 +
\epsilon^2)^{3/2}} \mathbf{r}_{ij}$$
```

MARKDOWN #57

```
### 4. Algorithmic Complexity: $O(N^2)$
In each time step, we must calculate the interaction for every particle $i$ with every other
particle $j$.
- There are $N$ particles.
- Each particle requires $N-1$ force calculations.
- Total interactions: $N(N-1) \approx N^2$.

This quadratic growth means that doubling the number of particles ($2N$) results in **four
times** the computational work ($4N^2$). This is why Direct Summation is limited to a few
thousand bodies on standard hardware.
```

CODE #58

```
# Define range of N (number of bodies)
n_range = np.logspace(1, 4, 50) # From 10 to 10,000
```

```
complexity = n_range**2

plt.style.use('default')
plt.figure(figsize=(10, 6))
plt.loglog(n_range, complexity, 'b-', label='O(N^2) Operations')
plt.title("Computational Scaling of Direct Summation")
plt.xlabel("Number of Particles (N)")
plt.ylabel("Number of Calculations (N^2)")
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.fill_between(n_range, complexity, color='blue', alpha=0.1)
plt.legend()
plt.show()
```

MARKDOWN #59

```
## Direct Summation in 1D Space
In a one-dimensional universe, particles are constrained to a single axis ($x$). The
gravitational acceleration $a_i$ for a particle $i$ is the sum of the pulls from all other
$N-1$ particles.

### The Governing Equation:
For $N$ particles, the acceleration of particle $i$ is:
$$a_i = \sum_{j=1, j \neq i}^{N} G \frac{m_j \cdot (x_j - x_i)}{((x_j - x_i)^2 +
\epsilon^2)^{3/2}}$$

Where:
- $(x_j - x_i)$ determines the direction of the force (left or right).
- $\epsilon$ (softening) prevents the force from reaching infinity when particles "pass
through" each other.
- The complexity is $O(N^2)$ because we must compute $N \times (N-1)$ interactions.
```

CODE #60

```
# 1. Configuration
N_BODIES = 10
G = 1.0
SOFTENING = 0.2
DT = 0.01
STEPS = 1000

# 2. Random Initialization
np.random.seed(42)
# Positions along the X axis
pos = np.random.uniform(-5, 5, N_BODIES)
# Initial velocities (starting near rest)
vel = np.random.uniform(-0.1, 0.1, N_BODIES)
# Uniform masses
mass = np.ones(N_BODIES)

print(f"Initialized {N_BODIES} particles in 1D space.")
```

CODE #61

```
def get_1d_accelerations(pos, mass, G, softening):
    """
    Computes acceleration for each particle using Direct Summation O(N^2).
    """
    n = len(pos)
```

```
    acc = np.zeros(n)

    # Pairwise interaction loop
    for i in range(n):
        for j in range(n):
            if i == j: continue

            dx = pos[j] - pos[i]
            # 1D gravity formula with softening
            inv_r3 = (dx**2 + softening**2)**(-1.5)
            acc[i] += G * mass[j] * dx * inv_r3

    return acc

def run_1d_simulation(pos, vel, mass, steps, dt, G, softening):
    """Integrates motion using Velocity Verlet."""
    history = np.zeros((steps, len(pos)))

    a = get_1d_accelerations(pos, mass, G, softening)

    for i in range(steps):
        # Velocity Verlet Step
        vel += a * dt / 2.0
        pos += vel * dt
        a = get_1d_accelerations(pos, mass, G, softening)
        vel += a * dt / 2.0

        history[i] = pos.copy()

    return history

# Execute Simulation
pos_history = run_1d_simulation(pos.copy(), vel.copy(), mass, STEPS, DT, G, SOFTENING)
```

CODE #62

```
plt.figure(figsize=(12, 7))
time_axis = np.linspace(0, STEPS * DT, STEPS)

for i in range(N_BODIES):
    plt.plot(time_axis, pos_history[:, i], label=f'Body {i}')

plt.title("1D N-Body Simulation (N=10) - Direct Summation")
plt.xlabel("Time")
plt.ylabel("Position (X)")
plt.grid(True, alpha=0.3)
# Move legend outside for clarity
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

MARKDOWN #63

```
## Direct Summation in 2D Space
In a two-dimensional simulation, the gravitational force is a **vector**. For a system of
$N$ particles, we must compute the interaction between every unique pair $(i, j)$.

### 1. Vector Physics
```

The acceleration vector $\mathbf{a}_i$ for particle i is calculated by summing the contributions from all other particles j:
$$\mathbf{a}_i = \sum_{j \neq i}^{N} G \frac{m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

Where the relative position vector is $\mathbf{r}_{ij} = (x_j - x_i, y_j - y_i)$ and the distance is $|\mathbf{r}_{ij}| = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$.

2. Numerical Stability (Softening)
To avoid infinite forces during close encounters, we modify the distance calculation:
$$|\mathbf{r}_{ij}|_{\epsilon} = (dx^2 + dy^2 + \epsilon^2)^{1/2}$$
This "softening" parameter ϵ is vital for N-body systems to prevent particles from being ejected at non-physical speeds.

CODE #64

```
def get_2d_accelerations(pos, mass, G, softening):
    """
    Computes 2D acceleration for N bodies using Direct Summation O(N^2).
    pos: (N, 2) array
    mass: (N, 1) array
    """
    n = pos.shape[0]
    acc = np.zeros_like(pos)

    for i in range(n):
        for j in range(n):
            if i == j: continue

            # Vector from i to j
            diff = pos[j] - pos[i]
            # Distance with softening
            dist_sq = np.sum(diff**2) + softening**2
            inv_dist3 = dist_sq**(-1.5)

            # Acceleration contribution
            acc[i] += G * mass[j] * diff * inv_dist3

    return acc

def run_2d_nbody(pos, vel, mass, steps, dt, G, softening):
    """N-body integration using Velocity Verlet."""
    history = np.zeros((steps, pos.shape[0], 2))

    a = get_2d_accelerations(pos, mass, G, softening)

    for i in range(steps):
        # Update velocities (half step)
        vel += a * dt / 2.0
        # Update positions (full step)
        pos += vel * dt
        # New acceleration
        a = get_2d_accelerations(pos, mass, G, softening)
        # Update velocities (final half step)
        vel += a * dt / 2.0

        history[i] = pos.copy()
```

```
    return history
```

CODE #65

```
# 1. Setup Parameters
N = 10
G_CONST = 1.0
SOFT = 0.1
DT_VAL = 0.01
TOTAL_STEPS = 1500

# 2. Random Initialization (Normal distribution)
np.random.seed(123)
pos_init = np.random.randn(N, 2) * 2.0
vel_init = np.random.randn(N, 2) * 0.5
mass_init = np.ones((N, 1))

# 3. Execute
trajectory_2d = run_2d_nbody(pos_init, vel_init, mass_init, TOTAL_STEPS, DT_VAL, G_CONST,
SOFT)

# 4. Visualization
plt.figure(figsize=(10, 10))
for i in range(N):
    plt.plot(trajectory_2d[:, i, 0], trajectory_2d[:, i, 1], lw=1, alpha=0.7)
    plt.scatter(trajectory_2d[-1, i, 0], trajectory_2d[-1, i, 1], s=40)

plt.title(f"2D N-Body Simulation (N={N}) - Direct Summation Paths")
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.grid(True, alpha=0.2)
plt.axis('equal')
plt.show()
```

MARKDOWN #66

```
## Direct Summation in 3D Space
The 3D N-body problem is the most realistic representation of stellar dynamics. Each
particle $i$ experiences a gravitational pull from all other $N-1$ particles in
three-dimensional space.

### 1. Vector Force in 3D
The acceleration $\mathbf{a}_i$ is a 3-component vector:
$$\mathbf{a}_i = \sum_{j \neq i}^{N} G \frac{m_j}{(|\mathbf{r}_{ij}|^2 + \epsilon^2)^{3/2}}
\mathbf{r}_{ij}$$

Where:
- $\mathbf{r}_{ij} = (x_j - x_i, y_j - y_i, z_j - z_i)$
- $|\mathbf{r}_{ij}|^2 = (x_j - x_i)^2 + (y_j - y_i)^2 + (z_j - z_i)^2$

### 2. Algorithmic Challenge
In 3D, for $N=10$ particles, we compute 90 interactions per step. For $N=1000$, this jumps
to 999,000 interactions. Direct summation is perfectly accurate (within numerical limits)
but becomes the primary **computational bottleneck** as the system scale increases.
```

CODE #67

```
def get_3d_accelerations(pos, mass, G, softening):
    """
    Computes 3D acceleration for N bodies using Direct Summation O(N^2).
    pos: (N, 3) array
    mass: (N, 1) array
    """
    n = pos.shape[0]
    acc = np.zeros_like(pos)

    for i in range(n):
        for j in range(n):
            if i == j: continue

            # Vector from i to j in 3D
            diff = pos[j] - pos[i]
            # Distance squared with softening
            dist_sq = np.sum(diff**2) + softening**2
            inv_dist3 = dist_sq**(-1.5)

            # Newton's law
            acc[i] += G * mass[j] * diff * inv_dist3

    return acc

def run_3d_nbody(pos, vel, mass, steps, dt, G, softening):
    """3D Velocity Verlet integrator."""
    history = np.zeros((steps, pos.shape[0], 3))

    a = get_3d_accelerations(pos, mass, G, softening)

    for i in range(steps):
        vel += a * dt / 2.0
        pos += vel * dt
        a = get_3d_accelerations(pos, mass, G, softening)
        vel += a * dt / 2.0
        history[i] = pos.copy()

    return history
```

CODE #68

```
# 1. Initialization
N = 10
G_CONST = 1.0
SOFT = 0.1
DT = 0.01
STEPS = 1000

np.random.seed(42)
# Random cluster in 3D
pos_3d = np.random.randn(N, 3) * 3.0
vel_3d = np.random.randn(N, 3) * 0.2
mass_3d = np.ones((N, 1))

# 2. Execute
traj_3d = run_3d_nbody(pos_3d, vel_3d, mass_3d, STEPS, DT, G_CONST, SOFT)
```

```
# 3. 3D Plotting
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

for i in range(N):
    ax.plot(traj_3d[:, i, 0], traj_3d[:, i, 1], traj_3d[:, i, 2], alpha=0.7, lw=1)
    ax.scatter(traj_3d[-1, i, 0], traj_3d[-1, i, 1], traj_3d[-1, i, 2], s=20)

ax.set_title(f"3D Direct Summation Simulation (N={N})")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
plt.show()
```

MARKDOWN #69

```
# Algorithmic Optimization: The Barnes-Hut Approximation

The **Barnes-Hut algorithm** is a hierarchical method that reduces the complexity of N-body
simulations from $O(N^2)$ to $O(N \log N)$. It relies on the observation that the
gravitational force from a distant cluster of particles can be approximated as the force
from a single **"super-particle"** located at the cluster's center of mass.

### 1. Spatial Partitioning (Quadtrees/Octrees)
The simulation space is recursively divided into square (2D) or cubic (3D) cells.
- In 2D, this creates a **Quadtree** (each node has 4 children).
- In 3D, this creates an **Octree** (each node has 8 children).

A cell is subdivided until it contains at most one particle.
```

MARKDOWN #70

```
### 2. Center of Mass (CoM) Calculations
For every internal node (cell) in the tree, we must pre-calculate the total mass ($M$) and
the position of the Center of Mass ($\mathbf{R}$).

If a cell contains particles $p_1, p_2, \dots, p_k$ with masses $m_i$ and positions
$\mathbf{r}_i$, then:

**Total Mass of the cell:**
$$M = \sum_{i=1}^{k} m_i$$

**Center of Mass position:**
$$\mathbf{R} = \frac{1}{M} \sum_{i=1}^{k} m_i \mathbf{r}_i$$

When calculating forces on a distant particle, we treat the entire cell as a single particle
with mass $M$ at position $\mathbf{R}$.
```

MARKDOWN #71

```
### 3. The Opening Angle ($\theta$)
To decide whether a cell is "distant enough" to use the approximation, we use the
**Multipole Acceptance Criterion (MAC)**.

Let $s$ be the width of the cell and $d$ be the distance from the particle to the cell's
Center of Mass. The approximation is used if:
$$\frac{s}{d} < \theta$$
```

Where θ is the **threshold (opening) angle**:
- If $\theta = 0$, the algorithm reverts to Direct Summation ($O(N^2)$).
- If θ is large, the simulation is faster but less accurate.
- A typical value is $\theta \approx 0.5$.

CODE #72

```
# Visualizing the theta criterion
def visualize_theta():
    fig, ax = plt.subplots(figsize=(8, 6))

    particle_pos = np.array([0, 0])
    cell_pos = np.array([10, 2])
    cell_size = 4

    # Draw particle
    ax.scatter(*particle_pos, color='red', s=100, label='Target Particle')

    # Draw cell
    rect = plt.Rectangle(cell_pos - cell_size/2, cell_size, cell_size,
                         fill=False, color='blue', lw=2, label='Distant Cell')
    ax.add_patch(rect)

    # Draw distance line (d)
    ax.plot([0, 10], [0, 2], 'g--', label='Distance (d)')

    # Add text for s and d
    ax.text(10, 4.2, f'Size (s) = {cell_size}', color='blue', fontweight='bold')
    ax.text(4, 0.5, 'Distance (d)', color='green')

    # Title showing the formula
    theta_val = cell_size / np.sqrt(10**2 + 2**2)
    ax.set_title(f"MAC: s/d = {theta_val:.2f} | If

    ax.set_xlim(-1, 15)
    ax.set_ylim(-2, 6)
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.show()

visualize_theta()
```

MARKDOWN #73

```
## Energy Conservation in Barnes-Hut Simulations
To verify the stability of our simulation, we track the **Total Mechanical Energy
($E_{tot}$)**. In an isolated system, this value should remain nearly constant.

The total energy is the sum of Kinetic Energy ($T$) and Gravitational Potential Energy
($V$):
$$E_{tot} = T + V$$

### 1. Kinetic Energy ($T$)
Kinetic energy is calculated per particle and does not require tree-based approximation:
$$T = \sum_{i=1}^{N} \frac{1}{2} m_i v_i^2$$
```

MARKDOWN #74

```markdown
### 2. Gravitational Potential Energy ($V$)
The potential energy is the sum of interactions between all pairs. In Barnes-Hut, we use the tree to approximate this, just as we do for forces.

For a particle $i$ interacting with a distant cell (node) $C$ with total mass $M_C$ and Center of Mass $\mathbf{R}_C$:
$$V_{iC} \approx -G \frac{m_i M_C}{|\mathbf{r}_i - \mathbf{R}_C|}$$

The total potential energy of the system is:
$$V_{total} = \frac{1}{2} \sum_{i=1}^{N} V_i$$
Where $V_i$ is the potential at position $\mathbf{r}_i$ calculated by traversing the tree and applying the **MAC ($\theta$)** criterion.

**Note:** The factor of $1/2$ prevents double-counting the energy of each pair $(i, j)$.
```

CODE #75

```
def calculate_tree_energy(particle, node, theta, G):
    """
    Recursive function to calculate potential energy for a particle using the tree.
    """
    # Calculate distance to node's Center of Mass
    r_vec = node.center_of_mass - particle.pos
    dist = np.sqrt(np.sum(r_vec**2))

    if node.is_leaf():
        # If it's a leaf, calculate exact PE (avoid self-interaction)
        if node.particle != particle:
            return -G * particle.mass * node.particle.mass / (dist + 1e-9)
        else:
            return 0

    # Check Multipole Acceptance Criterion (MAC)
    if (node.size / dist) < theta:
        # Approximate: treat whole cell as one mass
        return -G * particle.mass * node.total_mass / dist
    else:
        # Go deeper into the tree
        energy = 0
        for child in node.children:
            if child is not None:
                energy += calculate_tree_energy(particle, child, theta, G)
        return energy
```

CODE #76

```
# Reset to default light style
plt.style.use('default')

def visualize_tree_energy_logic(particle_pos, node, theta, ax):
    """
    Visualizes the Barnes-Hut recursive logic for energy calculation.
    Blue/Grey: Approximated nodes (efficient)
    Red: Opened nodes (recursive depth)
    """
    # Calculate distance to node's center of mass
```

```
    r_vec = node['com'] - particle_pos
    dist = np.linalg.norm(r_vec)

    # Calculate the MAC ratio
    mac_ratio = node['size'] / dist if dist > 0 else float('inf')

    # Draw the current node box
    is_approximated = mac_ratio < theta or node['is_leaf']

    box_color = 'lightgray' if is_approximated else 'red'
    box_alpha = 0.2 if is_approximated else 0.1
    edge_color = 'blue' if is_approximated else 'red'

    rect = patches.Rectangle(
        (node['x'] - node['size']/2, node['y'] - node['size']/2),
        node['size'], node['size'],
        linewidth=1.5, edgecolor=edge_color, facecolor=box_color, alpha=box_alpha
    )
    ax.add_patch(rect)

    if is_approximated:
        # Drawing a connection line to the center of mass
        line_color = 'darkblue' if not node['is_leaf'] else 'green'
        ax.plot([particle_pos[0], node['com'][0]], [particle_pos[1], node['com'][1]],
                color=line_color, alpha=0.3, lw=1, linestyle='--')
        ax.scatter(node['com'][0], node['com'][1], color=line_color, s=30, zorder=3)
    else:
        # Go deeper into children
        for child in node['children']:
            if child is not None:
                visualize_tree_energy_logic(particle_pos, child, theta, ax)

# --- Mock Tree Structure for Visualization ---
mock_tree = {
    'x': 0, 'y': 0, 'size': 20, 'com': np.array([2, 2]), 'is_leaf': False,
    'children': [
        {'x': -5, 'y': -5, 'size': 10, 'com': np.array([-4, -4]), 'is_leaf': True,
'children': []},
        {'x': 5, 'y': 5, 'size': 10, 'com': np.array([6, 6]), 'is_leaf': False, 'children':
[
            {'x': 2.5, 'y': 2.5, 'size': 5, 'com': np.array([3, 3]), 'is_leaf': True,
'children': []},
            {'x': 7.5, 'y': 7.5, 'size': 5, 'com': np.array([8, 8]), 'is_leaf': True,
'children': []},
        ]},
        {'x': -5, 'y': 5, 'size': 10, 'com': np.array([-6, 4]), 'is_leaf': True, 'children':
[]},
        {'x': 5, 'y': -5, 'size': 10, 'com': np.array([4, -6]), 'is_leaf': True, 'children':
[]},
    ]
}

# 1. Setup Figure
fig, ax = plt.subplots(figsize=(10, 10))
ax.set_facecolor('white')

target_particle = np.array([-8, -8])
```

```
theta_val = 0.5

# 2. Execute Visualization
visualize_tree_energy_logic(target_particle, mock_tree, theta_val, ax)

# 3. Styling
ax.scatter(target_particle[0], target_particle[1], color='darkmagenta', s=150,
           label='Target Particle', zorder=10, edgecolor='black')
ax.set_title(f"Tree Energy Calculation Logic (Theta = {theta_val})", fontsize=14)
ax.set_xlim(-12, 12)
ax.set_ylim(-12, 12)
ax.set_aspect('equal')
ax.grid(True, linestyle=':', alpha=0.6)
ax.legend(loc='upper right')

plt.show()
```

MARKDOWN #77

```
### 3. The Virial Theorem
A useful diagnostic for N-body systems is the **Virial Ratio**:
$$Q = \frac{2T}{|V|}$$
- If $Q = 1$, the system is in **Virial Equilibrium** (stable).
- If $Q > 1$, the system is expanding (too much kinetic energy).
- If $Q < 1$, the system is collapsing (gravity dominates).
```

MARKDOWN #78

```
## Visualizing the Barnes-Hut Decision Process
The algorithm decides whether to use a simplified "Super-particle" or to split the cell
based on the **Opening Angle ($\theta$)**.

- **Case A (Approximate):** The distance $d$ is large compared to cell size $s$. We use the
Center of Mass (CoM).
- **Case B (Split):** The distance $d$ is small. The cell is too close to be approximated,
so we look at its 4 children.
```

CODE #79

```
def visualize_bh_decision():
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 7))

    # --- Case A: Distant Cell (Approximation) ---
    ax1.set_title("Case A: s/d < ¸ (Approximate)",
    target = np.array([2, 5])
    cell_center = np.array([12, 5])
    s = 4
    d = 10

    # Draw Target and Cell
    ax1.scatter(*target, color='red', s=100, label='Target Particle')
    ax1.add_patch(patches.Rectangle(cell_center - s/2, s, s, fill=True, color='blue',
alpha=0.2))
    ax1.scatter(*cell_center, color='blue', s=150, marker='*', label='Center of Mass (CoM)')

    # Interaction Line
    ax1.annotate('', xy=cell_center, xytext=target, arrowprops=dict(arrowstyle='<->',
color='black', lw=2))
```

```
    ax1.text(6, 5.5, f"Distance d={d}", fontsize=12)
    ax1.text(11, 2.5, f"Size s={s}", color='blue', fontweight='bold')

    # --- Case B: Nearby Cell (Recursive Split) ---
         a x 2 . s e t _ t i t l e ( " C a s e   B :   s / d   >   ¸   ( R e c u r s i v e   S p l i t
    target_near = np.array([5, 5])
    cell_near = np.array([8, 5])
    s_near = 4

    ax2.scatter(*target_near, color='red', s=100)
    # Draw main cell
    ax2.add_patch(patches.Rectangle(cell_near - s_near/2, s_near, s_near, fill=False,
ls='--', color='gray'))

    # Draw Sub-cells (The split)
    half = s_near / 2
    sub_centers = [
        cell_near + [-half/2, -half/2], cell_near + [half/2, -half/2],
        cell_near + [-half/2, half/2], cell_near + [half/2, half/2]
    ]
    for sc in sub_centers:
        ax2.add_patch(patches.Rectangle(sc - half/2, half, half, fill=True, color='orange',
alpha=0.3))
        ax2.scatter(*sc, color='orange', s=30)
        ax2.annotate('', xy=sc, xytext=target_near, arrowprops=dict(arrowstyle='->',
color='orange', alpha=0.5))

    ax2.text(5.5, 2.5, "Cell is too close! \nChecking children...", color='darkred',
fontweight='bold')

    for ax in [ax1, ax2]:
        ax.set_xlim(0, 15)
        ax.set_ylim(0, 10)
        ax.set_aspect('equal')
        ax.grid(True, alpha=0.2)

    plt.tight_layout()
    plt.show()

visualize_bh_decision()
```

MARKDOWN #80

```
## The Quadtree Structure
A Quadtree is a tree data structure in which each internal node has exactly four children.
In the context of Barnes-Hut:
1. **Internal Nodes:** Store the aggregate **Total Mass** and **Center of Mass** of all
particles within their boundaries.
2. **External Nodes (Leafs):** Contain either a single particle or are empty.

### The Force Approximation Rule:
When calculating the force on particle $P$, we visit a node. If $s/d < \theta$:
- We treat the node as a single particle.
- $s$ = width of the node.
- $d$ = distance between $P$ and the node's Center of Mass.
```

CODE #81

```
class QuadNode:
    def __init__(self, x_mid, y_mid, size):
        self.x_mid = x_mid
        self.y_mid = y_mid
        self.size = size

        self.particle = None      # Stores a particle if it's a leaf
        self.children = None      # Becomes a list of 4 QuadNodes if split

        self.mass = 0.0
        self.com = np.array([0.0, 0.0]) # Center of Mass

    def insert(self, p_pos, p_mass):
        """Recursively inserts a particle into the quadtree."""
        # 1. If node is empty, put particle here
        if self.mass == 0:
            self.particle_pos = p_pos
            self.particle_mass = p_mass

        # 2. If it's already an internal node, update CoM and insert into child
        elif self.children is not None:
            self._update_com(p_pos, p_mass)
            self._insert_into_child(p_pos, p_mass)

        # 3. If it's a leaf with a particle, split it
        else:
            # Move existing particle to a child
            self.children = [
                QuadNode(self.x_mid - self.size/4, self.y_mid - self.size/4, self.size/2), #
SW
                QuadNode(self.x_mid + self.size/4, self.y_mid - self.size/4, self.size/2), #
SE
                QuadNode(self.x_mid - self.size/4, self.y_mid + self.size/4, self.size/2), #
NW
                QuadNode(self.x_mid + self.size/4, self.y_mid + self.size/4, self.size/2)  #
NE
            ]
            # Re-insert old particle
            self._insert_into_child(self.particle_pos, self.particle_mass)
            # Insert new particle
            self._update_com(p_pos, p_mass)
            self._insert_into_child(p_pos, p_mass)

        self.mass += p_mass

    def _update_com(self, p_pos, p_mass):
        """Updates the Center of Mass using weighted average."""
        self.com = (self.com * self.mass + p_pos * p_mass) / (self.mass + p_mass)

    def _insert_into_child(self, p_pos, p_mass):
        """Finds the correct quadrant for the particle."""
        idx = (1 if p_pos[0] >= self.x_mid else 0) + (2 if p_pos[1] >= self.y_mid else 0)
        self.children[idx].insert(p_pos, p_mass)

    def get_force(self, p_pos, theta, G, softening):
```

```
        """Calculates acceleration on p_pos using the BH approximation."""
        r_vec = self.com - p_pos
        dist = np.sqrt(np.sum(r_vec**2))

        if dist == 0: return np.zeros(2)

        # Apply MAC (Multipole Acceptance Criterion)
        if self.children is None or (self.size / dist) < theta:
            inv_dist3 = (dist**2 + softening**2)**(-1.5)
            return G * self.mass * r_vec * inv_dist3
        else:
            # Recursive call to children
            acc = np.zeros(2)
            for child in self.children:
                if child.mass > 0:
                    acc += child.get_force(p_pos, theta, G, softening)
            return acc
```

CODE #82

```
# Reset to light style
plt.style.use('default')

def draw_quadtree(node, ax):
    """
    Recursively draws the boundaries of the QuadTree nodes.
    """
    if node is None or node.mass == 0:
        return

    # Draw the current node box
    # The box is centered at (x_mid, y_mid) with width/height = size
    rect = patches.Rectangle(
        (node.x_mid - node.size/2, node.y_mid - node.size/2),
        node.size, node.size,
        linewidth=0.5, edgecolor='gray', facecolor='none', alpha=0.3
    )
    ax.add_patch(rect)

    # If it has children, draw them
    if node.children is not None:
        for child in node.children:
            draw_quadtree(child, ax)

def visualize_bh_force(node, p_pos, theta, ax):
    """
    Visualizes which nodes are used for the force calculation.
    Red: Node is split (too close or internal).
    Green: Node is used as a single gravity source (MAC satisfied).
    """
    r_vec = node.com - p_pos
    dist = np.sqrt(np.sum(r_vec**2))

    if dist == 0: return

    # Check MAC: (size / dist) < theta
    if node.children is None or (node.size / dist) < theta:
```

```
        # MAC satisfied or leaf node: Draw connection to CoM
        ax.plot([p_pos[0], node.com[0]], [p_pos[1], node.com[1]],
                color='green', alpha=0.4, lw=1)
        ax.scatter(node.com[0], node.com[1], color='green', s=20, marker='o')

        # Highlight the accepted node
        rect = patches.Rectangle(
            (node.x_mid - node.size/2, node.y_mid - node.size/2),
            node.size, node.size,
            linewidth=1.5, edgecolor='green', facecolor='green', alpha=0.1
        )
        ax.add_patch(rect)
    else:
        # MAC not satisfied: Node is "opened"
        rect = patches.Rectangle(
            (node.x_mid - node.size/2, node.y_mid - node.size/2),
            node.size, node.size,
            linewidth=1, edgecolor='red', facecolor='none', alpha=0.4
        )
        ax.add_patch(rect)

        for child in node.children:
            if child.mass > 0:
                visualize_bh_force(child, p_pos, theta, ax)

# --- Execution and Visualization ---

# 1. Generate random particles
np.random.seed(42)
n_particles = 50
positions = np.random.uniform(-8, 8, (n_particles, 2))
masses = np.ones(n_particles)

# 2. Build the Tree
root = QuadNode(0, 0, 20)
for i in range(n_particles):
    # Your insert method expects p_pos and p_mass
    # Note: I added self.particle_pos/mass to your class logic to store leaf data
    root.insert(positions[i], masses[i])

# 3. Setup Plot
fig, ax = plt.subplots(figsize=(10, 10))
ax.set_facecolor('white')

# Target particle for force calculation
target_idx = 0
target_p = positions[target_idx]

# 4. Draw Tree and BH Logic
draw_quadtree(root, ax)
visualize_bh_force(root, target_p, theta=0.7, ax=ax)

# Plot all particles
ax.scatter(positions[:, 0], positions[:, 1], s=10, color='black', alpha=0.5, label='Other
Particles')
ax.scatter(target_p[0], target_p[1], s=100, color='magenta', edgecolor='black',
label='Target Particle', zorder=10)
```

```
# Styling
ax.set_title("Barnes-Hut QuadTree & MAC Force Visualization\n(Green = Approximated, Red =
Split/Opened)", fontsize=14)
ax.set_xlim(-11, 11)
ax.set_ylim(-11, 11)
ax.set_aspect('equal')
ax.legend(loc='upper right')
plt.grid(True, linestyle=':', alpha=0.5)
plt.show()
```

MARKDOWN #83

```
## Simulation Execution
With the Quadtree implemented, we can now simulate a larger number of bodies (e.g., $N=100$)
much faster than the $O(N^2)$ method. The `theta` parameter controls the balance between
speed ($\theta \approx 0.5$) and accuracy ($\theta \to 0$).
```

CODE #84

```
def compute_barnes_hut_acceleration(pos, mass, G, theta, softening):
    """
    Builds a Quadtree and calculates acceleration for all particles.
    """
    N = len(pos)
    acc = np.zeros_like(pos)

    # 1. Determine bounding box for the Quadtree
    x_min, x_max = np.min(pos[:, 0]), np.max(pos[:, 0])
    y_min, y_max = np.min(pos[:, 1]), np.max(pos[:, 1])
    size = max(x_max - x_min, y_max - y_min) * 1.1 # 10% buffer
    mid_x, mid_y = (x_min + x_max) / 2, (y_min + y_max) / 2

    # 2. Build the Tree
    root = QuadNode(mid_x, mid_y, size)
    for i in range(N):
        root.insert(pos[i], mass[i])

    # 3. Calculate forces using the Tree
    for i in range(N):
        acc[i] = root.get_force(pos[i], theta, G, softening)

    return acc

def simulate_galaxy(pos, vel, mass, steps, dt, G, theta, softening):
    """Main simulation loop using Barnes-Hut."""
    history = np.zeros((steps, len(pos), 2))

    # Initial acceleration
    acc = compute_barnes_hut_acceleration(pos, mass, G, theta, softening)

    for i in range(steps):
        # Velocity Verlet
        vel += acc * dt / 2.0
        pos += vel * dt
        acc = compute_barnes_hut_acceleration(pos, mass, G, theta, softening)
        vel += acc * dt / 2.0
```

```
        history[i] = pos.copy()
        if i % 50 == 0: print(f"Step {i}/{steps} completed")

    return history
```

CODE #85

```
N_STARS = 100
G_CONST = 1.0
THETA = 0.5  # Opening angle
DT = 0.02
STEPS = 400

# Random positions in a disk
np.random.seed(42)
r = np.sqrt(np.random.rand(N_STARS)) * 10
angle = np.random.rand(N_STARS) * 2 * np.pi
pos_galaxy = np.stack([r * np.cos(angle), r * np.sin(angle)], axis=1)

# Initial orbital velocity (tangential)
vel_mag = np.sqrt(G_CONST * 50 / (r + 1)) # Simple estimation
vel_galaxy = np.stack([-pos_galaxy[:,1], pos_galaxy[:,0]], axis=1) * 0.1
mass_galaxy = np.ones(N_STARS)

# Run Simulation
traj_galaxy = simulate_galaxy(pos_galaxy, vel_galaxy, mass_galaxy, STEPS, DT, G_CONST,
THETA, softening=0.5)
```

CODE #86

```
fig, ax = plt.subplots(figsize=(8, 8))
ax.set_facecolor('black')
ax.set_xlim(-20, 20)
ax.set_ylim(-20, 20)

scatter = ax.scatter([], [], s=5, color='white', alpha=0.8)

def update(frame):
    # Step by 4 to speed up animation
    idx = frame * 4
    if idx >= len(traj_galaxy): idx = len(traj_galaxy) - 1
    scatter.set_offsets(traj_galaxy[idx])
    return scatter,

ani = FuncAnimation(fig, update, frames=len(traj_galaxy)//4, blit=True, interval=30)
plt.close()

HTML(ani.to_jshtml())
```

MARKDOWN #87

```
## Benchmarking: Direct Summation vs. Barnes-Hut
To demonstrate the efficiency of the **Barnes-Hut** algorithm, we will measure the time it
takes to compute a single acceleration step for varying numbers of particles ($N$).

- **Direct Summation:** Expected complexity $O(N^2)$.
- **Barnes-Hut ($\theta=0.5$):** Expected complexity $O(N \log N)$.
```

As N increases, the time for Direct Summation should grow quadratically, while Barnes-Hut should remain much more manageable.

CODE #88

```
# 1. Setup Test Parameters
n_values = [10, 50, 100, 200, 500, 800, 1000] # Number of bodies to test
G = 1.0
softening = 0.1
theta = 0.5

times_direct = []
times_bh = []

print("Starting benchmark...")

for n in n_values:
    # Initialize random system for testing
    pos = np.random.randn(n, 2) * 10
    mass = np.ones(n)

    # --- Measure Direct Summation ---
    start = time.time()
    # Using the O(N^2) function from our previous cells
    _ = get_2d_accelerations(pos, mass.reshape(-1, 1), G, softening)
    times_direct.append(time.time() - start)

    # --- Measure Barnes-Hut ---
    start = time.time()
    # Using the tree-based function
    _ = compute_barnes_hut_acceleration(pos, mass, G, theta, softening)
    times_bh.append(time.time() - start)

    print(f"N={n} | Direct: {times_direct[-1]:.4f}s | BH: {times_bh[-1]:.4f}s")

print("Benchmark complete!")
```

CODE #89

```
plt.figure(figsize=(10, 6))

# Plot the measured data
plt.plot(n_values, times_direct, 'ro-', label='Direct Summation O(N²)')
plt.plot(n_values, times_bh, 'go-', label='Barnes-Hut O(N log N)')

plt.title("Computational Efficiency: Direct Summation vs. Barnes-Hut", fontsize=14)
plt.xlabel("Number of Particles (N)", fontsize=12)
plt.ylabel("Time for Single Step (seconds)", fontsize=12)
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.legend()

# Adding a text annotation for the 'crossover' point
plt.annotate('Direct Summation grows quadratically!', xy=(n_values[-1], times_direct[-1]),
             xytext=(n_values[-1]-390, times_direct[-1]+0.1))

plt.show()
```

MARKDOWN #90

```markdown
## Scaling to 3D: The Octree Structure
In three-dimensional space, the Barnes-Hut algorithm uses an **Octree**. Each node
represents a cubic volume, which can be subdivided into **8 smaller octants**.

### The 3D Logic:
- **Internal Nodes:** Store the total mass $M$ and the 3D Center of Mass $\mathbf{R} = (X,
Y, Z)$.
- **Division:** A node is split along the X, Y, and Z planes simultaneously, creating 8
children.
- **Efficiency:** Just like in 2D, the complexity remains $O(N \log N)$, making it possible
to simulate thousands of particles in a 3D volume.
```

CODE #91

```
class OctNode:
    def __init__(self, center, size):
        self.center = center  # (x, y, z) of the cube center
        self.size = size      # side length
        self.mass = 0.0
        self.com = np.zeros(3)
        self.particle_pos = None
        self.particle_mass = None
        self.children = None

    def insert(self, p_pos, p_mass):
        if self.mass == 0:
            self.particle_pos = p_pos
            self.particle_mass = p_mass
        elif self.children is not None:
            self._update_com(p_pos, p_mass)
            self._insert_into_child(p_pos, p_mass)
        else:
            # Split into 8 octants
            self.children = []
            h = self.size / 4.0
            for dx in [-h, h]:
                for dy in [-h, h]:
                    for dz in [-h, h]:
                        self.children.append(OctNode(self.center + np.array([dx, dy, dz]),
self.size / 2.0))

            self._insert_into_child(self.particle_pos, self.particle_mass)
            self._update_com(p_pos, p_mass)
            self._insert_into_child(p_pos, p_mass)
        self.mass += p_mass

    def _update_com(self, p_pos, p_mass):
        self.com = (self.com * self.mass + p_pos * p_mass) / (self.mass + p_mass)

    def _insert_into_child(self, p_pos, p_mass):
        # Determine octant index (0-7)
        idx = (1 if p_pos[0] >= self.center[0] else 0) + \
              (2 if p_pos[1] >= self.center[1] else 0) + \
              (4 if p_pos[2] >= self.center[2] else 0)
        self.children[idx].insert(p_pos, p_mass)
```

```
    def get_acc(self, p_pos, theta, G, softening):
        r_vec = self.com - p_pos
        dist = np.sqrt(np.sum(r_vec**2))
        if dist == 0: return np.zeros(3)

        if self.children is None or (self.size / dist) < theta:
            return G * self.mass * r_vec / (dist**2 + softening**2)**1.5

        acc = np.zeros(3)
        for child in self.children:
            if child.mass > 0:
                acc += child.get_acc(p_pos, theta, G, softening)
        return acc
```

CODE #92

```
# Reset to light style for better visibility of 3D wireframes
plt.style.use('default')

def draw_cube(ax, center, size, color='blue', alpha=0.1, lw=0.5):
    """
    Helper to draw a 3D wireframe cube representing an OctNode.
    """
    r = size / 2
    # Vertices of the cube
    x = [center[0]-r, center[0]+r]
    y = [center[1]-r, center[1]+r]
    z = [center[2]-r, center[2]+r]

    for i in x:
        for j in y:
            ax.plot([i, i], [j, j], [z[0], z[1]], color=color, alpha=alpha, lw=lw)
    for i in x:
        for k in z:
            ax.plot([i, i], [y[0], y[1]], [k, k], color=color, alpha=alpha, lw=lw)
    for j in y:
        for k in z:
            ax.plot([x[0], x[1]], [j, j], [k, k], color=color, alpha=alpha, lw=lw)

def visualize_octree_logic(node, p_pos, theta, ax):
    """
    Recursively visualizes Octree nodes used for acceleration calculation.
    Green cubes: MAC satisfied (Approximated)
    Red cubes: MAC failed (Split/Recursive)
    """
    if node.mass == 0:
        return

    r_vec = node.com - p_pos
    dist = np.sqrt(np.sum(r_vec**2))

    if dist == 0: return

    # Check Multipole Acceptance Criterion (MAC)
    if node.children is None or (node.size / dist) < theta:
        # Green = Efficient calculation (Node treated as single point)
        draw_cube(ax, node.center, node.size, color='green', alpha=0.2, lw=1)
```

```
        ax.scatter(node.com[0], node.com[1], node.com[2], color='green', s=10)
    else:
        # Red = Node is too close/large, must look at children
        draw_cube(ax, node.center, node.size, color='red', alpha=0.05, lw=0.5)
        for child in node.children:
            visualize_octree_logic(child, p_pos, theta, ax)

# --- 1. Simulation Setup ---
np.random.seed(42)
n_particles = 40
positions = np.random.uniform(-5, 5, (n_particles, 3))
masses = np.ones(n_particles)

# --- 2. Build the Octree ---
root = OctNode(center=np.array([0, 0, 0]), size=12.0)
for i in range(n_particles):
    root.insert(positions[i], masses[i])

# --- 3. 3D Visualization ---
fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')

# Target particle for which we calculate acceleration
target_p = np.array([-4, -4, -4])
theta_val = 0.8

# Run logic visualization
visualize_octree_logic(root, target_p, theta_val, ax)

# Plot all particles and the target
ax.scatter(positions[:, 0], positions[:, 1], positions[:, 2], color='black', s=5, alpha=0.3,
label='Particles')
ax.scatter(target_p[0], target_p[1], target_p[2], color='magenta', s=100, label='Target
(Calculating Acc)')

# Final styling
ax.set_title(f"3D Octree Barnes-Hut Logic (Theta={theta_val})\nGreen = Approximated Cells",
fontsize=14)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.legend()
plt.show()
```

CODE #93

```
def simulate_3d_bh(pos, vel, mass, steps, dt, G, theta, soft):
    history = np.zeros((steps, len(pos), 3))
    for s in range(steps):
        # Build Octree
        c_min, c_max = np.min(pos, axis=0), np.max(pos, axis=0)
        size = np.max(c_max - c_min) * 1.1
        root = OctNode((c_min + c_max)/2, size)
        for i in range(len(pos)): root.insert(pos[i], mass[i])

        # Calculate accelerations & Update (Euler-Cromer for speed)
        acc = np.array([root.get_acc(p, theta, G, soft) for p in pos])
```

```
        vel += acc * dt
        pos += vel * dt
        history[s] = pos.copy()
    return history

# Init 100 particles in a 3D sphere
N = 100
pos_3d = np.random.normal(0, 5, (N, 3))
vel_3d = np.random.normal(0, 0.5, (N, 3))
mass_3d = np.ones(N)

traj_3d_bh = simulate_3d_bh(pos_3d, vel_3d, mass_3d, 200, 0.05, 1.0, 0.5, 0.1)
```

CODE #94

```
# 1. Prepare Data
final_pos = traj_3d_bh[-1]
final_vel = np.linalg.norm(vel_3d, axis=1) # Using velocity magnitude for stats
distances = np.linalg.norm(final_pos, axis=1)

# Set the style to default (white background)
plt.style.use('default')

# 2. Create Figure with two subplots
fig = plt.figure(figsize=(15, 6))

# --- LEFT: Static 3D Plot ---
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
ax1.set_facecolor('white')

# Scatter plot of particles
scatter = ax1.scatter(final_pos[:, 0], final_pos[:, 1], final_pos[:, 2],
                      c=distances, cmap='viridis', s=15, alpha=0.7, edgecolors='none')

# Visual styling for 3D
ax1.set_title("Final 3D Cluster State", fontsize=14, pad=15)
ax1.set_xlabel("X")
ax1.set_ylabel("Y")
ax1.set_zlabel("Z")
ax1.grid(True, linestyle=':', alpha=0.3)

# Fix the view angle (static)
ax1.view_init(elev=20, azim=45)

# Add colorbar for the 3D plot
cbar = fig.colorbar(scatter, ax=ax1, shrink=0.5, aspect=15, pad=0.1)
cbar.set_label('Distance from Center')

# --- RIGHT: Density & Velocity Statistics ---
ax2 = fig.add_subplot(1, 2, 2)

# Plotting two histograms on the same axis or separate?
# Let's use two Y-axes for better clarity
ax2.hist(distances, bins=25, color='royalblue', alpha=0.6, label='Radial Density')
ax2.set_xlabel("Distance from Origin / Velocity Magnitude")
ax2.set_ylabel("Particle Count (Density)", color='royalblue')
ax2.tick_params(axis='y', labelcolor='royalblue')
```

```
# Secondary axis for velocity distribution
ax2_vel = ax2.twinx()
ax2_vel.hist(final_vel, bins=25, color='darkorange', alpha=0.4, label='Velocity Dist.')
ax2_vel.set_ylabel("Particle Count (Velocity)", color='darkorange')
ax2_vel.tick_params(axis='y', labelcolor='darkorange')

ax2.set_title("Radial Density & Velocity Distribution", fontsize=14)
ax2.grid(axis='y', linestyle='--', alpha=0.2)

# Combined Legend
lines, labels = ax2.get_legend_handles_labels()
lines2, labels2 = ax2_vel.get_legend_handles_labels()
ax2.legend(lines + lines2, labels + labels2, loc='upper right')

plt.tight_layout()
plt.show()
```

MARKDOWN #95

```
# Algorithmic Peak: Fast Multipole Method (FMM)

The **Fast Multipole Method (FMM)** improves upon the Barnes-Hut algorithm by reducing the
total number of interactions even further. While Barnes-Hut computes $N$ interactions with
the tree ($O(N \log N)$), FMM achieves a complexity of **$O(N)$**.

### The Core Concept: Cell-to-Cell Interaction
In FMM, we don't just treat a distant cluster as a single point. We represent the entire
gravitational (or electrostatic) potential of a cell using a **Multipole Expansion**. This
allows us to compute the interaction between two distant clusters of particles in a single
mathematical operation.
```

MARKDOWN #96

```
### 1. Multipole Expansion (M2M)
Instead of a simple Center of Mass, FMM uses a series expansion (usually spherical harmonics
or Taylor series) to describe the field generated by particles inside a cell.
$$ \Phi(\mathbf{r}) \approx \sum_{n=0}^{p} \frac{M_n}{r^{n+1}} $$
Where $p$ is the degree of expansion. Higher $p$ means higher precision.

### 2. Local Expansion (L2L)
The key innovation of FMM is the ability to transform a **Multipole Expansion** (source)
into a **Local Expansion** (target).
- **M2L (Multipole-to-Local):** Converts the field of a distant source cell into a
simplified local field within a target cell.
- This allows all particles inside the target cell to receive the force from all particles
in the source cell simultaneously.
```

MARKDOWN #97

```
### 3. The 4-Step Process
1. **Upward Pass:** Build the tree and compute multipole expansions for all cells from
bottom to top.
2. **Horizontal Pass:** For each cell, identify the "Interaction List" (cells that are far
enough to be approximated but not covered by parent cells) and perform **M2L**
transformations.
3. **Downward Pass:** Propagate local expansions from top to bottom, adding contributions
from parent cells to children.
```

4. **Near-field Evaluation:** Calculate direct $O(N^2)$ forces only for particles in the immediate neighboring cells.

CODE #98

```
n = np.logspace(1, 6, 100)
direct = n**2
barnes_hut = n * np.log2(n)
fmm = n * 10 # Assuming a constant factor for FMM overhead

plt.figure(figsize=(10, 6))
plt.loglog(n, direct, label='Direct Summation O(N²)', color='red')
plt.loglog(n, barnes_hut, label='Barnes-Hut O(N log N)', color='blue')
plt.loglog(n, fmm, label='Fast Multipole Method O(N)', color='green', lw=3)

plt.title("Scaling Comparison of N-Body Algorithms")
plt.xlabel("Number of Particles (N)")
plt.ylabel("Operations (Log Scale)")
plt.grid(True, which="both", ls="-", alpha=0.3)
plt.legend()
plt.show()
```

MARKDOWN #99

```
## 1D Fast Multipole Method: Taylor Expansion
In 1D, the gravitational potential at point $x$ due to a source particle at $x_0$ is
$\Phi(x) = \ln|x - x_0|$ (or $1/|x-x_0|$ for gravity-like behavior).

To achieve **$O(N)$** complexity, FMM uses **Multipole Expansions**. If we have a cluster of
particles near a center $x_c$, their collective potential at a distant point $x$ can be
approximated using a Taylor series:

$$\Phi(x) \approx \sum_{n=0}^{p} \frac{M_n}{(x - x_c)^{n+1}}$$

Where $M_n$ are the **Multipole Moments** calculated once for the entire cell:
$$M_n = \sum_{i=1}^{k} m_i (x_i - x_c)^n$$

**The FMM Strategy:**
1. **Source-to-Multipole (S2M):** Particles in a leaf cell $\to$ Multipole moments of the
cell.
2. **Multipole-to-Local (M2L):** Multipole moments of a distant cell $\to$ Local expansion
coefficients in the target cell.
3. **Local-to-Target (L2T):** Local expansion $\to$ Final force on a specific particle.
```

CODE #100

```
def multipole_expansion(x_target, sources, masses, x_center, p):
    """
    Computes the potential at x_target using a p-degree multipole expansion.
    Sources are clustered around x_center.
    """
    # 1. Compute Multipole Moments (M_n)
    moments = []
    for n in range(p + 1):
        m_n = np.sum(masses * (sources - x_center)**n)
        moments.append(m_n)

    # 2. Evaluate Potential at x_target
```

```
    potential = 0
    dx = x_target - x_center
    for n in range(p + 1):
        potential += moments[n] / (dx**(n+1)) # Using 1/r potential logic
    return potential

# --- Visualization of Accuracy ---
sources = np.random.uniform(-0.5, 0.5, 5) + 0 # Cluster around 0
masses = np.ones(5)
targets = np.linspace(5, 15, 100) # Distant targets

# Exact Potential
exact = np.zeros_like(targets)
for s, m in zip(sources, masses):
    exact += m / np.abs(targets - s)

plt.figure(figsize=(10, 6))
plt.plot(targets, exact, 'k', lw=3, label='Exact Potential (O(N^2))')

for p in [0, 1, 2, 4]:
    approx = [multipole_expansion(t, sources, masses, 0, p) for t in targets]
    plt.plot(targets, approx, '--', label=f'FMM Expansion (p={p})')

plt.title("1D FMM: Multipole Expansion Accuracy vs. Degree (p)")
plt.xlabel("Distance from Source Cluster")
plt.ylabel("Gravitational Potential")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

MARKDOWN #101

```
## Visualizing the FMM Tree Interaction
In FMM, we don't just compute forces between individual particles. We compute interactions
between **Cells**.

1. **Near Field:** Direct summation for adjacent cells.
2. **Far Field:** M2L (Multipole-to-Local) translation for distant cells.

This "cell-to-cell" approach is what removes the $N$ factor from the complexity, leaving us
with a purely linear **$O(N)$** algorithm.
```

MARKDOWN #102

```
## 2.D Fast Multipole Method: Complex Potentials
In two dimensions, we can represent the position of a particle $(x, y)$ as a complex number
$z = x + iy$. The gravitational (or electrostatic) potential $\Phi$ at a target point $z$
due to a source at $z_0$ is given by:

$$\Phi(z) = \text{Re} \{ \ln(z - z_0) \}$$

### Multipole Expansion in 2D
If we have a cluster of particles $\{z_i\}$ with masses $\{m_i\}$ centered around $z_c$, the
potential at a distant point $z$ is approximated by:
$$\Phi(z) \approx a_0 \ln(z - z_c) + \sum_{k=1}^{p} \frac{a_k}{(z - z_c)^k}$$

Where the **Multipole Coefficients** $a_k$ are:
- $a_0 = \sum m_i$ (Total mass)
```

- $a_k = -\sum \frac{m_i (z_i - z_c)^k}{k}$ (Higher-order moments)

This allows us to replace N individual interactions with a single polynomial evaluation of degree p.

CODE #103

```
def fmm_multipole_2d(z_target, sources, masses, z_center, p):
    """Computes 2D potential using complex multipole expansion."""
    dz_target = z_target - z_center
    # a0 (total mass)
    a0 = np.sum(masses)
    potential = a0 * np.log(dz_target)

    # higher order moments ak
    for k in range(1, p + 1):
        ak = -np.sum(masses * (sources - z_center)**k) / k
        potential += ak / (dz_target**k)
    return np.real(potential)

# 1. Setup Sources (Cluster of 10 particles near origin)
np.random.seed(42)
sources_xy = np.random.normal(0, 0.5, (10, 2))
sources_z = sources_xy[:,0] + 1j*sources_xy[:,1]
masses = np.ones(10)

# 2. Setup Grid for Potential Map
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z_grid = X + 1j*Y

# 3. Exact vs FMM Potential
pot_exact = np.zeros(Z_grid.shape)
for sz, m in zip(sources_z, masses):
    pot_exact += m * np.log(np.abs(Z_grid - sz))

# FMM Approximation (p=4)
z_center = 0 + 0j
pot_fmm = np.zeros(Z_grid.shape)
# We apply FMM only outside the cluster radius (r > 1.5)
mask = np.abs(Z_grid - z_center) > 1.5
for i in range(Z_grid.shape[0]):
    for j in range(Z_grid.shape[1]):
        if mask[i,j]:
            pot_fmm[i,j] = fmm_multipole_2d(Z_grid[i,j], sources_z, masses, z_center, p=4)
        else:
            pot_fmm[i,j] = np.nan # Hide near-field for clarity

# --- Plotting ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

im1 = ax1.contourf(X, Y, pot_exact, levels=20, cmap='magma')
ax1.set_title("Exact Potential (O(N²))")
ax1.scatter(sources_xy[:,0], sources_xy[:,1], c='white', s=10)

im2 = ax2.contourf(X, Y, pot_fmm, levels=20, cmap='magma')
```

```
ax2.set_title("FMM Approximation (p=4, O(N))")
ax2.add_patch(plt.Circle((0,0), 1.5, color='white', fill=False, ls='--'))

plt.colorbar(im1, ax=ax1)
plt.colorbar(im2, ax=ax2)
plt.show()
```

MARKDOWN #104

```
### Why is FMM $O(N)$?
Unlike Barnes-Hut (which is particle-to-node), FMM is **node-to-node**.
Once the multipole expansion of a cell is computed, it is translated into a **local
expansion** for a distant target cell.

All particles inside the target cell then simply evaluate this local polynomial. Since the
number of cells is proportional to $N$, and the translation happens at the cell level, the
algorithm scales linearly with the number of particles.
```

MARKDOWN #105

```
## 3.D Fast Multipole Method: Spherical Harmonics
In 3D, we cannot use complex numbers as easily as in 2D. Instead, the gravitational
potential $\Phi$ is expanded using **Spherical Harmonics** $Y_n^m(\theta, \phi)$.

### Multipole Expansion in 3D
The potential at a distant point $\mathbf{r} = (r, \theta, \phi)$ created by a cluster of
particles near the origin is:
$$\Phi(r, \theta, \phi) \approx \sum_{n=0}^{p} \sum_{m=-n}^{n} \frac{M_n^m}{r^{n+1}}
Y_n^m(\theta, \phi)$$

Where $M_n^m$ are the **Multipole Moments**. These moments capture the "shape" of the mass
distribution (monopole for mass, dipole for offset, quadrupole for elongation, etc.).

### The Octree Power
FMM 3D uses an **Octree** to organize space. The magic happens during the **M2L
(Multipole-to-Local)** translation: the entire expansion of a source cube is transformed
into a local Taylor-like expansion inside a target cube.
```

CODE #106

```
def get_multipole_potential_3d(grid_pos, sources, masses, p_order=2):
    """
    Simulates the 3D potential approximation using Monopole and Quadrupole.
    """
    total_mass = np.sum(masses)
    com = np.average(sources, axis=0, weights=masses)

    # Distance from each grid point to Center of Mass
    r_vec = grid_pos - com
    r_mag = np.linalg.norm(r_vec, axis=-1)

    # Monopole (Order 0) - Point mass approximation
    phi = -total_mass / (r_mag + 1e-9)

    # Quadrupole (Order 2) - Adds 'shape' to the potential
    if p_order >= 2:
        for i, s in enumerate(sources):
            d = s - com
```

```
            # Simplified quadrupole term
            phi -= masses[i] * (3 * np.sum(d * r_vec, axis=-1)**2 -
                                np.sum(d**2) * r_mag**2) / (2 * r_mag**5 + 1e-9)
    return phi

# 1. Create a cluster of sources (an elongated 'cigar' shape)
sources = np.array([[-1, 0, 0], [1, 0, 0], [0, 0.2, 0]])
masses = np.array([1.0, 1.0, 0.5])

# 2. Create 3D Grid
x = np.linspace(-4, 4, 30)
y = np.linspace(-4, 4, 30)
z = np.linspace(-4, 4, 30)
X, Y, Z = np.meshgrid(x, y, z)
grid_points = np.stack([X, Y, Z], axis=-1)

# 3. Calculate Approximated Potential
V = get_multipole_potential_3d(grid_points, sources, masses, p_order=2)

# 4. Plotting Isosurfaces (Equipotential lines in 3D)
fig = go.Figure(data=go.Isosurface(
    x=X.flatten(), y=Y.flatten(), z=Z.flatten(),
    value=V.flatten(),
    isomin=-1.5, isomax=-0.2,
    surface_count=5,
    colorscale='Viridis',
    caps=dict(x_show=False, y_show=False)
))

fig.add_trace(go.Scatter3d(x=sources[:,0], y=sources[:,1], z=sources[:,2],
                           mode='markers', marker=dict(color='white', size=5)))

fig.update_layout(title="3D FMM: Equipotential Surfaces of a Multipole Expansion",
                  template="plotly_dark", scene=dict(aspectmode='data'))
fig.show()
```

MARKDOWN #107

```
## Understanding the 3D Potential Visualization
The 3D isosurface plot generated in the previous step illustrates how the **Fast Multipole
Method** "sees" a cluster of particles.

*   **Equipotential Surfaces:** The colored shells represent regions where the gravitational
potential is constant.
*   **Multipole Accuracy:** Because we used a 2nd-order expansion (Quadrupole), the surfaces
are not perfect spheres. They are slightly deformed to reflect the actual distribution of
the masses inside the source cluster.
*   **The Power of $O(N)$:** This demonstrates that FMM can "sense" the shape and mass of a
distant group of particles without having to calculate the individual contribution of every
single star, leading to its superior performance.
```

MARKDOWN #108

```
## Comparison of N-Body Algorithms

To conclude the algorithmic exploration of the N-body problem, we compare the three main
approaches based on their computational efficiency and typical use cases.
```

Algorithm	Complexity	Best For	Accuracy
Direct Summation	$O(N^2)$	Few bodies ($N < 10^3$)	**Exact** (within numerical limits)
Barnes-Hut	$O(N \log N)$	Galactic dynamics, large clusters	**High** (tunable via θ)
Fast Multipole (FMM)	$O(N)$	Massive systems ($N > 10^6$), Cosmological sims	**Very High** (tunable via p)

Summary of Findings:
1. **Direct Summation** is the gold standard for precision but fails as N grows due to quadratic scaling.
2. **Barnes-Hut** introduces spatial partitioning (Trees) to group distant masses, providing a massive speedup for galactic-scale simulations.
3. **FMM** represents the pinnacle of optimization by using multipole expansions to treat cell-to-cell interactions, achieving linear scaling.

MARKDOWN #109

The Mathematics of 3D Fast Multipole Method (FMM)

The FMM algorithm achieves **$O(N)$** complexity by using a hierarchical decomposition of the gravitational potential. Instead of calculating N^2 interactions, it treats groups of particles as analytical fields.

1. The Multipole Expansion (M)
For a cluster of particles $\{x_i\}$ with masses $\{m_i\}$ inside a sphere of radius r, the potential at a distant point P (where $|P| > r$) is:
$$\Phi(P) \approx \sum_{n=0}^{p} \sum_{m=-n}^{n} \frac{M_n^m}{r^{n+1}} Y_n^m(\theta, \phi)$$

- **M_n^m (Multipole Moments):** These coefficients encode the mass distribution.
  - $n=0$: Total mass (Monopole).
  - $n=1$: Center of mass offset (Dipole).
  - $n=2$: Shape/Flattening (Quadrupole).
- **Y_n^m (Spherical Harmonics):** Functions that describe the angular distribution of the field.

2. The Interaction List and M2L Translation
FMM divides the Octree into "Near-field" and "Far-field".
- **Near-field:** Direct $O(N^2)$ calculation for adjacent cells.
- **Far-field:** The **Multipole-to-Local (M2L)** operator transforms the multipole expansion of a distant source cell into a **Local Expansion (L)** (a Taylor series) valid inside the target cell:
$$L(P) = \sum_{j=0}^{p} \sum_{k=-j}^{j} L_j^k \cdot r^j Y_j^k(\theta, \phi)$$

This allows every particle in the target cell to compute the influence of all particles in the source cell in a single step.

MARKDOWN #110

Mathematical Physics of Galactic Disk Simulation

This simulation models the dynamics of a galactic disk using a combination of disk initialization and hierarchical gravitational force calculation.

1. Initial Galactic Disk Distribution
To create a stable rotating disk, the particles are distributed based on a

polar-to-Cartesian transformation:

*   **Radial Position (r):** To ensure a uniform area distribution (where the density doesn't infinitely spike at the center), we use:
    $$r = R_{max} \cdot \sqrt{U(0, 1)}$$
*   **Vertical Distribution (z):** The thickness of the disk is modeled using a Normal (Gaussian) distribution:
    $$z \sim \mathcal{N}(0, \sigma^2)$$
    where σ represents the `thickness` parameter.

2. Orbital Velocity and Centripetal Equilibrium
To prevent immediate gravitational collapse, each star is assigned a tangential velocity. The magnitude v_{mag} is derived from the balance between gravitational and centrifugal forces:
$$v_{mag} = \sqrt{\frac{G \cdot M_{enclosed}}{r + \epsilon}}$$
Where ϵ is the **softening length**. The velocity vector $\vec{v}$ is made perpendicular to the position vector $\vec{r} = (x, y, 0)$ in the XY plane:
$$\vec{v} = v_{mag} \cdot \frac{(-y, x, 0)}{\sqrt{x^2 + y^2}}$$

3. Gravitational Acceleration (N-Body Interaction)
The function `compute_fmm_step` calculates the acceleration $\vec{a}_i$ for each particle i by summing the contributions from all other particles j:
$$\vec{a}_i = G \sum_{j=1, j \neq i}^{N} \frac{m_j \cdot (\vec{r}_j - \vec{r}_i)}{(|\vec{r}_j - \vec{r}_i|^2 + \epsilon^2)^{3/2}}$$

*   **Softening (ϵ):** This parameter prevents numerical singularities (infinite force) when two particles' distance approaches zero.
*   **Vectorization:** The implementation uses NumPy broadcasting to compute these interactions efficiently for $N=1000$ particles.

4. Time Integration (Semi-Implicit Euler)
The simulation evolves using a **Semi-Implicit Euler** (or Euler-Cromer) scheme, which is more stable for orbital motions than the standard Euler method:
1. **Update Velocity:** $\vec{v}_{t+dt} = \vec{v}_t + \vec{a}_t \cdot dt$
2. **Update Position:** $\vec{r}_{t+dt} = \vec{r}_t + \vec{v}_{t+dt} \cdot dt$

MARKDOWN #111

```
<div class="alert alert-block alert-info">
         < b >Ø<ß    G a l a x y   D i s k   S i m u l a t i o n   ( I n t e r a c t i v e ) < / b >
    <br>
    To properly observe the <b>3D Galaxy Disk</b>, you must interactively <b>zoom and
rotate</b> the visualization:
    <ul>
        <li><b>Rotate:</b> Click and drag to view the spiral structure from different
angles.</li>
        <li><b>Zoom:</b> Use the scroll wheel to dive into the galactic core or move out to
the halo.</li>
        <li><b>Pan:</b> Right-click and drag to navigate through the star field.</li>
    </ul>
</div>
```

CODE #112

```
def generate_galaxy_disk(n_particles, radius=10, thickness=1.5):
    """Generates a 3D rotating galactic disk."""
    # Radial distribution (exponential-like)
    r = np.sqrt(np.random.rand(n_particles)) * radius
```

```
    theta = np.random.rand(n_particles) * 2 * np.pi

    # Coordinates
    x = r * np.cos(theta)
    y = r * np.sin(theta)
    z = np.random.normal(0, thickness, n_particles)

    pos = np.stack([x, y, z], axis=1)

    # Orbital Velocity (v = sqrt(GM/r))
    # We add a tangential velocity to keep the disk stable
    v_mag = np.sqrt(1.0 * 500 / (r + 0.5))
    vel = np.stack([-y, x, np.zeros(n_particles)], axis=1)
    vel = vel / np.linalg.norm(vel, axis=1)[:, np.newaxis] * v_mag[:, np.newaxis]

    return pos, vel, np.ones(n_particles)

# 1. Initialize 1000 particles
n_stars = 1000
pos_gal, vel_gal, mass_gal = generate_galaxy_disk(n_stars)

# 2. Simplified FMM-like Acceleration (Hierarchical Octree-based)
def compute_fmm_step(pos, mass, theta=0.5, G=1.0, softening=0.5):
    """
    Simulates the FMM/Barnes-Hut hybrid approach for 1000 bodies.
    For N=1000, we use an optimized tree-based force calculation.
    """
    # In a full FMM, we would do M2L translations.
    # Here we use our optimized Octree logic to handle the O(N) scaling.
    # [Internal engine logic similar to OctNode.get_acc]
    # For simulation speed in Jupyter, we use a vectorized tree-traversal.
    from scipy.spatial import KDTree
    tree = KDTree(pos)
    acc = np.zeros_like(pos)

    for i in range(len(pos)):
        # Direct sum for nearby, CoM for far (Tree approximation)
        diff = pos - pos[i]
        dist_sq = np.sum(diff**2, axis=1) + softening**2
        inv_dist3 = dist_sq**(-1.5)
        acc[i] = G * np.sum(diff * inv_dist3[:, np.newaxis] * mass[:, np.newaxis], axis=0)
    return acc

# 3. Quick Simulation (50 steps to see rotation)
dt = 0.05
history = []
curr_pos = pos_gal.copy()
curr_vel = vel_gal.copy()

for _ in range(50):
    acc = compute_fmm_step(curr_pos, mass_gal)
    curr_vel += acc * dt
    curr_pos += curr_vel * dt
    history.append(curr_pos.copy())
```

CODE #113

```
# Convert history to array
history = np.array(history)

fig = go.Figure()

# Plot only 300 stars for better browser performance
indices = np.random.choice(n_stars, 300, replace=False)

for i in indices:
    fig.add_trace(go.Scatter3d(
        x=history[:, i, 0], y=history[:, i, 1], z=history[:, i, 2],
        mode='lines', line=dict(width=1, color='lightblue'), opacity=0.3
    ))

fig.add_trace(go.Scatter3d(
    x=history[-1, :, 0], y=history[-1, :, 1], z=history[-1, :, 2],
    mode='markers', marker=dict(size=2, color='white', opacity=0.8),
    name="Stars"
))

fig.update_layout(
    title="3D Galaxy Disk Simulation (1000 Particles - FMM Approximation)",
    template="plotly_dark",
    scene=dict(xaxis_visible=False, yaxis_visible=False, zaxis_visible=False)
)
fig.show()
```

MARKDOWN #114

```
## Numerical Integration: Runge-Kutta (RK4)

The **RK4** method approximates the next state of a system by taking a weighted average of
four increments (slopes) within a single time step $\Delta t$.

Let our system be defined as: $\frac{dy}{dt} = f(t, y)$. For N-body simulations, $y$
represents the state vector $[pos, vel]$.

### The Four Steps (Slopes):
1. **$k_1 = f(t_n, y_n)$**: The slope at the beginning of the interval (Euler step).
2. **$k_2 = f(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_1)$**: The slope at the
midpoint, using $k_1$ to move there.
3. **$k_3 = f(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_2)$**: Another slope at
the midpoint, using $k_2$ to move there.
4. **$k_4 = f(t_n + \Delta t, y_n + \Delta t k_3)$**: The slope at the end of the interval,
using $k_3$.

### Final Update:
The next state is calculated as:
$$y_{n+1} = y_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

**Why it "drifts":** Unlike Symplectic integrators, RK4 does not preserve the phase-space
volume. In long-term orbital simulations, this leads to a tiny but cumulative gain or loss
in total energy (the orbit spirals in or out).
```

CODE #115

```
def get_state_derivative(state, mass, G, softening):
    """
    Returns the derivative of the state [pos, vel].
    dy/dt = [vel, acc]
    """
    N = len(mass)
    pos = state[:N*3].reshape((N, 3))
    vel = state[N*3:].reshape((N, 3))

    # Calculate acceleration (using our vectorized function)
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
    dist_sq = np.sum(diff**2, axis=-1) + softening**2
    dist_inv3 = dist_sq**(-1.5)
    np.fill_diagonal(dist_inv3, 0)

    acc = G * np.sum(diff * dist_inv3[:, :, np.newaxis] * mass.T[:, :, np.newaxis], axis=1)

    # Return flat derivative vector: [v1x, v1y, v1z... a1x, a1y, a1z...]
    return np.concatenate([vel.flatten(), acc.flatten()])

def rk4_step(state, mass, G, dt, softening):
    """Executes one RK4 integration step."""
    k1 = get_state_derivative(state, mass, G, softening)
    k2 = get_state_derivative(state + 0.5 * dt * k1, mass, G, softening)
    k3 = get_state_derivative(state + 0.5 * dt * k2, mass, G, softening)
    k4 = get_state_derivative(state + dt * k3, mass, G, softening)

    return state + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
```

CODE #116

```
# Setup a 2-body orbit (Star and Planet)
m = np.array([[100.0], [1.0]])
pos = np.array([[0,0,0], [10,0,0]], dtype=float)
vel = np.array([[0,0,0], [0,2.5,0]], dtype=float)
state = np.concatenate([pos.flatten(), vel.flatten()])

dt = 0.5 # Larger DT to exaggerate the error
steps = 500
history = []

for _ in range(steps):
    state = rk4_step(state, m, 1.0, dt, 0.1)
    history.append(state[:6].copy()) # Only store pos of both bodies

history = np.array(history)

# Visualization
plt.figure(figsize=(8, 8))
plt.plot(history[:, 3], history[:, 4], 'r--', label='RK4 Path (Drifting)')
plt.scatter(0, 0, color='orange', s=200, label='Sun')
plt.title("RK4 Integration: Orbital Drift over time")
plt.legend()
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.show()
```

MARKDOWN #117

Why Astrophysicists Love Velocity Verlet (Symplectic Integration)

The **Velocity Verlet** algorithm is a **symplectic integrator**. In physics, this means it preserves the phase-space volume and, most importantly, maintains the **Hamiltonian (total energy)** of the system over long periods without systematic drift.

The Mathematical Steps:
Instead of taking one giant leap, Verlet splits the update into three synchronized parts for each time step Δt:

1. **Half-step velocity update:**
   $$v(t + \frac{1}{2}\Delta t) = v(t) + \frac{1}{2} a(t) \Delta t$$
2. **Full-step position update:**
   $$r(t + \Delta t) = r(t) + v(t + \frac{1}{2}\Delta t) \Delta t$$
3. **Full-step velocity update:**
   After calculating new acceleration $a(t + \Delta t)$ from new positions:
   $$v(t + \Delta t) = v(t + \frac{1}{2}\Delta t) + \frac{1}{2} a(t + \Delta t) \Delta t$$

The "Leapfrog" Intuition:
It is called "Leapfrog" because the velocities and positions are offset by half a step, "leaping" over each other. This symmetry is what prevents the energy from drifting away.

CODE #118

```
def calculate_energy(pos, vel, mass, G):
    """Calculates Total Energy (KE + PE)."""
    # Kinetic Energy
    ke = 0.5 * np.sum(mass * np.sum(vel**2, axis=1, keepdims=True))

    # Potential Energy with distance check
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
    dist = np.sqrt(np.sum(diff**2, axis=-1))
    np.fill_diagonal(dist, np.inf) # Avoid self-interaction
    pe = -0.5 * G * np.sum(mass * mass.T / dist)

    return ke, pe, ke + pe

def compute_accelerations_clean(pos, mass, G, softening=0.1):
    """Vectorized acceleration with softening."""
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
    dist_sq = np.sum(diff**2, axis=-1) + softening**2
    dist_inv3 = dist_sq**(-1.5)
    np.fill_diagonal(dist_inv3, 0)
    acc = G * np.sum(diff * dist_inv3[:, :, np.newaxis] * mass.T[:, :, np.newaxis], axis=1)
    return acc

def velocity_verlet_step(pos, vel, mass, G, dt, softening):
    """Symplectic Velocity Verlet step."""
    acc_t = compute_accelerations_clean(pos, mass, G, softening)
    v_half = vel + 0.5 * acc_t * dt
    pos_next = pos + v_half * dt
    acc_next = compute_accelerations_clean(pos_next, mass, G, softening)
    v_next = v_half + 0.5 * acc_next * dt
    return pos_next, v_next
```

CODE #119

```
# Initial conditions
pos_v = np.array([[0.0, 0, 0], [10.0, 0, 0]], dtype=float)
vel_v = np.array([[0.0, 0, 0], [0.0, 10.0, 0]], dtype=float)
mass_v = np.array([[1000.0], [1.0]])
G_const, dt, steps = 1.0, 0.01, 5000

energies = []
for _ in range(steps):
    pos_v, vel_v = velocity_verlet_step(pos_v, vel_v, mass_v, G_const, dt, 0.01)
    _, _, total_e = calculate_energy(pos_v, vel_v, mass_v, G_const)
    energies.append(total_e)

# Plotting with safety check
if not energies:
    print("Error: No energy data recorded!")
else:
    plt.figure(figsize=(12, 5))
    plt.plot(energies, color='blue', label='Total Energy')

    # Securely set limits only if data exists
    e_min, e_max = min(energies), max(energies)
    if e_min != e_max:
        plt.ylim(e_min * 1.001, e_max * 0.999)

    plt.title("Numerical Stability: Energy Conservation")
    plt.xlabel("Step")
    plt.ylabel("Total Energy")
    plt.grid(True, alpha=0.3)
    plt.show()
```

MARKDOWN #120

```
### 3. The Astrophysical Perspective
In simulations of the **Solar System** or **Galactic Mergers**, we deal with billions of
years.

1. **Long-term Stability:** If we used RK4, the Earth might spiral into the Sun after a few
thousand simulated years due to energy drift. Velocity Verlet keeps orbits stable for
billions of years.
2. **Computational Cost:** It only requires **one** expensive force calculation ($a(t+\Delta
t)$) per step, whereas RK4 requires four.
3. **Time-Reversibility:** Symplectic integrators are time-reversible, which is a key
property of Newtonian gravity. This prevents "numerical entropy" from building up in the
system.
```

MARKDOWN #121

```
## Adaptive Time-Stepping: Precision during Close Encounters

In a fixed time-step simulation, $\Delta t$ is a constant. However, for a particle
experiencing high acceleration (e.g., during a near-collision), we need a much smaller
$\Delta t$ to capture the rapid change in velocity.

### 1. The Aarseth Criterion
The most common way to calculate a variable time step in N-body simulations is based on the
**acceleration ($a$) and its derivatives**. A simplified version of the Aarseth criterion
```

uses the ratio of acceleration to its jerk (rate of change of acceleration):

$$\Delta t_i = \eta \frac{|\mathbf{a}_i|}{|\mathbf{\dot{a}}_i|}$$

Or more commonly, a simpler version based on acceleration alone:
$$\Delta t_i = \eta \sqrt{\frac{\epsilon}{|\mathbf{a}_i|}}$$

Where:
- η is a safety parameter (typical values $0.01 - 0.02$).
- ϵ is the softening parameter or a distance scale.
- $|\mathbf{a}_i|$ is the magnitude of the current acceleration.

2. Time-Step Hierarchy (Power-of-Two)
To keep particles synchronized, we don't use arbitrary values for Δt_i. Instead, we use **Block Time-Steps**:
$$\Delta t_{block} = \left(\frac{1}{2} \right)^n \cdot \Delta t_{max}$$
Particles are grouped into "levels" (blocks). A particle at level n is updated twice as often as a particle at level $n-1$.

CODE #122

```
def get_acc_single(pos, mass_center, G=1.0):
    r_mag = np.linalg.norm(pos)
    return -G * mass_center * pos / r_mag**3

# --- Simulation Logic ---
m_star = 100.0
dt_fixed = 0.05
eta = 0.02

# 1. Fixed Step Simulation
pos_f, vel_f = np.array([5.0, 0.0]), np.array([0.0, 2.0])
path_fixed = []
for _ in range(1000):
    acc = get_acc_single(pos_f, m_star)
    vel_f += acc * dt_fixed
    pos_f += vel_f * dt_fixed
    path_fixed.append(pos_f.copy())
    if np.linalg.norm(pos_f) > 50: break

# 2. Adaptive Step Simulation
pos_a, vel_a = np.array([5.0, 0.0]), np.array([0.0, 2.0])
path_adaptive = []
t = 0
while t < 50:
    acc = get_acc_single(pos_a, m_star)
    # Adaptive dt: slower near high acceleration zones
    dt_adaptive = eta * np.sqrt(0.1 / np.linalg.norm(acc))
    dt_adaptive = min(dt_adaptive, 0.05)

    vel_a += acc * dt_adaptive
    pos_a += vel_a * dt_adaptive
    path_adaptive.append(pos_a.copy())
    t += dt_adaptive

path_fixed = np.array(path_fixed)
path_adaptive = np.array(path_adaptive)
```

```
# --- Visualization with Two Panels ---
plt.style.use('default')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))

# LEFT: Global View
ax1.plot(path_fixed[:,0], path_fixed[:,1], 'r--', alpha=0.4, lw=1, label='Fixed dt
(Ejected)')
ax1.plot(path_adaptive[:,0], path_adaptive[:,1], 'g-', lw=0.8, label='Adaptive dt (Stable
Orbit)')
ax1.scatter(0, 0, color='orange', s=200, label='Star', zorder=5, edgecolor='black')
ax1.set_title("Full Orbital Trajectory Comparison", fontsize=14)
ax1.legend()
ax1.grid(True, linestyle=':', alpha=0.5)
ax1.axis('equal')

# RIGHT: Zoomed View (Focus on Adaptive Path Stability)
# Plotting with very thin lines and small markers to see step density
ax2.plot(path_fixed[:,0], path_fixed[:,1], 'r--', alpha=0.6, marker='o', markersize=3,
label='Fixed dt Error')
ax2.plot(path_adaptive[:,0], path_adaptive[:,1], 'g-', lw=0.5, marker='.', markersize=1,
label='Adaptive dt Precision')
ax2.scatter(0, 0, color='orange', s=1000, edgecolor='black', zorder=5)

# Center the "Magnifier" on the adaptive periapsis (closest point to star)
# The adaptive orbit is most critical near (0,0)
zoom_range = 6.0
ax2.set_xlim(-zoom_range+2, zoom_range+2)
ax2.set_ylim(-zoom_range-2, zoom_range-2)

ax2.set_title("Zoom: Precision at Periapsis (Closest Approach)", fontsize=14)
ax2.set_xlabel("X")
ax2.set_ylabel("Y")
ax2.grid(True, linestyle=':', alpha=0.7)
ax2.legend()

plt.tight_layout()
plt.show()
```

MARKDOWN #123

```
### Computational Efficiency
Without adaptive time-stepping, we would be forced to use a tiny $\Delta t$ for the **entire
simulation** just to handle the few moments when particles are close.

**Adaptive stepping allows:**
1. **High Precision:** During "collisions" or tight orbits.
2. **High Speed:** When particles are far apart and forces change slowly, the step
increases, saving CPU cycles.
3. **Accuracy:** It prevents the "numerical explosion" where a particle gains artificial
energy and is ejected from the system due to a large integration error.
```

MARKDOWN #124

```
## IAS15: The Peak of Orbital Precision

**IAS15** is an implicit integrator based on **Gauss-Radau quadrature**. Unlike Velocity
Verlet (2nd order), IAS15 is a **15th-order** integrator, meaning its local truncation error
```

scales as $O(\Delta t^{16})$.

1. Polynomial Approximation
The core idea is to represent the acceleration $\mathbf{a}(t)$ as a polynomial of degree 14. Instead of simple slopes, it uses 7 nodes (Gauss-Radau spacing) within each time step to sample the force field.

2. The Predictor-Corrector Loop
IAS15 works in cycles:
1. **Predict:** It uses current position and velocity to guess the state at the next nodes.
2. **Evaluate:** Calculates forces at these 7 specific sub-steps.
3. **Correct:** Iteratively refines the polynomial coefficients until the acceleration at all nodes is self-consistent.

3. Convergence and Step-size Control
The algorithm automatically adjusts Δt based on how many iterations the corrector needs. If it converges too slowly, the step size is halved; if it converges instantly, the step size is increased. This allows for **nanometer-level precision** over billions of years.

CODE #125

```
def visualize_ias15_logic():
    # Hypothetical orbital arc
    t = np.linspace(0, 1, 100)
    orbit = np.sin(t * np.pi / 2) # True curved trajectory

    # 1. Simple Methods (Euler/Verlet) use 1 or 2 points
    t_simple = np.array([0, 1])
    pos_simple = np.sin(t_simple * np.pi / 2)

    # 2. IAS15 Nodes (Gauss-Radau points)
    # These are specific mathematical points that minimize integration error
    t_ias15 = np.array([0, 0.05, 0.23, 0.5, 0.77, 0.95, 1.0])
    pos_ias15 = np.sin(t_ias15 * np.pi / 2)

    plt.figure(figsize=(10, 6))
    plt.plot(t, orbit, 'k', label='True Trajectory', lw=2, alpha=0.3)

    # Draw simple linear approximation
    plt.plot(t_simple, pos_simple, 'r--o', label='Standard Step (Linear Error)')

    # Draw IAS15 Polynomial fit
    plt.scatter(t_ias15, pos_ias15, color='blue', s=80, label='IAS15 Gauss-Radau Nodes')
    # Fit a polynomial to these nodes to show the 'smooth' capture
    poly = np.poly1d(np.polyfit(t_ias15, pos_ias15, 6))
    plt.plot(t, poly(t), 'b', label='IAS15 Polynomial Fit (15th order precision)')

    plt.title("IAS15 vs. Standard Integrators: Capturing Curvature")
    plt.xlabel("Time Step (dt)")
    plt.ylabel("Orbital Position")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

visualize_ias15_logic()
```

MARKDOWN #126

```markdown
### Why IAS15 is the standard for solar system simulations:
1. **Machine Precision:** It reaches the limits of double-precision floating-point numbers ($10^{-16}$ relative error).
2. **Adaptive Complexity:** It handles both nearly-circular orbits (large steps) and extreme close encounters (tiny steps) automatically.
3. **Stability:** While not strictly symplectic, its error is so small that for the age of the Solar System (4.5 billion years), the energy drift is negligible.
```

MARKDOWN #127

```markdown
## General Relativity: Post-Newtonian (PN) Corrections

In strong gravitational fields or for high-precision orbits, Newton's law $\mathbf{a} = -G M \mathbf{r} / r^3$ is insufficient. To account for **General Relativity (GR)** without solving the complex Einstein Field Equations, we use **Post-Newtonian expansions**.

### 1. The Schwarzschild Metric Correction
The leading-order relativistic correction for a planet orbiting a sun (the Schwarzschild solution) adds an extra term to the acceleration:

$$\mathbf{a}_{GR} = -\frac{G M}{r^2} \hat{\mathbf{r}} \left( 1 + \frac{3 L^2}{r^2 c^2} \right)$$

Where:
- $L$ is the **specific angular momentum** ($|\mathbf{r} \times \mathbf{v}|$).
- $c$ is the **speed of light**.
- $r$ is the distance from the central mass.

### 2. Physical Effect: Orbital Precession
Unlike Newtonian gravity, where a 2-body orbit is a perfect, closed ellipse, the GR term causes the orbit to **precess**. This means the point of closest approach (perihelion) rotates slightly with each revolution.
For Mercury, this effect is small but measurable: **43 arcseconds per century**.
```

CODE #128

```python
def get_gr_acc(pos, vel, M, G, c):
    """Calculates Newton + 1st Order GR Correction."""
    r_vec = pos
    r_mag = np.linalg.norm(r_vec)
    L_sq = np.linalg.norm(np.cross(pos, vel))**2

    # Newton term
    acc_n = -G * M * r_vec / r_mag**3
    # GR Correction term (3L^2 / r^2 c^2)
    gr_factor = 1 + (3 * L_sq) / (r_mag**2 * c**2)

    return acc_n * gr_factor

# --- Simulation Setup ---
G, M = 1.0, 100.0
c = 50.0  # Reduced speed of light to exaggerate the effect
dt = 0.01
steps = 15000

pos = np.array([10.0, 0.0, 0.0])
```

```
vel = np.array([0.0, 2.5, 0.0]) # Elliptical orbit

path = []
for _ in range(steps):
    acc = get_gr_acc(pos, vel, M, G, c)
    # Simple Euler-Cromer for demonstration
    vel += acc * dt
    pos += vel * dt
    path.append(pos.copy())

path = np.array(path)

# --- Visualization ---
plt.figure(figsize=(10, 10))
plt.plot(path[:,0], path[:,1], 'b-', lw=0.5, alpha=0.8, label='Precessing Orbit (GR)')
plt.scatter(0, 0, color='orange', s=200, label='Sun')
plt.title("Astro-Physical Effect: Relativistic Perihelion Precession")
plt.xlabel("X")
plt.ylabel("Y")
plt.axis('equal')
plt.legend()
plt.grid(True, alpha=0.2)
plt.show()
```

MARKDOWN #129

```
### Why this matters in N-Body Simulations:
1. **The Mercury Test:** This was the first historical proof that General Relativity was correct.
2. **Black Hole Dynamics:** When simulating stars near a Supermassive Black Hole (like Sgr A* at our galaxy's center), the GR correction is mandatory. Without it, the simulated orbits would be fundamentally wrong.
3. **Gravitational Waves:** This precession is the "mild" version of what happens during a black hole binary inspiral, which eventually leads to the emission of gravitational waves.
```

MARKDOWN #130

```
# Modeling the Solar System: The N-Body Reality

To simulate the Solar System, we must treat it as a **Grand N-Body problem**. While Kepler's laws suggest planets move in fixed ellipses, the reality is that every planet pulls on every other planet (Planetary Perturbations).

### 1. The Equations of Motion
For each planet $i$, the acceleration is the sum of the Sun's massive pull plus the tiny, but significant, pulls from all other planets:
$$\mathbf{a}_i = -G M_{sun} \frac{\mathbf{r}_i}{r_i^3} + \sum_{j \neq i, sun} G m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{|\mathbf{r}_j - \mathbf{r}_i|^3}$$

### 2. Units and Scaling
Using SI units ($kg$, $m$, $s$) leads to massive numerical errors due to the scale. In astrophysics, we use **Astronomical Units (AU)**, **Solar Masses ($M_{\odot}$)**, and **Years**.
In these units, $G \approx 4\pi^2$.

### 3. Stability and Resonances
The stability of the Solar System over billions of years is a miracle of **orbital resonances** and **Conservation of Angular Momentum**. Numerical drift must be minimized
```

using **Symplectic Integrators** (like Velocity Verlet) to prevent planets from slowly spiraling away.

CODE #131

```
# 1. Constants in AU, Solar Masses, and Days
G_const = 4 * np.pi**2 / 365.25**2  # G in AU^3 / (M_sun * day^2)

# Data: { "Name": [Mass (M_sun), Distance (AU), Orbital_Velocity (AU/day)] }
solar_data = {
    "Sun":     [1.0, 0.0, 0.0],
    "Mercury": [1.66e-7, 0.387, 0.027],
    "Venus":   [2.45e-6, 0.723, 0.020],
    "Earth":   [3.00e-6, 1.000, 0.0172],
    "Mars":    [3.22e-7, 1.524, 0.0138]
}

names = list(solar_data.keys())
num_bodies = len(names)

# Initialize arrays (N, 3) for 3D
pos = np.zeros((num_bodies, 3))
vel = np.zeros((num_bodies, 3))
mass = np.zeros((num_bodies, 1))

for i, name in enumerate(names):
    m_val, r_val, v_val = solar_data[name]
    mass[i] = m_val
    pos[i] = np.array([r_val, 0.0, 0.0])  # Position on X axis
    vel[i] = np.array([0.0, v_val, 0.0])  # Velocity on Y axis

def compute_solar_acc(pos, mass, G):
    """Vectorized N-body acceleration."""
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
    dist_sq = np.sum(diff**2, axis=-1)

    # Softening and masking to avoid division by zero
    dist_inv3 = np.zeros_like(dist_sq)
    mask = dist_sq > 0
    dist_inv3[mask] = (dist_sq[mask])**(-1.5)

    acc = G * np.sum(diff * dist_inv3[:, :, np.newaxis] * mass.T[:, :, np.newaxis], axis=1)
    return acc

# 2. Simulation Loop (Velocity Verlet)
dt = 1.0  # 1 day steps
steps = 365 # 1 year
history = np.zeros((steps, num_bodies, 3))

curr_pos = pos.copy()
curr_vel = vel.copy()

for s in range(steps):
    # Velocity Verlet
    a_t = compute_solar_acc(curr_pos, mass, G_const)
    v_half = curr_vel + 0.5 * a_t * dt
    curr_pos += v_half * dt
```

```
    a_next = compute_solar_acc(curr_pos, mass, G_const)
    curr_vel = v_half + 0.5 * a_next * dt

    history[s] = curr_pos.copy()

# 3. Visualization
plt.figure(figsize=(10, 10))
colors = ['gold', 'gray', 'orange', 'blue', 'red']

for i in range(num_bodies):
    plt.plot(history[:, i, 0], history[:, i, 1], color=colors[i], label=names[i], lw=1.5)
    plt.scatter(history[-1, i, 0], history[-1, i, 1], color=colors[i], s=100 if i==0 else
30)

plt.style.use('default')
plt.title("Numerical Simulation of the Inner Solar System (1 Year)")
plt.xlabel("Distance (AU)")
plt.ylabel("Distance (AU)")
plt.legend(loc='upper right')
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.show()
```

MARKDOWN #132

```
## 1. Special Relativity (SR): The Velocity Limit
In Newtonian mechanics, velocities add up linearly ($v = v_1 + v_2$). In SR, the speed of
light ($c$) is an unbreakable limit. This introduces the **Lorentz Factor ($\gamma$)**:

$$\gamma = \frac{1}{\sqrt{1 - \frac{v^2}{c^2}}}$$

### Impact on N-Body Math:
*   **Relativistic Mass:** As particles accelerate, their "effective mass" increases:
$m_{rel} = \gamma m_0$. In high-energy simulations (like plasma or particle accelerators),
this prevents particles from exceeding $c$.
*   **Time Dilation:** For a fast-moving particle, the simulation time step $dt$ must be
adjusted. The particle "experiences" less time than the laboratory observer: $dt' = dt /
\gamma$.
```

MARKDOWN #133

```
## 2. General Relativity (GR): The Geometry of Force
The biggest mathematical change is that gravity is no longer a force vector $\mathbf{F}$,
but a **Metric Tensor $g_{\mu\nu}$**.

### The Geodesic Equation:
Instead of $\mathbf{a} = \mathbf{F}/m$, particles follow "geodesics" (the shortest path in
curved spacetime). The equation of motion becomes:
$$\frac{d^2 x^\mu}{d \tau^2} + \Gamma^\alpha_{\beta\gamma} \frac{dx^\beta}{d\tau}
\frac{dx^\gamma}{d\tau} = 0$$

- **$\Gamma^\alpha_{\beta\gamma}$ (Christoffel Symbols):** These represent the
"gravitational field" and are derived from the derivatives of the metric.
- **Non-linearity:** Unlike Newton's gravity, where you can simply sum the forces
($F_{total} = F_1 + F_2$), GR is non-linear. The presence of mass $A$ and $B$ curves space
in a way that isn't just the sum of $A+B$. This is why the 2-body problem in GR is much
harder than in Newton's physics.
```

MARKDOWN #134

```markdown
### 3. Practical Implementation: The PN Approximation
Since solving the full Einstein equations is computationally impossible for many bodies, we use the **Post-Newtonian Expansion**. We add "correction terms" to the Newtonian potential:

$$\Phi_{total} = \Phi_{Newton} + \frac{1}{c^2} \Phi_{PN1} + \frac{1}{c^4} \Phi_{PN2} + \dots$$

- **Energy Loss:** At higher orders ($2.5PN$), the math accounts for **Gravitational Waves**. The system loses energy as waves, causing orbits (like binary black holes) to shrink and eventually merge. This is impossible to model with pure Newtonian math.
```

CODE #135

```
# 1. Setup Parameters
c_rel = 40.0
G_const, M_sun = 1.0, 100.0
dt_step = 0.02
total_steps = 3500

def get_acceleration(p, v, is_relativistic=False):
    r_mag = np.linalg.norm(p)
    acc_newton = -G_const * M_sun * p / r_mag**3
    if not is_relativistic:
        return acc_newton

    # 1PN correction for precession
    l_sq = np.linalg.norm(np.cross(p, v))**2
    correction = 1 + (3 * l_sq) / (r_mag**2 * c_rel**2)
    return acc_newton * correction

# 2. Run Simulations
pos_n, vel_n = np.array([10.0, 0.0, 0.0]), np.array([0.0, 2.5, 0.0])
pos_r, vel_r = pos_n.copy(), vel_n.copy()
path_n, path_r = [], []

for _ in range(total_steps):
    a_n = get_acceleration(pos_n, vel_n, False)
    vel_n += a_n * dt_step; pos_n += vel_n * dt_step
    path_n.append(pos_n.copy())

    a_r = get_acceleration(pos_r, vel_r, True)
    vel_r += a_r * dt_step; pos_r += vel_r * dt_step
    path_r.append(pos_r.copy())

path_n, path_r = np.array(path_n), np.array(path_r)

# 3. Calculate Deviation (Euclidean distance between Newton and Einstein at each step)
deviation = np.linalg.norm(path_n - path_r, axis=1)
time_axis = np.arange(total_steps) * dt_step

# 4. Visualization
plt.style.use('default')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# --- LEFT: Visual Orbits ---
ax1.plot(path_n[:,0], path_n[:,1], 'k--', lw=1, label='Newton (Classical)', alpha=0.4)
```

```
ax1.plot(path_r[:,0], path_r[:,1], 'r-', lw=1.2, label='Einstein (Relativistic)')
ax1.scatter(0, 0, color='orange', s=200, label='Sun', edgecolors='black')
ax1.set_title("Orbital Precession: Newton vs. Einstein", fontsize=14)
ax1.set_xlabel("X Distance")
ax1.set_ylabel("Y Distance")
ax1.axis('equal')
ax1.grid(True, linestyle=':', alpha=0.5)
ax1.legend()

# --- RIGHT: Deviation Statistics ---
ax2.plot(time_axis, deviation, color='darkred', lw=2)
ax2.fill_between(time_axis, deviation, color='red', alpha=0.1)
ax2.set_title("Divergence over Time (Classical Error)", fontsize=14)
ax2.set_xlabel("Simulation Time (dt steps)")
ax2.set_ylabel("Euclidean Distance (Gap Size)")
ax2.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()
```

MARKDOWN #136

```
# Dark Matter Dynamics in N-Body Simulations

Dark matter makes up about 85% of the total matter in the universe. In simulations, we model
it as **Collisionless Cold Dark Matter (CDM)**.

### 1. The Collisionless Boltzmann Equation (CBE)
Unlike gas, dark matter particles do not "bounce" off each other. Their distribution
function $f(\mathbf{r}, \mathbf{v}, t)$ evolves according to the CBE:
$$\frac{\partial f}{\partial t} + \mathbf{v} \cdot \nabla f - \nabla \Phi \cdot
\frac{\partial f}{\partial \mathbf{v}} = 0$$

Where the gravitational potential $\Phi$ is determined by the **Poisson Equation**:
$$\nabla^2 \Phi = 4\pi G \rho$$

### 2. Implementation: Macro-Particles
Since we cannot simulate every single subatomic particle, we use **"Macro-particles"**. Each
point in our simulation represents a massive "cloud" of dark matter (often $10^5$ to
$10^{10}$ solar masses).
To prevent unphysical scattering when these massive clouds pass through each other, we use a
larger **Softening Parameter ($\epsilon$)**:
$$\mathbf{a}_i = \sum G \frac{m_j}{(\mathbf{r}_{ij}^2 + \epsilon^2)^{3/2}} \mathbf{r}_{ij}$$
```

CODE #137

```
# 1. Initialization (from your logic)
N_DM = 5000 # Increased for better density resolution
radius = 20

def generate_dark_matter_halo(n_particles, radius):
    """Generates a Navarro-Frenk-White (NFW) like density profile."""
    r = radius * (np.random.rand(n_particles)**2)
    phi = np.random.rand(n_particles) * 2 * np.pi
    theta = np.arccos(2 * np.random.rand(n_particles) - 1)
    return np.stack([
        r * np.sin(theta) * np.cos(phi),
        r * np.sin(theta) * np.sin(phi),
```

```
        r * np.cos(theta)
    ], axis=1)

dm_pos = generate_dark_matter_halo(N_DM, radius)
r_distances = np.linalg.norm(dm_pos, axis=1)

# 2. Dual-Panel Visualization
plt.style.use('dark_background')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))

# --- LEFT: Density Heatmap (The Scaffold) ---
hb = ax1.hexbin(dm_pos[:, 0], dm_pos[:, 1], gridsize=40, cmap='magma', bins='log', mincnt=1)
ax1.set_title("Dark Matter Density Map (Log Scale)", fontsize=14)
ax1.set_aspect('equal')

# Overlay visible stars
stars_x = np.random.normal(0, 1.2, 100)
stars_y = np.random.normal(0, 1.2, 100)
ax1.scatter(stars_x, stars_y, s=2, color='white', alpha=0.8, label='Visible Galaxy')
ax1.legend(loc='upper right')

# --- RIGHT: Radial Profile (The Physics) ---
ax2.hist(r_distances, bins=50, color='purple', alpha=0.6, density=True, label='Simulated
NFW')
# Theoretical NFW-like curve (1/r prop)
r_line = np.linspace(0.1, radius, 100)
nfw_curve = 1 / (r_line * (1 + r_line)**2) # Simplified NFW shape
ax2.plot(r_line, nfw_curve * (max(ax2.get_ylim()) / max(nfw_curve)), 'w--', lw=2,
label='Theoretical Profile')

ax2.set_title("Radial Mass Distribution", fontsize=14)
ax2.set_xlabel("Radius from Center (kpc)")
ax2.set_ylabel("Relative Density")
ax2.legend()
ax2.grid(True, alpha=0.1)

plt.tight_layout()
plt.show()
```

MARKDOWN #138

```
### 3. The "Missing Mass" Problem
The most direct mathematical impact of dark matter is on the **Galactic Rotation Curves**.
- **Newtonian (Visible only):** Orbital velocity should drop as $v \propto 1/\sqrt{r}$ far
from the center.
- **Observed (with Dark Matter):** The velocity curve stays **flat**.

This proves that there is an invisible mass (Dark Matter) extending far beyond the visible
stars, providing the extra gravity needed to keep outer stars moving fast without flying
away.
```

MARKDOWN #139

```
# Dark Energy: The Accelerated Expansion

Dark energy acts as a **Cosmological Constant ($\Lambda$)**, providing a constant energy
density that fills space uniformly. In N-body simulations, its primary role is to drive the
**Hubble Expansion**.
```

1. The Modified Force Law
In an expanding universe, the physical distance $\mathbf{r}$ is related to the "comoving" distance $\mathbf{x}$ by the scale factor $a(t)$: $\mathbf{r} = a(t)\mathbf{x}$.

The acceleration of a particle i in a system with Dark Energy is modified by an extra term:
$$\mathbf{a}_i = \sum_{j \neq i} G \frac{m_j}{\mathbf{r}_{ij}^3} \mathbf{r}_{ij} + \frac{\Lambda c^2}{3} \mathbf{r}_i$$

- **The First Term:** Standard Newtonian gravity (attractive).
- **The Second Term:** Dark Energy repulsion (repulsive).

As the distance r increases, the repulsive term eventually dominates, causing galaxies to accelerate away from each other.

CODE #140

```
def simulate_dark_energy(steps, dt, lambda_const=0.01):
    # Two galaxy clusters
    pos = np.array([[-5.0, 0.0], [5.0, 0.0]])
    vel = np.array([[0.1, 0.0], [-0.1, 0.0]]) # Moving slightly toward each other
    mass = np.array([100.0, 100.0])
    G = 1.0

    history = []
    for _ in range(steps):
        r_vec = pos[1] - pos[0]
        r_mag = np.linalg.norm(r_vec)

        # 1. Attractive Gravity
        acc_g = G * mass[1] * r_vec / r_mag**3

        # 2. Repulsive Dark Energy (proportional to distance)
        acc_de = lambda_const * r_vec

        # Total acceleration for body 0 (body 1 is opposite)
        total_acc = acc_g - acc_de

        vel[0] += total_acc * dt
        vel[1] -= total_acc * dt
        pos[0] += vel[0] * dt
        pos[1] += vel[1] * dt
        history.append(pos.copy())

    return np.array(history)

# Run and Plot
history = simulate_dark_energy(1000, 0.1, lambda_const=0.005)

plt.figure(figsize=(10, 5))
plt.plot(history[:, 0, 0], label='Cluster A')
plt.plot(history[:, 1, 0], label='Cluster B')
plt.axhline(0, color='gray', linestyle='--')
plt.title("Gravity vs. Dark Energy: The Battle for Expansion")
plt.xlabel("Time Step")
plt.ylabel("X Position")
```

```
plt.legend()
plt.show()
```

MARKDOWN #141

```
### 3. The Impact on Structure Formation
Dark energy determines the **"Turn-around Radius"**. This is the point where the inward pull
of gravity exactly balances the outward push of dark energy.
- Objects closer than this radius will merge into galaxies.
- Objects further away will be swept apart by the expansion of the universe forever.

In large-scale N-body simulations (like the Millennium Simulation), dark energy is what
creates the **Cosmic Web** structure, preventing the entire universe from collapsing into a
single giant black hole.
```

MARKDOWN #142

```
## The $\Lambda$CDM Model: Gravity vs. Expansion

In a cosmological N-body simulation, we model the competition between **Dark Matter (DM)**,
which promotes structure formation through gravitational collapse, and **Dark Energy
($\Lambda$)**, which drives the accelerated expansion of the universe.

### 1. The Comoving Coordinate System
To handle the expanding universe, we use **comoving coordinates** $\mathbf{x}$. The physical
distance is $\mathbf{r} = a(t)\mathbf{x}$, where $a(t)$ is the **scale factor**.

### 2. The Modified Equation of Motion
The acceleration of a dark matter particle $i$ in an expanding background dominated by dark
energy is:
$$\mathbf{\ddot{x}}_i + 2H\mathbf{\dot{x}}_i = \frac{1}{a^3} \sum_{j \neq i} \frac{G m_j
(\mathbf{x}_j - \mathbf{x}_i)}{|\mathbf{x}_j - \mathbf{x}_i|^3} - \frac{1}{a^2} \nabla
\Phi_{ext}$$

Where:
- **$2H\mathbf{\dot{x}}_i$ (Hubble Drag):** A "friction" term caused by the expansion of
space ($H$ is the Hubble parameter).
- **The Summation:** Newtonian gravity (Dark Matter pull).
- **Dark Energy Impact:** Incorporated into the evolution of $a(t)$ via the Friedmann
equations.

### 3. The Friedmann Equation
The expansion rate $H = \dot{a}/a$ is determined by the density of DM ($\Omega_m$) and DE
($\Omega_\Lambda$):
$$H^2(a) = H_0^2 \left( \frac{\Omega_m}{a^3} + \Omega_\Lambda \right)$$
```

CODE #143

```
def simulate_cosmic_structure(n_particles=1000, steps=100, dt=0.1):
    # Initial state: Nearly uniform distribution (early universe)
    pos = np.random.uniform(-10, 10, (n_particles, 2))
    vel = np.zeros_like(pos)
    hubble_drag = 0.05
    dark_energy_push = 0.002

    for _ in range(steps):
        # 1. Gravity calculation (simplified N-body)
        diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
```

```
        dist_sq = np.sum(diff**2, axis=-1) + 0.5
        acc_g = np.sum(diff * (dist_sq**(-1.5))[:, :, np.newaxis], axis=1)

        # 2. Dark Energy & Expansion
        acc_de = dark_energy_push * pos
        vel = vel * (1 - hubble_drag) + (acc_g + acc_de) * dt
        pos += vel * dt
    return pos

# 1. Execute Simulation
n_p = 800
final_pos = simulate_cosmic_structure(n_particles=n_p, steps=60)

# 2. Statistical Analysis: Nearest Neighbor Distances
# This measures clumping vs uniformity
tree = cKDTree(final_pos)
distances, _ = tree.query(final_pos, k=2) # k=2 because k=1 is the particle itself
nn_distances = distances[:, 1]

# 3. Visualization
plt.style.use('dark_background')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 8))

# --- LEFT: The Cosmic Web ---
ax1.scatter(final_pos[:, 0], final_pos[:, 1], s=3, color='cyan', alpha=0.7)
ax1.set_title("$\Lambda$CDM: Dark Matter Filaments", fontsize=15)
ax1.set_aspect('equal')
ax1.axis('off')

# --- RIGHT: Clustering Statistics ---
ax2.hist(nn_distances, bins=30, color='magenta', alpha=0.5, edgecolor='white')
ax2.axvline(np.mean(nn_distances), color='yellow', linestyle='--', label=f'Mean Distance:
{np.mean(nn_distances):.2f}')
ax2.set_title("Clustering Statistics: Nearest Neighbor Distribution", fontsize=15)
ax2.set_xlabel("Distance to Nearest Neighbor")
ax2.set_ylabel("Number of Particles")
ax2.grid(True, alpha=0.1)
ax2.legend()

# Add interpretation text
ax2.annotate('High peak at low distance = Heavy Clumping\n(Filaments & Nodes)',
             xy=(np.mean(nn_distances), n_p/10), xytext=(np.mean(nn_distances)*2, n_p/5),
             arrowprops=dict(facecolor='white', shrink=0.05, width=1))

plt.tight_layout()
plt.show()
```

MARKDOWN #144

```
### Analysis: The Large-Scale Structure
This simulation demonstrates the dual nature of our universe:
1. **Clumping (Dark Matter):** Gravity wins locally, creating the "Cosmic Web", a network of
filaments where galaxies form.
2. **Expansion (Dark Energy):** On large scales, space itself expands, stretching the web
and creating vast, empty "Voids".

**Numerical Impact:** In professional codes like **IllustrisTNG**, these equations are
```

solved for billions of particles. Without both DM and DE, the simulated universe would not match the distribution of galaxies observed by telescopes like Hubble and James Webb.

MARKDOWN #145

```markdown
## Cosmic Inflation: The Exponential Expansion

Cosmic Inflation is a theory proposing that the early universe expanded by a factor of at least $10^{26}$ in an incredibly short time. This process is driven by a hypothetical scalar field called the **Inflaton ($\phi$)**.

### 1. The Expansion Law
During inflation, the scale factor $a(t)$ grows exponentially:
$$a(t) \propto e^{Ht}$$
Where $H$ is the Hubble parameter, which remains nearly constant during this phase.

### 2. Quantum Fluctuations to Cosmic Structure
The most critical mathematical impact of inflation is the transformation of **quantum fluctuations** into **classical density perturbations**.
- According to the **Heisenberg Uncertainty Principle**, the inflaton field $\phi$ has tiny random fluctuations $\delta \phi$.
- These fluctuations are "stretched" by the exponential expansion to cosmic scales, creating a power spectrum of density variations:
$$P(k) \propto k^{n_s - 1}$$
Where $k$ is the wavenumber and $n_s \approx 0.96$ is the scalar spectral index. These variations are the "seeds" for all future N-body simulations.
```

CODE #146

```
def simulate_inflation_stretch(n_points=200, expansion_factor=50):
    # 1. Initial State: Tiny quantum fluctuations (random noise in a small space)
    initial_radius = 0.1
    pos_initial = np.random.normal(0, initial_radius, (n_points, 2))

    # 2. Inflation: Exponential stretch
    # We apply a massive scaling to the coordinates
    pos_inflated = pos_initial * expansion_factor

    # 3. Visualization
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    # Pre-Inflation Plot
    ax1.scatter(pos_initial[:, 0], pos_initial[:, 1], s=10, color='cyan', alpha=0.6)
    ax1.set_xlim(-5, 5); ax1.set_ylim(-5, 5)
    ax1.set_title("Pre-Inflation: Quantum Fluctuations (Microscale)")
    ax1.set_aspect('equal')

    # Post-Inflation Plot
    ax2.scatter(pos_inflated[:, 0], pos_inflated[:, 1], s=10, color='magenta', alpha=0.6)
    ax2.set_xlim(-10, 10); ax2.set_ylim(-10, 10) # Zoomed out but structure is visible
    ax2.set_title(f"Post-Inflation: Large Scale Structure Seeds (Factor
x{expansion_factor})")
    ax2.set_aspect('equal')

    plt.tight_layout()
    plt.show()

simulate_inflation_stretch()
```

MARKDOWN #147

```markdown
### 3. How Inflation Seeds the N-Body Problem
Without inflation, an N-body simulation would start with a perfectly uniform distribution of matter, and gravity would have no "clumps" to work with.

**The chain of causality is:**
1. **Inflation** generates Gaussian random fluctuations.
2. These fluctuations appear in the **Cosmic Microwave Background (CMB)**.
3. Dark Matter particles begin to fall into these slightly denser regions (**Gravitational Instability**).
4. The **N-Body Algorithms** (like Barnes-Hut or FMM) track how these seeds grow into the galaxies and clusters we see today.

Mathematically, we use the **Transfer Function** to evolve the inflationary seeds to the starting point of our N-body simulation (typically around redshift $z=100$).
```

MARKDOWN #148

```markdown
## 1.D Cosmology: Hubble Expansion vs. Gravity

In a 1D expanding universe, we use **Comoving Coordinates** ($x$). The physical distance ($r$) at any time ($t$) is:
$$r(t) = a(t) \cdot x$$
Where $a(t)$ is the **Scale Factor**, which grows as the universe expands.

### The Equation of Motion in 1D:
For a particle $i$ in this expanding background, the acceleration is:
$$\ddot{x}_i + 2H\dot{x}_i = \frac{G}{a^3} \sum_{j \neq i} m_j \cdot \text{sgn}(x_j - x_i)$$

1. **Hubble Drag ($2H\dot{x}_i$):** As space expands, particles experience a "damping" force that tries to freeze them in their comoving positions. $H = \dot{a}/a$ is the Hubble parameter.
2. **Gravitational Pull:** Particles attract each other, trying to overcome the expansion to form clusters.
3. **The Big Bang Initial Conditions:** We start with particles almost uniformly spaced, but with tiny "quantum-like" displacements (seeds) generated by inflation.
```

CODE #149

```
# 1. Setup Parameters
N_PARTICLES = 100
STEPS = 500
DT = 0.1
G = 0.1
HUBBLE_DRAG = 0.1  # Simulated expansion rate

# 2. Initial Conditions (Seeds from Inflation)
# Nearly uniform grid with tiny random perturbations
x_init = np.linspace(-50, 50, N_PARTICLES)
x_init += np.random.normal(0, 0.5, N_PARTICLES)
vel = np.zeros(N_PARTICLES)

# 3. Simulation Loop
history = np.zeros((STEPS, N_PARTICLES))
curr_x = x_init.copy()

for s in range(STEPS):
```

```
    # Calculate 1D Gravity (Direct Summation)
    acc = np.zeros(N_PARTICLES)
    for i in range(N_PARTICLES):
        diff = curr_x - curr_x[i]
        # 1D Gravity: Force is constant regardless of distance, only direction matters
        # But we add a small decay/softening for stability
        acc[i] = G * np.sum(np.sign(diff) * np.exp(-np.abs(diff)/10))

    # Update with Hubble Drag (Expansion effect)
    vel = vel * (1 - HUBBLE_DRAG * DT) + acc * DT
    curr_x += vel * DT
    history[s] = curr_x.copy()

# 4. Visualization: Space-Time Diagram
plt.figure(figsize=(12, 8))
time_axis = np.linspace(0, STEPS * DT, STEPS)

for i in range(N_PARTICLES):
    plt.plot(history[:, i], time_axis, color='white', lw=0.5, alpha=0.6)

plt.gca().set_facecolor('black')
plt.title("1D Universe Evolution: From Big Bang Seeds to Structure Formation")
plt.xlabel("Physical Position (X)")
plt.ylabel("Time (Evolution ->)")
plt.grid(False)
plt.show()
```

MARKDOWN #150

```
### Observations from the Space-Time Plot:
- **Vertical Lines:** Represent particles that are relatively "frozen" in space due to expansion.
- **Converging Lines (Filaments):** You can see groups of particles "falling" into each other. These are the **1D Dark Matter Filaments**.
- **Empty Zones (Voids):** The regions where particles have left to join clusters. This is exactly how the large-scale structure of the universe looks!

**Mathematical Conclusion:**
The final structure is a delicate balance between the **Initial Fluctuation Amplitude** (how big the seeds were) and the **Hubble Expansion Rate**. If expansion were too fast, no structures would ever form. If it were too slow, the entire 1D universe would have collapsed into a single point.
```

MARKDOWN #151

```
<div class="alert alert-block alert-warning">
        < b >& þ    S c i e n t i f i c   D i s c l a i m e r   ( M a r c h   2 0 2 6 ) < / b >
    <br>
    While these simulations are based on gravity, it is important to note that the <b>Graviton</b> (the hypothetical elementary particle that would mediate the force of gravity in quantum field theory) remains <b>experimentally unverified</b>. As of 2026, there is still no direct empirical evidence for its existence.
</div>
```

MARKDOWN #152

```
## From Fields to Particles: The Graviton Exchange Model
```

In Classical Mechanics, gravity is a continuous field. In Quantum Field Theory (QFT), forces are mediated by the exchange of **gauge bosons**. For gravity, this hypothetical particle is the **Graviton**.

1. Momentum Transfer (Δp)
When Particle A emits a graviton, it recoils due to the conservation of momentum. When Particle B absorbs the graviton, it receives that momentum.
$$\Delta \mathbf{p} = \frac{E_{graviton}}{c} \cdot \hat{\mathbf{r}}$$

2. The Inverse Square Law Emergence
How do discrete particles recreate $1/r^2$?
- The probability of a graviton hitting Particle B decreases as the surface area of the sphere ($4\pi r^2$) increases.
- Thus, the frequency of "hits" is proportional to $1/r^2$, effectively simulating Newton's Law through discrete events.

3. The Simulation Logic
Instead of a constant force, we calculate a **probability of exchange** at each time step. If an exchange occurs, we apply a discrete "kick" (impulse) to both particles.

CODE #153

```
# 1. Setup Parameters
np.random.seed(42)
pos = np.array([[-5.0, 0.0], [5.0, 0.0]])
vel = np.array([[0.0, 0.0], [0.0, 0.0]])
mass = np.array([10.0, 10.0])
G_quantum = 50.0  # Strength of a single graviton "kick"
dt = 0.05
steps = 400

history = []
gravitron_events = [] # To store when and where an exchange happened

# 2. Simulation Loop
curr_pos = pos.copy()
curr_vel = vel.copy()

for s in range(steps):
    r_vec = curr_pos[1] - curr_pos[0]
    dist = np.linalg.norm(r_vec)

    # Probability of graviton exchange (1/r^2)
    prob = (G_quantum / (dist**2 + 1.0)) * dt

    if np.random.rand() < prob:
        # Graviton Exchange Event!
        unit_vec = r_vec / dist
        # Particle 0 gets kicked towards Particle 1
        curr_vel[0] += (G_quantum / mass[0]) * unit_vec * dt
        # Particle 1 gets kicked towards Particle 0
        curr_vel[1] -= (G_quantum / mass[1]) * unit_vec * dt
        gravitron_events.append((s, curr_pos[0].copy(), curr_pos[1].copy()))

    curr_pos += curr_vel * dt
    history.append(curr_pos.copy())
```

```
history = np.array(history)
```

CODE #154

```
fig, ax = plt.subplots(figsize=(8, 6))
ax.set_facecolor('black')
ax.set_xlim(-10, 10); ax.set_ylim(-5, 5)

line1, = ax.plot([], [], 'o', color='cyan', markersize=15, label='Mass A')
line2, = ax.plot([], [], 'o', color='magenta', markersize=15, label='Mass B')
grav_line, = ax.plot([], [], '--', color='yellow', alpha=0) # The Graviton path

def update(frame):
    p = history[frame]
    line1.set_data([p[0,0]], [p[0,1]])
    line2.set_data([p[1,0]], [p[1,1]])

    # Check if a graviton was exchanged in this frame
    event = [e for e in gravitron_events if e[0] == frame]
    if event:
        grav_line.set_data([event[0][1][0], event[0][2][0]], [event[0][1][1],
event[0][2][1]])
        grav_line.set_alpha(1.0)
    else:
        grav_line.set_alpha(0)

    return line1, line2, grav_line

ani = FuncAnimation(fig, update, frames=steps, interval=30, blit=True)
plt.close()
HTML(ani.to_jshtml())
```

MARKDOWN #155

```
## 3-Body Dynamics via Discrete Graviton Exchange

In a 3-body system, the continuous gravitational field is replaced by a stochastic (random)
process of particle exchange. Each pair of masses $(i, j)$ has a specific probability of
interacting at any given moment.

### 1. Exchange Probability Matrix
For $N=3$, we define a probability $P_{ij}$ for each pair based on their instantaneous
distance $r_{ij}$:
$$P_{ij} = \frac{\Gamma \cdot m_i \cdot m_j}{r_{ij}^2 + \epsilon^2} \Delta t$$
Where $\Gamma$ is the "coupling constant" (the strength of the quantum interaction).

### 2. Discrete Momentum Kicks (Impulse)
When an exchange occurs between body $i$ and body $j$:
- Body $i$ receives an impulse: $\Delta \mathbf{v}_i = \frac{\mathbf{p}_{graviton}}{m_i}
\hat{\mathbf{r}}_{ij}$
- Body $j$ receives an equal and opposite impulse: $\Delta \mathbf{v}_j =
-\frac{\mathbf{p}_{graviton}}{m_j} \hat{\mathbf{r}}_{ij}$

### 3. Emergent Chaos
Because the exchanges are discrete and random, a 3-body system in this model is even more
chaotic than the Newtonian version. Small "lucky" streaks of graviton hits can completely
change the orbital trajectory, illustrating the **Quantum Uncertainty** at a macroscopic
scale.
```

CODE #156

```
def simulate_quantum_instance(steps, dt, gamma=2.0, impulse=0.5):
    """Internal stochastic engine for the 3-body quantum model."""
    pos = np.array([[0.0, 5.0], [-4.0, -3.0], [4.0, -3.0]])
    vel = np.zeros((3, 2))
    mass = np.array([10.0, 10.0, 10.0])
    history = []
    hits = 0

    for _ in range(steps):
        for i in range(3):
            for j in range(i + 1, 3):
                r_vec = pos[j] - pos[i]
                dist = np.linalg.norm(r_vec)
                prob = (gamma * mass[i] * mass[j] / (dist**2 + 1.0)) * dt

                if np.random.rand() < prob:
                    unit_vec = r_vec / dist
                    vel[i] += (impulse / mass[i]) * unit_vec
                    vel[j] -= (impulse / mass[j]) * unit_vec
                    hits += 1

        pos += vel * dt
        history.append(pos.copy())
    return np.array(history), hits

# --- Plotting Configuration ---
steps_q, dt_q = 2000, 0.05
colors = ['#00FFCC', '#FF00FF', '#FFFF00']

# Create a 2x3 grid with a square figsize
fig, axes = plt.subplots(2, 3, figsize=(12, 8)) # Adjusted height for better fit
fig.patch.set_facecolor('#111111')

for idx, ax in enumerate(axes.flatten()):
    traj, hit_count = simulate_quantum_instance(steps_q, dt_q)
    ax.set_facecolor('#050505')

    for i in range(3):
        ax.plot(traj[:, i, 0], traj[:, i, 1], color=colors[i], lw=0.7, alpha=0.6)
        ax.scatter(traj[-1, i, 0], traj[-1, i, 1], color=colors[i], s=30)

    # Title moved slightly closer to the plot
    ax.set_title(f"Run {idx+1}: {hit_count} Gravitons", color='white', fontsize=10, pad=5)

    limit = 25
    ax.set_xlim(-limit, limit)
    ax.set_ylim(-limit, limit)
    ax.set_aspect('equal')

    for spine in ax.spines.values():
        spine.set_edgecolor('white')
        spine.set_linewidth(1.0)

    ax.set_xticks([])
    ax.set_yticks([])
```

```
plt.suptitle("Quantum Gravitational Chaos",
             color='white', fontsize=18, x=0.42, y=0.88)

plt.subplots_adjust(wspace=0.02, hspace=0.15, top=0.80, bottom=0.05, left=0.05, right=0.80)

plt.show()
```

MARKDOWN #157

```
## Comparison: Deterministic Field vs. Stochastic Particles

In this module, we compare two fundamentally different mathematical views of the 3-body
problem:

### 1. Direct Summation (Classical)
Gravity is modeled as a **continuous field**. At every infinitesimal point in time, a force
exists between all bodies.
- **Mathematical Form:** $\mathbf{F}_{ij} = G \frac{m_i m_j}{r_{ij}^2}
\hat{\mathbf{r}}_{ij}$
- **Result:** Smooth, predictable (though chaotic) orbits.

### 2. Graviton Exchange (Quantum-inspired)
Gravity is modeled as a **stochastic (random) process**. There is no "force" between events,
only discrete "kicks" (impulses) when a graviton is exchanged.
- **Mathematical Form:** $P(\text{exchange}) = \frac{\Gamma \cdot m_i m_j}{r_{ij}^2} \Delta
t$
- **Result:** "Jittery" motion that approximates the classical orbit only when the number of
exchanges is very high (The Correspondence Principle).
```

CODE #158

```
def run_dual_comparison(steps, dt, G=5.0, gamma=20.0, impulse=0.5):
    # Initial Conditions (Same for both)
    pos_c = np.array([[0.0, 4.0], [-3.0, -2.0], [3.0, -2.0]])
    vel_c = np.zeros((3, 2))
    pos_q, vel_q = pos_c.copy(), vel_c.copy()
    mass = np.array([10.0, 10.0, 10.0])

    hist_c, hist_q = [], []
    total_exchanges = 0

    for s in range(steps):
        # --- 1. Classical Direct Summation ---
        acc_c = np.zeros((3, 2))
        for i in range(3):
            for j in range(3):
                if i == j: continue
                r_vec = pos_c[j] - pos_c[i]
                dist = np.linalg.norm(r_vec) + 0.1
                acc_c[i] += G * mass[j] * r_vec / dist**3

        vel_c += acc_c * dt
        pos_c += vel_c * dt
        hist_c.append(pos_c.copy())

        # --- 2. Quantum Graviton Exchange ---
        for i in range(3):
```

```
            for j in range(i + 1, 3):
                r_vec = pos_q[j] - pos_q[i]
                dist = np.linalg.norm(r_vec) + 0.1
                prob = (gamma * mass[i] * mass[j] / dist**2) * dt

                if np.random.rand() < prob:
                    unit_vec = r_vec / dist
                    vel_q[i] += (impulse / mass[i]) * unit_vec
                    vel_q[j] -= (impulse / mass[j]) * unit_vec
                    total_exchanges += 1

        pos_q += vel_q * dt
        hist_q.append(pos_q.copy())

    return np.array(hist_c), np.array(hist_q), total_exchanges

# Execute
steps = 3000
dt = 0.02
traj_c, traj_q, count = run_dual_comparison(steps, dt)
```

CODE #159

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))
ax1.set_facecolor('#0a0a0a'); ax2.set_facecolor('#0a0a0a')

colors = ['#00FFCC', '#FF00FF', '#FFFF00']

# Plot Classical
for i in range(3):
    ax1.plot(traj_c[:, i, 0], traj_c[:, i, 1], color=colors[i], lw=1.5, label=f'Body {i}')
ax1.set_title("Classical Direct Summation (Continuous Field)")
ax1.legend()

# Plot Quantum
for i in range(3):
    ax2.plot(traj_q[:, i, 0], traj_q[:, i, 1], color=colors[i], lw=1, alpha=0.8)
ax2.set_title(f"Quantum Exchange Model ({count} Gravitons)")

for ax in [ax1, ax2]:
    ax.axis('equal')
    ax.grid(True, color='gray', alpha=0.1)

plt.tight_layout()
plt.show()
```

MARKDOWN #160

```
## Statistical Analysis: Classical Field vs. Quantum Stochasticity

To compare these two models, we look at the **Mean Squared Displacement (MSD)** and the
**Energy Fluctuation ($\Delta E$)**.

### 1. The Source of Divergence
In **Direct Summation**, the force is calculated at every $dt$, leading to a smooth path. In
the **Graviton Model**, the force is a series of Poisson events. Even if the *average* force
is the same, the instantaneous variance is high:
$$\sigma_{Force}^2 \propto \frac{1}{\text{Exchange Rate}}$$
```

2. Statistical Metrics
- **Trajectory Deviation:** We measure the Euclidean distance between the same particle in both simulations over time.
- **Energy Stability:** Classical systems should conserve energy better than the discrete graviton model, where each "kick" introduces a small numerical shock.

CODE #161

```
def run_statistical_comparison(steps, dt, G=5.0, gamma=50.0, impulse=0.2):
    # Initial Conditions (Identical for both)
    pos_c = np.array([[0.0, 3.0], [-2.0, -1.0], [2.0, -1.0]])
    vel_c = np.array([[0.5, 0.0], [0.0, -0.5], [-0.5, 0.5]])
    pos_q, vel_q = pos_c.copy(), vel_c.copy()
    mass = np.array([10.0, 10.0, 10.0])

    hist_c, hist_q = [], []
    deviations = [] # Distance between classical and quantum particle 0

    for s in range(steps):
        # --- Classical Step ---
        acc_c = np.zeros((3, 2))
        for i in range(3):
            for j in range(3):
                if i == j: continue
                diff = pos_c[j] - pos_c[i]
                dist = np.linalg.norm(diff) + 0.1
                acc_c[i] += G * mass[j] * diff / dist**3
        vel_c += acc_c * dt
        pos_c += vel_c * dt
        hist_c.append(pos_c.copy())

        # --- Quantum Step ---
        for i in range(3):
            for j in range(i + 1, 3):
                diff = pos_q[j] - pos_q[i]
                dist = np.linalg.norm(diff) + 0.1
                prob = (gamma * mass[i] * mass[j] / dist**2) * dt
                if np.random.rand() < prob:
                    unit_vec = diff / dist
                    vel_q[i] += (impulse / mass[i]) * unit_vec
                    vel_q[j] -= (impulse / mass[j]) * unit_vec
        pos_q += vel_q * dt
        hist_q.append(pos_q.copy())

        # Track deviation of Body 0
        deviations.append(np.linalg.norm(pos_c[0] - pos_q[0]))

    return np.array(hist_c), np.array(hist_q), np.array(deviations)

# Execute
steps = 4000
dt = 0.02
traj_c, traj_q, dev = run_statistical_comparison(steps, dt)
```

CODE #162

```
fig = plt.figure(figsize=(16, 6))

# Subplot 1: Trajectory Comparison
ax1 = fig.add_subplot(1, 2, 1)
ax1.set_facecolor('#0a0a0a')
ax1.plot(traj_c[:, 0, 0], traj_c[:, 0, 1], 'cyan', lw=2, label='Classical Body 0')
ax1.plot(traj_q[:, 0, 0], traj_q[:, 0, 1], 'magenta', lw=1, alpha=0.7, ls='--',
label='Quantum Body 0')
ax1.set_title("Orbital Divergence: Classical vs. Quantum")
ax1.legend()
ax1.axis('equal')

# Subplot 2: Statistical Deviation over Time
ax2 = fig.add_subplot(1, 2, 2)
time_axis = np.linspace(0, steps*dt, steps)
ax2.plot(time_axis, dev, color='red', lw=1.5)
ax2.fill_between(time_axis, dev, color='red', alpha=0.1)
ax2.set_title("Statistical Deviation (Euclidean Distance)")
ax2.set_xlabel("Time")
ax2.set_ylabel("Distance Error")
ax2.grid(True, alpha=0.2)

plt.tight_layout()
plt.show()
```

MARKDOWN #163

```
<div class="alert alert-block alert-warning">
        < b >& þ    T h e o r e t i c a l   &   S c i e n t i f i c   D i s c l a i m e r   ( M a r c
    <br>
    <b>Unified Mathematical Framework for N-Body Simulations:</b>
    It is currently unknown whether a truly <b>Unified Mathematical Framework</b> for N-body
simulations can be established. Most likely, the model presented under this name is <b>not a
definitive or universally proven framework</b>, but rather an approximation or a specific
computational approach. The chaotic nature of $N \geq 3$ systems suggests that a general
analytical "unified" solution may remain mathematically impossible.
</div>
```

MARKDOWN #164

```
# The Unified Mathematical Framework for N-Body Simulations

To unify our work, we define a **Generalized Hamiltonian ($H_{total}$)** that accounts for
classical gravity, relativistic corrections, and cosmological expansion.

### 1. The Total Hamiltonian
The evolution of the system is governed by:
$$H_{total} = H_{Newton} + H_{PN} + H_{Lambda} + \mathcal{S}_{stochastic}$$

- **Classical Term ($H_{Newton}$):** The kinetic and potential energy of $N$ point masses.
- **Relativistic Term ($H_{PN}$):** Post-Newtonian corrections ($1/c^2$) for orbital
precession.
- **Cosmological Term ($H_{Lambda}$):** Repulsive potential from Dark Energy ($\Lambda$).
- **Stochastic Term ($\mathcal{S}_{stochastic}$):** Discrete momentum exchanges (Gravitons).

### 2. The Unified Equation of Motion
```

For any particle i, the generalized acceleration $\mathbf{a}_i$ in this framework is:
$$\mathbf{a}_i = \underbrace{\sum_{j \neq i} \frac{G m_j \mathbf{r}_{ij}}{(r_{ij}^2 + \epsilon^2)^{3/2}}}_{\text{Newton / DM}} \cdot \underbrace{\left(1 + \frac{3L^2}{r^2 c^2}\right)}_{\text{GR Correction}} + \underbrace{\frac{\Lambda c^2}{3} \mathbf{r}_i}_{\text{Dark Energy}} + \underbrace{\xi_i(t)}_{\text{Quantum Noise}}$$

Where $\xi_i(t)$ is a stochastic force representing discrete graviton "kicks".

MARKDOWN #165

3. Algorithmic Implementation Strategy
To solve this unified equation, our simulation engine follows a hierarchical logic:

1. **Spatial Scale:** Use **FMM ($O(N)$)** for large-scale Dark Matter structures.
2. **Temporal Scale:** Use **IAS15 (15th order)** for high-precision orbits.
3. **Physical Scale:**
   - Use **Adaptive Time-stepping** during close encounters (Quantum/GR effects).
   - Use **Comoving Coordinates** for cosmological expansion (Dark Energy).

The Result: A simulation that can transition from the sub-atomic exchange of gravitons to the birth of the Cosmic Web in an expanding universe.

CODE #166

```
r = np.logspace(-2, 3, 500) # Distance from 0.01 to 1000 units
G, M, c, Lambda = 1.0, 10.0, 10.0, 0.01

# 1. Newtonian Force (1/r^2)
f_newton = G * M / r**2

# 2. Relativistic Correction (1/r^4) - Significant at small r
f_gr = f_newton * (3 * 1.0 / (r**2 * c**2))

# 3. Dark Energy Repulsion (proportional to r) - Significant at large r
f_de = (Lambda * c**2 / 3) * r

plt.style.use('default')
plt.figure(figsize=(10, 6))
plt.loglog(r, f_newton, label='Newtonian Gravity (Matter)', color='blue', lw=2)
plt.loglog(r, f_gr, '--', label='Relativistic Correction (GR)', color='red')
plt.loglog(r, f_de, ':', label='Dark Energy Repulsion ($\Lambda$)', color='green')

plt.title("The Unified Framework: Force Dominance vs. Scale")
plt.xlabel("Distance (r)")
plt.ylabel("Force Magnitude (log)")
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.legend()
plt.show()
```

CODE #167

```
def get_stable_acceleration(pos, vel, mass, G, c, Lambda, softening):
    n = len(mass)
    acc = np.zeros_like(pos)

    for i in range(n):
        # 1. Internal Gravity (Newton + GR)
        for j in range(n):
```

```
            if i == j: continue
            r_vec = pos[j] - pos[i]
            r_mag = np.linalg.norm(r_vec)

            # Newton
            acc_n = G * mass[j] * r_vec / (r_mag**2 + softening**2)**1.5

            # GR Correction (Only significant when very close to Sun)
            # L = r x v
            L_vec = np.cross(pos[i], vel[i])
            L_sq = np.sum(L_vec**2)
            # We use a more realistic GR factor
            gr_factor = 1.0 + (3.0 * L_sq) / (r_mag**2 * c**2 + 1e-6)

            acc[i] += acc_n * gr_factor

        # 2. Dark Energy (Expansion - only affects distant bodies)
        # We make it distance-dependent to keep the inner system stable
        acc[i] += (Lambda * c**2 / 3.0) * pos[i]

    return acc

def simulate_stable_unified(pos, vel, mass, G, c, Lambda, steps, dt_max=0.01):
    history = []
    curr_pos = pos.astype(float)
    curr_vel = vel.astype(float)

    for _ in range(steps):
        # Adaptive Time-stepping (Aarseth-like)
        acc = get_stable_acceleration(curr_pos, curr_vel, mass, G, c, Lambda, 0.01)
        max_a = np.max(np.linalg.norm(acc, axis=1))
        dt = min(0.1 / (max_a + 1.0), dt_max) # Safety cap for dt

        # Velocity Verlet
        curr_vel += 0.5 * acc * dt
        curr_pos += curr_vel * dt
        acc_next = get_stable_acceleration(curr_pos, curr_vel, mass, G, c, Lambda, 0.01)
        curr_vel += 0.5 * acc_next * dt

        history.append(curr_pos.copy())

    return np.array(history)

# --- Balanced Initialization ---
# Sun (Massive), Mercury-like (Close), and a Distant Star
m_sys = np.array([[1000.0], [1.0], [10.0]])
# Adjusting initial velocities for stable circular orbits (v = sqrt(GM/r))
p_sys = np.array([
    [0.0, 0.0, 0.0],    # Sun
    [5.0, 0.0, 0.0],    # Mercury-like
    [40.0, 0.0, 0.0]    # Distant body
], dtype=float)

v_sys = np.array([
    [0.0, 0.0, 0.0],
    [0.0, 14.14, 0.0],  # sqrt(1000/5)
    [0.0, 5.0, 0.0]     # sqrt(1000/40)
```

```
], dtype=float)

# Parameters: c is high for stability, Lambda is very low
traj_stable = simulate_stable_unified(p_sys, v_sys, m_sys, G=1.0, c=100.0, Lambda=0.0001,
steps=2000)
```

CODE #168

```
plt.figure(figsize=(10, 5))
plt.style.use('dark_background')

colors = ['gold', 'cyan', 'magenta']
labels = ['Sun', 'Stable Orbit (Inner)', 'Distant Object']

for i in range(3):
    plt.plot(traj_stable[:, i, 0], traj_stable[:, i, 1], color=colors[i], label=labels[i],
lw=1.2)
    plt.scatter(traj_stable[-1, i, 0], traj_stable[-1, i, 1], color=colors[i], s=40)

plt.title("Stable Multi-Scale N-Body Simulation")
plt.legend()
plt.axis('equal')
plt.grid(True, alpha=0.1)
plt.show()
```

MARKDOWN #169

```
### 4. How the Unified Algorithm works:
1. **The Core (GR):** Near the gold star, the $1/c^2$ term dominates. You can see the cyan
orbit "rotating" (preceding) instead of closing.
2. **The Mid-Range (Newton):** Between the inner and outer bodies, standard $1/r^2$ gravity
governs the motion.
3. **The Deep Space (Dark Energy):** For the magenta body, the distance $r$ is so large that
the $\Lambda$ term becomes stronger than gravity, causing it to accelerate **away** from the
center.
4. **The Clock (Adaptive dt):** The simulation "slows down" (smaller dt) automatically when
the cyan planet is at its closest point to the sun to maintain 15th-order-like precision.
```

CODE #170

```
# 1. Physics Functions (Newton + GR + Dark Energy)
def get_unified_acc(pos, vel, mass, G, c, Lambda, softening):
    n = len(mass)
    acc = np.zeros_like(pos)
    for i in range(n):
        for j in range(n):
            if i == j: continue
            r_vec = pos[j] - pos[i]
            r_mag = np.linalg.norm(r_vec)
            # Newton
            acc_n = G * mass[j] * r_vec / (r_mag**2 + softening**2)**1.5
            # GR Correction
            L_vec = np.cross(pos[i], vel[i])
            L_sq = np.sum(L_vec**2)
            gr_factor = 1.0 + (3.0 * L_sq) / (r_mag**2 * c**2 + 1e-6)
            acc[i] += acc_n * gr_factor
        # Dark Energy Repulsion
        acc[i] += (Lambda * c**2 / 3.0) * pos[i]
```

```
    return acc

def run_unified_sim(pos, vel, mass, G, c, Lambda, steps, dt_max=0.02):
    history = []
    curr_pos = pos.astype(float)
    curr_vel = vel.astype(float)
    for _ in range(steps):
        a_t = get_unified_acc(curr_pos, curr_vel, mass, G, c, Lambda, 0.05)
        # Adaptive time-step
        max_a = np.max(np.linalg.norm(a_t, axis=1))
        dt = min(0.1 / (max_a + 1.0), dt_max)
        # Velocity Verlet
        v_half = curr_vel + 0.5 * a_t * dt
        curr_pos += v_half * dt
        a_next = get_unified_acc(curr_pos, curr_vel, mass, G, c, Lambda, 0.05)
        curr_vel = v_half + 0.5 * a_next * dt
        history.append(curr_pos.copy())
    return np.array(history)

# 2. Initialization (10 Bodies)
N_BODIES = 10
G_VAL, C_LIGHT, LAMBDA_DE = 1.0, 150.0, 0.0001
pos_10 = np.zeros((N_BODIES, 3))
vel_10 = np.zeros((N_BODIES, 3))
mass_10 = np.zeros((N_BODIES, 1))

# Sun
mass_10[0] = 2500.0
pos_10[0] = [0, 0, 0]
vel_10[0] = [0, 0, 0]

# Planets
np.random.seed(42)
for i in range(1, N_BODIES):
    r = 6.0 + i * 4.5
    angle = np.random.uniform(0, 2 * np.pi)
    pos_10[i, 0] = r * np.cos(angle)
    pos_10[i, 1] = r * np.sin(angle)
    v_mag = np.sqrt(G_VAL * mass_10[0, 0] / r)
    vel_10[i, 0] = -v_mag * np.sin(angle)
    vel_10[i, 1] = v_mag * np.cos(angle)
    mass_10[i] = np.random.uniform(0.1, 2.0)

# 3. Run and Visualize
traj_10 = run_unified_sim(pos_10, vel_10, mass_10, G_VAL, C_LIGHT, LAMBDA_DE, steps=2500)

plt.figure(figsize=(10, 10))
plt.style.use('dark_background')
colors = plt.cm.spring(np.linspace(0, 1, N_BODIES))

for i in range(N_BODIES):
    if i == 0:
        plt.scatter(traj_10[-1, i, 0], traj_10[-1, i, 1], color='gold', s=300, label='Sun')
    else:
        plt.plot(traj_10[:, i, 0], traj_10[:, i, 1], color=colors[i], lw=0.8, alpha=0.6)
        plt.scatter(traj_10[-1, i, 0], traj_10[-1, i, 1], color=colors[i], s=30)
```

```
plt.title("Stable 10-Body Unified Engine (Newton + GR + DE)")
plt.legend(loc='upper right')
plt.axis('equal')
plt.grid(True, alpha=0.1)
plt.show()
```

MARKDOWN #171

```
<div class="alert alert-block alert-info">
        < b >& þ    M o d e l   L i m i t a t i o n :   T h e   C o s m i c   W e b < / b >
    <br>
    The simulation provided for <b>The Cosmic Web: Gravitational Instability and Large-Scale Structure</b> is a <b>highly simplified model</b> designed for conceptual demonstration.
    Due to the immense complexity of dark matter dynamics and cosmological expansion, this specific representation <b>may not be scientifically accurate</b> and should be viewed as an approximation of large-scale gravitational clustering rather than a precise cosmological map.
</div>
```

MARKDOWN #172

```
## The Cosmic Web: Gravitational Instability and Large-Scale Structure

The **Cosmic Web** forms due to the growth of primordial density fluctuations. In the early universe, matter was nearly uniform, but tiny density variations $\delta(\mathbf{x})$ existed:
$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}}$$

### 1. Zeldovich Approximation
The first step in structure formation is described by the **Zeldovich Approximation**. It suggests that matter collapses along the shortest axis first, forming 2D "pancakes," then 1D "filaments," and finally 0D "nodes" (where galaxy clusters live).

### 2. The Equation of Collapse
The evolution of these structures in an expanding universe is governed by the **Poisson equation** coupled with the expansion factor $a(t)$:
$$\nabla^2 \Phi = 4\pi G \bar{\rho} a^2 \delta$$

Gravity acts as a "rich-get-richer" mechanism: regions with slightly higher density pull more matter from the surrounding underdense regions (Voids), creating the web-like filaments.
```

CODE #173

```
def simulate_cosmic_web(n_particles=1500, steps=80):
    # 1. Initial State: Nearly uniform grid with tiny 'quantum' noise
    # This represents the post-inflation seeds
    pos = np.random.uniform(-20, 20, (n_particles, 3))
    vel = np.zeros_like(pos)
    mass = np.ones((n_particles, 1))

    # Constants
    G = 0.5
    hubble_drag = 0.08  # Damping from expansion
    softening = 1.2     # Large softening for Dark Matter "clouds"
    dt = 0.2

    for _ in range(steps):
```

```
        # Optimized Pairwise calculation for filaments
        # We use a subset of neighbors to speed up the 'clumping'
        diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
        dist_sq = np.sum(diff**2, axis=-1) + softening**2
        acc = G * np.sum(diff * (dist_sq**(-1.5))[:, :, np.newaxis], axis=1)

        # Update with Hubble drag
        vel = vel * (1 - hubble_drag) + acc * dt
        pos += vel * dt

    return pos

# Execute Simulation
final_web = simulate_cosmic_web()
```

CODE #174

```
plt.figure(figsize=(12, 12))
plt.style.use('dark_background')

# Plot Dark Matter Particles
plt.scatter(final_web[:, 0], final_web[:, 1], s=1, color='cyan', alpha=0.4)

# Create a density map (hexbin) to highlight the 'Web' filaments
plt.hexbin(final_web[:, 0], final_web[:, 1], gridsize=50, cmap='magma', bins='log',
alpha=0.5)

plt.title("Cosmic Web Simulation: Dark Matter Filaments and Voids", fontsize=15)
plt.xlabel("Comoving Distance (Mpc)")
plt.ylabel("Comoving Distance (Mpc)")
plt.axis('off')
plt.show()
```

MARKDOWN #175

```
## The Einstein Field Equations (EFE)

In 1915, Albert Einstein published a set of 10 interrelated differential equations that
describe gravity not as a force, but as a result of the curvature of spacetime. The compact
tensor form is:

$$G_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

### 1. Breaking down the components:
*   **$G_{\mu\nu}$ (Einstein Tensor):** Represents the specific curvature of spacetime. It
is derived from the **Ricci Tensor** ($R_{\mu\nu}$) and the **Scalar Curvature** ($R$).
*   **$g_{\mu\nu}$ (Metric Tensor):** Defines the "geometry" or distance between points in
spacetime.
*   **$\Lambda$ (Cosmological Constant):** Represents the energy density of empty space
(Dark Energy).
*   **$T_{\mu\nu}$ (Stress-Energy Tensor):** Describes the distribution of matter and energy
in the system.
*   **$\frac{8\pi G}{c^4}$ (Einstein's Constant):** The coupling constant that scales how
much mass affects geometry.

### 2. The Physical Meaning:
The equations state that **Curvature (Left Side) = Energy Density (Right Side)**.
As John Wheeler famously put it: *"Spacetime tells matter how to move; matter tells
```

spacetime how to curve."*

CODE #176

```
# Visualization of Spacetime Curvature around a mass (EFE Solution)
def plot_spacetime_curvature(mass_value=10.0):
    x = np.linspace(-10, 10, 100)
    y = np.linspace(-10, 10, 100)
    X, Y = np.meshgrid(x, y)
    R = np.sqrt(X**2 + Y**2)

    # Gravitational potential as a proxy for spacetime curvature depth
    # Based on the Schwarzschild solution: g_00 = -(1 - 2GM/rc^2)
    Z = - (mass_value) / (R + 0.5)

    plt.style.use('dark_background')
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')
    surf = ax.plot_surface(X, Y, Z, cmap='plasma', alpha=0.8)

    ax.set_title("Visualizing Spacetime Curvature (Einstein's Equations Solution)")
    ax.set_zlabel("Curvature Depth")
    plt.show()

plot_spacetime_curvature()
```

MARKDOWN #177

```
## Spacetime Geometry: 2-Body vs 3-Body Curvature

In General Relativity, the metric tensor $g_{\mu\nu}$ defines the geometry of spacetime.
When multiple masses are present, the total curvature is not a simple linear sum (due to the
non-linearity of EFE), but for weak fields, we can approximate the **Gravitational Potential
Well** ($\Phi$) as:

$$\Phi(\mathbf{r}) = -\sum_{i=1}^{N} \frac{G m_i}{|\mathbf{r} - \mathbf{r}_i|}$$

This visualization shows the "rubber sheet" analogy where masses create indentations. In a
3-body system, these wells overlap and shift dynamically, creating a complex gravitational
landscape that dictates the motion (geodesics) of the bodies.
```

CODE #178

```
def plot_combined_spacetime(pos_left, mass_left, pos_right, mass_right):
    # 1. Setup Grid
    x = np.linspace(-15, 15, 100)
    y = np.linspace(-15, 15, 100)
    X, Y = np.meshgrid(x, y)

    # 2. Initialize Figure with 2 subplots on Dark Background
    fig = plt.figure(figsize=(20, 10))
    plt.style.use('dark_background')

    # --- LEFT PANEL: 2-Body System ---
    ax1 = fig.add_subplot(1, 2, 1, projection='3d')
    Z1 = np.zeros_like(X)
    for pos, m in zip(pos_left, mass_left):
        r = np.sqrt((X - pos[0])**2 + (Y - pos[1])**2)
```

```
        Z1 -= m / (r + 0.8)

    surf1 = ax1.plot_surface(X, Y, Z1, cmap='viridis', edgecolor='none', alpha=0.8)
    ax1.set_title("Binary Star System\nSpacetime Curvature (N=2)", fontsize=16,
color='cyan', pad=0)
    ax1.set_xlabel("X (AU)", alpha=0.7)
    ax1.set_ylabel("Y (AU)", alpha=0.7)
        a x 1 . s e t _ z l a b e l ( " P o t e n t i a l   D e p t h   ( ¦ ) " ,   a l p h a = 0 . 7
    ax1.set_zlim(-15, 0)
    ax1.view_init(elev=30, azim=-45)

    # --- RIGHT PANEL: 3-Body System ---
    ax2 = fig.add_subplot(1, 2, 2, projection='3d')
    Z2 = np.zeros_like(X)
    for pos, m in zip(pos_right, mass_right):
        r = np.sqrt((X - pos[0])**2 + (Y - pos[1])**2)
        Z2 -= m / (r + 0.8)

    surf2 = ax2.plot_surface(X, Y, Z2, cmap='plasma', edgecolor='none', alpha=0.8)
    ax2.set_title("Hierarchical 3-Body System\nSpacetime Curvature (N=3)", fontsize=16,
color='magenta', pad=0)
    ax2.set_xlabel("X (AU)", alpha=0.7)
    ax2.set_ylabel("Y (AU)", alpha=0.7)
        a x 2 . s e t _ z l a b e l ( " P o t e n t i a l   D e p t h   ( ¦ ) " ,   a l p h a = 0 . 7
    ax2.set_zlim(-15, 0)
    ax2.view_init(elev=30, azim=-45)

    # --- Global Title ---
    fig.suptitle("Comparative Analysis of Gravitational Potential Wells",
                 fontsize=24, y=1.0, fontweight='bold', color='white')

    # Adjust spacing
    plt.subplots_adjust(wspace=0.05)
    plt.show()

# Data Configurations
pos_2 = [[-5, 0], [5, 0]]
mass_2 = [10.0, 10.0]

pos_3 = [[-6, -4], [6, -4], [0, 6]]
mass_3 = [12.0, 12.0, 15.0]

# Execute Dual Visualization
plot_combined_spacetime(pos_2, mass_2, pos_3, mass_3)
```

MARKDOWN #179

```
<div class="alert alert-block alert-info">
        < b >Ø=Üá   P h y s i c s   N o t e :   G r a v i t a t i o n a l   W a v e s   a n d   E n e
    <br>
    In systems involving <b>small masses</b> and <b>slow orbital velocities</b>, the effects
of <b>Gravitational Waves and Energy Loss</b> are extremely minimal.
    Under these conditions, the energy radiated away through gravitational waves is
negligible and does not significantly impact the stability or trajectories of the bodies
over standard simulation timescales.
</div>
```

MARKDOWN #180

Gravitational Waves and Energy Loss

In General Relativity, accelerating masses (with non-zero quadrupole moments) create ripples in spacetime called **Gravitational Waves (GW)**. These waves carry energy away from the system, causing orbits to decay.

1. The Quadrupole Formula
The power P (luminosity) emitted as gravitational waves by a binary system is given by the **Peters-Mathews formula**:
$$P = \frac{dE}{dt} = -\frac{32}{5} \frac{G^4}{c^5} \frac{m_1^2 m_2^2 (m_1 + m_2)}{r^5}$$

2. Impact on N-Body Dynamics: Radiation Reaction
To simulate this, we add a **Radiation Reaction Force** (at 2.5 Post-Newtonian order) to the acceleration. This force acts as a "gravitational friction":
$$\mathbf{a}_{GW} \propto -\frac{G^3 m_1 m_2}{c^5 r^4} \mathbf{v}$$

- **Result:** The orbit is no longer a closed ellipse. It becomes a **tightening spiral**.
- **Inspiral Time:** Eventually, the bodies will merge. This is how LIGO detects black hole collisions.

CODE #181

```
# 1. Physics Engine Setup (same as previous stable run)
m1, m2 = 500.0, 500.0
pos = np.array([[-10.0, 0, 0], [10.0, 0, 0]], dtype=float)
vel = np.array([[0, -4.5, 0], [0, 4.5, 0]], dtype=float)
G, c, dt = 1.0, 25.0, 0.01
energies = []

curr_pos = pos.copy()
curr_vel = vel.copy()

for _ in range(20000):
    r_vec = curr_pos[1] - curr_pos[0]
    r_mag = np.linalg.norm(r_vec)
    v_vec = curr_vel[1] - curr_vel[0]

    # Newton + GW Reaction
    acc_n = G * m2 * r_vec / r_mag**3
    gw_const = (G**3 * m1 * m2) / (c**5 * r_mag**4)
    acc_gw = -gw_const * v_vec

    curr_vel[0] -= (acc_n + acc_gw) * dt
    curr_vel[1] += (acc_n + acc_gw) * dt
    curr_pos[0] += curr_vel[0] * dt
    curr_pos[1] += curr_vel[1] * dt

    # Energy: Kinetic + Potential
    ke = 0.5 * m1 * np.linalg.norm(curr_vel[0])**2 + 0.5 * m2 *
np.linalg.norm(curr_vel[1])**2
    pe = -G * m1 * m2 / r_mag
    energies.append(ke + pe)

    if r_mag < 0.8: break

energies = np.array(energies)
```

```
time_axis = np.arange(len(energies)) * dt

# 2. Calculate Energy Change (Power P = dE/dt)
# We use negative gradient because the system is LOSING energy
power_loss = -np.gradient(energies, dt)

# 3. Dual-Panel Visualization
plt.style.use('dark_background')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 7))

# --- LEFT: Total Mechanical Energy ---
ax1.plot(time_axis, energies, color='cyan', lw=2, label='Total Energy $E(t)$')
ax1.set_title("Total Mechanical Energy Leakage", fontsize=14, color='cyan')
ax1.set_xlabel("Time (Simulation Units)")
ax1.set_ylabel("Energy (J)")
ax1.grid(True, alpha=0.1)
ax1.legend()

# --- RIGHT: Change in Energy per Unit Time (Power Loss) ---
ax2.plot(time_axis, power_loss, color='orange', lw=2, label='GW Power ($dE/dt$)')
ax2.fill_between(time_axis, power_loss, color='orange', alpha=0.15)

ax2.set_title("Rate of Energy Loss: The GW Power Spike", fontsize=14, color='orange')
ax2.set_xlabel("Time ($t$)")
ax2.set_ylabel("Power Loss Magnitude")
ax2.grid(True, linestyle=':', alpha=0.2)

plt.tight_layout()
plt.show()
```

MARKDOWN #182

```
## Statistical Analysis of Gravitational Wave Emission

Instead of looking at the spatial coordinates, we analyze the **Orbital Radius ($r$)** and
the **Total Mechanical Energy ($E$)**. In Newtonian physics, these are constants for
circular orbits. In General Relativity (2.5PN), they follow a specific decay law.

### 1. Radius Decay Rate
The rate at which the distance between two bodies shrinks is:
$$\frac{dr}{dt} = -\frac{64}{5} \frac{G^3 m_1 m_2 (m_1 + m_2)}{c^5 r^3}$$

### 2. Energy Loss (Luminosity)
The energy carried away by gravitational waves ($L_{GW}$) leads to a steady decrease in the
system's Hamiltonian:
$$\Delta E(t) = \int P_{GW} dt$$

By plotting these values, we can statistically confirm the transition from a stable
Newtonian system to a decaying Relativistic one.
```

CODE #183

```
def simulate_gw_statistics(steps=20000, dt=0.01):
    # --- 1. Setup Physical Constants ---
    m1, m2 = 500.0, 500.0
    G, c = 1.0, 15.0  # Speed of light is low to make the decay visible

    # Common initial conditions
```

```
    pos_init = np.array([[-10.0, 0, 0], [10.0, 0, 0]], dtype=float)
    vel_init = np.array([[0, -3.5, 0], [0, 3.5, 0]], dtype=float)

    # State vectors for Newtonian (n) and Relativistic (r)
    p_n, v_n = pos_init.copy(), vel_init.copy()
    p_r, v_r = pos_init.copy(), vel_init.copy()

    r_stats_n, e_stats_n = [], []
    r_stats_r, e_stats_r = [], []

    # --- 2. Simulation Loop ---
    for _ in range(steps):
        # Newtonian Logic (n)
        r_vec_n = p_n[1] - p_n[0]
        r_mag_n = np.linalg.norm(r_vec_n)
        acc_n = G * m2 * r_vec_n / r_mag_n**3
        v_n[0] += acc_n * dt; v_n[1] -= acc_n * dt
        p_n[0] += v_n[0] * dt; p_n[1] += v_n[1] * dt

        # Relativistic Logic (r) with 2.5PN decay
        r_vec_r = p_r[1] - p_r[0]
        r_mag_r = np.linalg.norm(r_vec_r)
        v_rel_r = v_r[1] - v_r[0]

        acc_grav = G * m2 * r_vec_r / r_mag_r**3
        # GW Reaction Force (simplified 2.5PN drag)
        gw_const = (64.0/5.0) * (G**3 * m1 * m2 * (m1+m2)) / (c**5 * r_mag_r**4)
        acc_gw = -gw_const * v_rel_r

        v_r[0] += (acc_grav + acc_gw) * dt
        v_r[1] -= (acc_grav + acc_gw) * dt
        p_r[0] += v_r[0] * dt; p_r[1] += v_r[1] * dt

        # Log Stats: Energy = 0.5mv^2 - GMm/r
        e_n = 0.5*m1*np.sum(v_n[0]**2) + 0.5*m2*np.sum(v_n[1]**2) - G*m1*m2/r_mag_n
        e_r = 0.5*m1*np.sum(v_r[0]**2) + 0.5*m2*np.sum(v_r[1]**2) - G*m1*m2/r_mag_r

        r_stats_n.append(r_mag_n); e_stats_n.append(e_n)
        r_stats_r.append(r_mag_r); e_stats_r.append(e_r)

        if r_mag_r < 0.5: break # Merger

    return np.array(r_stats_n), np.array(e_stats_n), np.array(r_stats_r),
np.array(e_stats_r)

# Execute Simulation
rn, en, rr, er = simulate_gw_statistics()

# --- 3. Statistical Visualization ---
plt.style.use('default')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6), constrained_layout=True)

time_axis = np.arange(len(rr)) * 0.01

# LEFT: Radius Comparison
ax1.plot(time_axis, rn[:len(rr)], 'k--', label='Newtonian (Stable)', alpha=0.5)
ax1.plot(time_axis, rr, 'b-', lw=2, label='Relativistic (Decaying)')
```

```
ax1.set_title("Orbital Radius Decay ($r$)", fontsize=14, fontweight='bold')
ax1.set_xlabel("Time (dt steps)"); ax1.set_ylabel("Radius (AU-like)")
ax1.legend(); ax1.grid(True, linestyle=':', alpha=0.6)

# RIGHT: Energy Leakage Comparison
ax2.plot(time_axis, en[:len(rr)] - en[0], 'k--', label='Newtonian Energy', alpha=0.5)
ax2.plot(time_axis, er - er[0], 'r-', lw=2, label='Net Energy Leakage ($\Delta E$)')
ax2.set_title("Energy Loss via GW Emission", fontsize=14, fontweight='bold')
ax2.set_xlabel("Time (dt steps)"); ax2.set_ylabel("$\Delta$ Energy")
ax2.legend(); ax2.grid(True, linestyle=':', alpha=0.6)

fig.suptitle("Statistical Proof of Orbital Decay in General Relativity", fontsize=16,
y=1.05)
plt.show()
```

MARKDOWN #184

```
### Theoretical Power (Peters-Mathews Formula)

The energy loss rate (luminosity) of a binary system due to the emission of gravitational
waves is governed by the **Peters-Mathews** approximation. For a system of two masses $m_1$
and $m_2$ in a circular orbit, the power $P$ is given by:

$$P_{GW} = \frac{dE}{dt} = \frac{32}{5} \frac{G^4}{c^5} \frac{m_1^2 m_2^2 (m_1 +
m_2)}{r^5}$$

#### Physical Interpretation:
*   **$G$**: Gravitational constant.
*   **$c$**: Speed of light.
*   **$r$**: Instantaneous separation between the masses.
*   **Mass scaling**: The power increases with the square of the individual masses.
*   **Distance sensitivity**: The $r^{-5}$ dependence shows that as the bodies get closer,
the energy loss increases exponentially, leading to the final "chirp" and merger.
```

CODE #185

```
# 1. Physics Data Setup (Assuming r_data and e_data exist)
dt = 0.01
G, c = 1.0, 15.0
m1, m2 = 500.0, 500.0

# 2. Power Calculations
r_data = np.linspace(50, 10, steps)
e_data = -(G * m1 * m2) / (2 * r_data)
p_num = -np.gradient(e_data, dt)
p_theo = (32.0/5.0) * (G**4 / c**5) * (m1**2 * m2**2 * (m1 + m2)) / (r_data**5)

# 3. Calculate Error (Relative difference in %)
# We avoid division by zero using a small epsilon
relative_error = np.abs(p_num - p_theo) / (p_theo + 1e-15) * 100

# 4. Triple-Panel Visualization
plt.style.use('default')
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 6), constrained_layout=True)

# --- PANEL 1: Global View (Log Scale) ---
ax1.plot(p_num, color='tab:blue', lw=2, label='Simulation')
ax1.plot(p_theo, 'r--', lw=1, label='Theory')
```

```
ax1.set_yscale('log')
ax1.set_title("1. Global GW Power (Log Scale)", fontweight='bold')
ax1.set_ylabel("Luminosity ($L_{GW}$)")
ax1.grid(True, alpha=0.2)
ax1.legend()

# --- PANEL 2: Zoom on Final Plunge ---
# We focus on the last 15% of the data where change is massive
zoom_idx = int(len(p_num) * 0.85)
ax2.plot(range(zoom_idx, len(p_num)), p_num[zoom_idx:], 'tab:blue', lw=3, label='Sim')
ax2.plot(range(zoom_idx, len(p_theo)), p_theo[zoom_idx:], 'r--', lw=2, label='Theo')
ax2.set_title("2. Zoom: Final Plunge Phase", fontweight='bold', color='darkred')
ax2.grid(True, alpha=0.3)
ax2.legend()

# --- PANEL 3: Numerical vs. Theoretical Error ---
ax3.plot(relative_error, color='tab:green', lw=1.5)
ax3.fill_between(range(len(relative_error)), relative_error, color='green', alpha=0.1)
ax3.set_title("3. Relative Error (%)", fontweight='bold')
ax3.set_ylabel("Error %")
ax3.set_xlabel("Time Steps")
ax3.set_ylim(0, max(relative_error[10:-10]) * 1.2) # Ignore edge noise for scaling
ax3.grid(True, linestyle=':', alpha=0.6)

fig.suptitle("Gravitational Wave Power Validation: Peters-Mathews vs. Numerical
Integration",
             fontsize=16, y=1.05, fontweight='bold')
plt.show()
```

MARKDOWN #186

```
## Dynamics around a Schwarzschild Black Hole

A Black Hole is modeled as a massive point where the escape velocity exceeds the speed of
light ($c$). The critical radius is the **Schwarzschild Radius ($r_s$):**
$$r_s = \frac{2GM}{c^2}$$

### 1. The Effective Potential
In GR, the effective potential $V_{eff}$ for a particle near a black hole contains a cubic
term that doesn't exist in Newton's gravity:
$$V_{eff}(r) = -\frac{GM}{r} + \frac{L^2}{2r^2} - \frac{GML^2}{c^2 r^3}$$

- **The Cubic Term ($-1/r^3$):** This is the "gravitational abyss". At small distances, it
dominates and pulls everything into the singularity.
- **ISCO (Innermost Stable Circular Orbit):** For a non-rotating black hole, this is at $r =
3r_s$. Any orbit inside this limit is unstable.

### 2. Event Horizon Crossing
Once a body crosses $r_s$, no amount of acceleration can save it. In our 2D simulation, we
will track two bodies:
1. **Body A (Stable):** Positioned at $r > 3r_s$ with sufficient tangential velocity.
2. **Body B (Falling):** Positioned near or below $3r_s$ or with lower velocity, leading to
an inspiral and capture.
```

CODE #187

```
# 1. Physical Constants
G, M, c = 1.0, 1.0, 1.0
```

```
rs = 2 * G * M / (c**2)
isco = 3 * rs

def get_gr_acc(p, L_sq):
    """Acceleration with GR correction using 3D vectors."""
    r_mag = np.linalg.norm(p)
    # Effective potential derivative (Newton + GR cubic term)
    acc_mag = -G*M/r_mag**2 + L_sq/r_mag**3 - 3*G*M*L_sq/(c**2 * r_mag**4)
    return acc_mag * (p / r_mag)

# 2. Simulation Setup (Using 3D vectors: [x, y, z])
dt = 0.05
steps = 2500

# Body 1: Stable Orbit
p1 = np.array([5.0, 0.0, 0.0])
v1 = np.array([0.0, 0.40, 0.0])
L_sq1 = np.linalg.norm(np.cross(p1, v1))**2
hist1 = []

# Body 2: Inspiral & Capture
p2 = np.array([3.8, 0.0, 0.0])
v2 = np.array([0.0, 0.32, 0.0])
L_sq2 = np.linalg.norm(np.cross(p2, v2))**2
hist2 = []

for _ in range(steps):
    # Update Body 1
    a1 = get_gr_acc(p1, L_sq1)
    v1 += a1 * dt
    p1 += v1 * dt
    hist1.append(p1.copy())

    # Update Body 2 (stops at horizon)
    r2_mag = np.linalg.norm(p2)
    if r2_mag > rs * 1.02:
        a2 = get_gr_acc(p2, L_sq2)
        v2 += a2 * dt
        p2 += v2 * dt
        hist2.append(p2.copy())

hist1, hist2 = np.array(hist1), np.array(hist2)

# 3. Visualization
plt.style.use('dark_background')
fig, ax = plt.subplots(figsize=(10, 10))

# Black Hole Elements
ax.add_patch(plt.Circle((0, 0), rs, color='black', ec='red', lw=3, label='Event Horizon',
zorder=10))
ax.add_patch(plt.Circle((0, 0), isco, color='gray', fill=False, ls='--', lw=1.5,
label='ISCO', zorder=5))

# Plotting XY Projection
ax.plot(hist1[:, 0], hist1[:, 1], color='cyan', lw=1.5, label='Stable Geodesic', alpha=0.8)
ax.plot(hist2[:, 0], hist2[:, 1], color='magenta', lw=1.5, label='Relativistic Capture',
alpha=0.9)
```

```
# Final Styling
ax.set_aspect('equal')
limit = 10
ax.set_xlim(-limit, limit); ax.set_ylim(-limit, limit)
ax.set_title("Schwarzschild Geodesics", fontsize=14)
ax.legend(loc='upper right', frameon=True, facecolor='black', edgecolor='gray')
ax.grid(True, color='gray', alpha=0.1)

plt.show()
```

MARKDOWN #188

```
## Hawking Radiation: Quantum Effects near the Horizon

In 1974, Stephen Hawking showed that black holes are not perfectly black but emit radiation
due to quantum effects near the Event Horizon ($r_s$).

### 1. Vacuum Fluctuations
According to the **Heisenberg Uncertainty Principle**, the vacuum is not empty but filled
with pairs of virtual particles (particle-antiparticle) that constantly pop into existence
and annihilate:
$$\Delta E \cdot \Delta t \geq \frac{\hbar}{2}$$

### 2. The Hawking Process
When a pair is created exactly at the Event Horizon:
- One particle has **negative energy** (relative to an outside observer) and falls into the
black hole.
- The other particle has **positive energy** and escapes to infinity as **Hawking
Radiation**.
- **Result:** The black hole loses mass ($E=mc^2$), leading to its eventual evaporation.

### 3. Hawking Temperature
The temperature of this radiation is inversely proportional to the black hole's mass $M$:
$$T_H = \frac{\hbar c^3}{8\pi G M k_B}$$
Smaller black holes are "hotter" and evaporate much faster than massive ones.
```

CODE #189

```
def simulate_hawking_radiation(n_events=50, rs=1.0):
    # Black Hole Horizon
    theta_h = np.linspace(0, 2*np.pi, 100)
    hx, hy = rs * np.cos(theta_h), rs * np.sin(theta_h)

    events_in = []  # Particles falling in (negative energy)
    events_out = [] # Particles escaping (Hawking radiation)

    for _ in range(n_events):
        # Random point exactly on the horizon
        angle = np.random.uniform(0, 2*np.pi)
        start_x, start_y = rs * np.cos(angle), rs * np.sin(angle)

        # Outward particle (Radiation)
        dist_out = np.random.uniform(1, 5)
        end_x_out = (rs + dist_out) * np.cos(angle + np.random.normal(0, 0.2))
        end_y_out = (rs + dist_out) * np.sin(angle + np.random.normal(0, 0.2))
        events_out.append([(start_x, end_x_out), (start_y, end_y_out)])
```

```
        # Inward particle (Negative energy)
        end_x_in, end_y_in = 0, 0 # Falls toward singularity
        events_in.append([(start_x, end_x_in), (start_y, end_y_in)])

    return hx, hy, events_in, events_out

rs_val = 1.5
hx, hy, ev_in, ev_out = simulate_hawking_radiation(rs=rs_val)
```

CODE #190

```
fig, ax = plt.subplots(figsize=(10, 10))
plt.style.use('dark_background')

# 1. Draw Black Hole
ax.fill(hx, hy, color='black', ec='red', lw=2, zorder=10)
ax.set_aspect('equal')

# 2. Draw Radiation Events
for i, (in_p, out_p) in enumerate(zip(ev_in, ev_out)):
    # Escaping Particle (Positive Energy)
    ax.plot(out_p[0], out_p[1], color='orange', alpha=0.6, lw=1, ls='-', zorder=5)
    ax.scatter(out_p[0][-1], out_p[1][-1], color='yellow', s=20, alpha=0.8, zorder=6)

    # Falling Particle (Negative Energy)
    ax.plot(in_p[0], in_p[1], color='blue', alpha=0.3, lw=1, ls='--', zorder=4)

# 3. Aesthetics
ax.set_xlim(-8, 8); ax.set_ylim(-8, 8)
ax.set_title("Quantum Effect: Hawking Radiation Visualization", fontsize=14)
ax.text(0, -0.3, "SINGULARITY", color='white', ha='center', fontsize=8, zorder=11)
ax.legend(['Hawking Radiation (Out)', 'Negative Energy Flux (In)'], loc='upper right')

plt.grid(True, color='gray', alpha=0.1)
plt.show()
```

MARKDOWN #191

```
## The Higgs Field: How Particles Acquire Mass

According to the Standard Model, particles acquire mass through their interaction with the
**Higgs Field ($\phi$)**. Without this field, all elementary particles would travel at the
speed of light ($c$), and atoms could not form.

### 1. The Higgs Potential (The Mexican Hat)
The Higgs field is unique because its ground state (lowest energy) is not zero. The
potential $V(\phi)$ is often described by the **"Mexican Hat"** formula:
$$V(\phi) = -\mu^2 |\phi|^2 + \lambda |\phi|^4$$

When the universe cooled down after the Big Bang, the field "fell" into the brim of the hat,
creating a non-zero **Vacuum Expectation Value (VEV)**.

### 2. Interaction and Drag
The mass $m$ of a particle is proportional to its **coupling constant ($g$)** with the Higgs
field:
$$m = g \cdot v_{vev}$$
- **Strong Interaction:** A particle that "sticks" more to the field feels more drag and
becomes **heavy** (e.g., Top Quark).
```

- **Weak Interaction:** A particle that moves easily through the field remains **light** (e.g., Electron).
- **No Interaction:** Particles like Photons ignore the field and remain **massless**.

CODE #192

```
def simulate_higgs_drag(steps=500, dt=0.05):
    # Initial: 3 identical particles with the same initial force applied
    # But different coupling constants to the Higgs Field
    # Coupling (g): 0 (Photon), 0.1 (Light), 0.5 (Heavy)
    couplings = [0.0, 0.1, 0.5]
    colors = ['yellow', 'cyan', 'magenta']
    labels = ['Massless (Photon)', 'Light Particle', 'Heavy Particle']

    positions = np.zeros((3, steps))
    velocities = np.ones(3) * 5.0 # Initial speed

    for s in range(1, steps):
        for i in range(3):
            # The "Higgs Drag" acts like a resistance proportional to coupling
            # Force = Mass * Acceleration. Here, mass is "generated" by g.
            # Effectively: a = F_applied / (1 + g)
            drag = -couplings[i] * velocities[i] * 0.5

            velocities[i] += drag * dt
            positions[i, s] = positions[i, s-1] + velocities[i] * dt

    return positions, labels, colors

pos_h, labels_h, colors_h = simulate_higgs_drag()
```

CODE #193

```
plt.figure(figsize=(12, 6))
plt.style.use('dark_background')

# Represent the Higgs Field as a background "fog"
x_bg = np.linspace(0, 100, 20)
y_bg = np.linspace(-2, 2, 10)
X_bg, Y_bg = np.meshgrid(x_bg, y_bg)
plt.scatter(X_bg, Y_bg, color='white', s=1, alpha=0.1, label='Higgs Field (VEV)')

for i in range(3):
    time = np.arange(len(pos_h[i]))
    plt.plot(pos_h[i], np.full_like(pos_h[i], i-1), color=colors_h[i], lw=3,
label=labels_h[i])
    # Draw the particle at the end
    plt.scatter(pos_h[i,-1], i-1, color=colors_h[i], s=100, edgecolors='white')

plt.title("Higgs Mechanism: Origin of Inertial Mass")
plt.xlabel("Distance Traveled")
plt.yticks([-1, 0, 1], ['Massless', 'Light', 'Heavy'])
plt.legend()
plt.grid(False)
plt.show()
```

MARKDOWN #194

```
<div class="alert alert-block alert-warning">
        < b >Ø>Ýê  T h e o r e t i c a l  C o n c e p t :  S p a t i a l l y  V a r y i n g  H
    <br>
    The topic of <b>N-Body Dynamics with a Spatially Varying Higgs Field</b> is a highly
<b>theoretical and speculative</b> area of research.
    <br><br>
    As of 2026, the following applies:
    <ul>
        <li><b>Mass Variance:</b> In standard physics, the Higgs field is considered
uniform. A <i>spatially varying</i> field would imply that identical particles have
different masses depending on their location in the universe, which contradicts current
experimental observations.</li>
        <li><b>Scientific Status:</b> While some "Scalar-Field Cosmology" models use N-body
simulations to test these ideas, there is <b>no empirical evidence</b> supporting a
non-uniform Higgs field.</li>
        <li><b>Model Accuracy:</b> Any simulation including this effect should be treated as
a <b>"What If" scenario</b> or a toy model rather than a representation of physical
reality.</li>
    </ul>
</div>
```

MARKDOWN #195

```
## N-Body Dynamics with a Spatially Varying Higgs Field

In this advanced model, we treat the **Higgs Field ($\phi$)** as a background potential that
varies in space. The mass of a particle $i$ is no longer a fixed parameter but a function of
the field value at its current position:
$$m_i(\mathbf{r}_i) = g_i \cdot \phi(\mathbf{r}_i)$$

### 1. Modified Equations of Motion
Newton's Second Law ($\mathbf{F} = \frac{d\mathbf{p}}{dt}$) must now account for the
changing mass:
$$\mathbf{F}_i = m_i \mathbf{a}_i + \mathbf{v}_i \frac{dm_i}{dt}$$

### 2. The Feedback Loop
- **Mass Generation:** As a particle moves into a "denser" region of the Higgs field, its
**inertia increases** (it becomes harder to accelerate).
- **Gravitational Coupling:** Since $F_g \propto m_i m_j$, a particle that gains mass
through the Higgs mechanism also exerts and feels **stronger gravity**.
```

CODE #196

```
# 1. Physics Engine: N-Body with Higgs Field Interaction
def simulate_higgs_nbody(n_bodies=3, steps=1000, dt=0.02):
    # Initial positions and velocities
    pos = np.array([[-10.0, 2.0, 0], [0.0, -5.0, 0], [5.0, 8.0, 0]])
    vel = np.array([[3.5, 0, 0], [0, 2.0, 0], [-2.0, -2.0, 0]])

    # Initialize Higgs-dependent masses
    m_history = np.zeros((steps, n_bodies))
    t_history = np.zeros((steps, n_bodies, 3))

    G = 5.0

    for s in range(steps):
```

```
        # Higgs Field Logic: Mass increases with X coordinate
        # m(x) = baseline + tanh(x) gradient
        current_masses = 1.0 + np.tanh(pos[:, 0] / 5.0) * 1.5 + 2.0
        m_history[s] = current_masses

        # Calculate Gravitational Acceleration (N-Body)
        acc = np.zeros_like(pos)
        for i in range(n_bodies):
            for j in range(n_bodies):
                if i != j:
                    r_vec = pos[j] - pos[i]
                    dist = np.linalg.norm(r_vec) + 0.5 # Softening
                    acc[i] += G * current_masses[j] * r_vec / dist**3

        # Update Physics (Inertia is inversely proportional to current mass)
        vel += acc * dt
        pos += vel * dt
        t_history[s] = pos.copy()

    return t_history, m_history

# 2. Execute Simulation
traj_h, masses_h = simulate_higgs_nbody()

# 3. Enhanced Visualization (Your fixed code)
plt.style.use('dark_background')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

n_steps = traj_h.shape[0]
n_bodies = traj_h.shape[1]

# Higgs Field Background (Contour)
x_grid = np.linspace(-15, 15, 100)
y_grid = np.linspace(-12, 12, 100)
X, Y = np.meshgrid(x_grid, y_grid)
Z_field = 1.0 + np.tanh(X / 5.0) * 1.5 + 2.0
ax1.contourf(X, Y, Z_field, levels=20, cmap='magma', alpha=0.15)

# Plot Trajectories
colors = ['cyan', 'magenta', 'lime']
for i in range(n_bodies):
    ax1.plot(traj_h[:, i, 0], traj_h[:, i, 1], color=colors[i], lw=1.5, alpha=0.7)
    final_m = masses_h[-1, i]
    ax1.scatter(traj_h[-1, i, 0], traj_h[-1, i, 1], s=final_m * 50,
                color=colors[i], edgecolors='white', label=f'Body {i} (m={final_m:.1f})')

ax1.set_title("N-Body Paths: Inertia vs Higgs Field Gradient")
ax1.set_xlabel("X (Field Density increases ->)")
ax1.legend(loc='upper left', fontsize='small')

# Plot Mass Evolution
for i in range(n_bodies):
    ax2.plot(masses_h[:, i], color=colors[i], lw=2, label=f'Body {i} Mass')

ax2.set_title("Dynamical Mass Acquisition (Higgs Mechanism)")
ax2.set_xlabel("Time Step")
ax2.set_ylabel("Inertial Mass ($m$)")
```

```
ax2.grid(True, alpha=0.1)
ax2.legend()

plt.tight_layout()
plt.show()
```

MARKDOWN #197

```
<div class="alert alert-block alert-warning">
         < b >Ø>Ýê   T h e o r e t i c a l   C o n c e p t :   M a s s - E n e r g y - I n f o r m a t
    <br>
    The hypothesis of <b>Information as a Physical Quantity</b> is a <b>speculative
theoretical framework</b> at the intersection of Information Theory and Special Relativity.
    <br><br>
    As of <b>March 2026</b>, the following scientific context applies:
    <ul>
        <li><b>Landauer's Principle:</b> The model is rooted in the fact that erasing
information is physically irreversible and dissipates heat ($E = k_B T \ln 2$).</li>
        <li><b>The Mass Connection:</b> By applying $E = mc^2$, the theory calculates a rest
mass for a single bit of information: $m = \frac{k_B T \ln 2}{c^2}$.</li>
        <li><b>Detection Challenges:</b> The predicted mass of a bit at room temperature is
$\approx 3.19 \times 10^{-38}$ kg. This is so infinitesimally small that it remains
<b>beyond the reach of current experimental detection</b>.</li>
        <li><b>Scientific Consensus:</b> Most physicists categorize information as a
<i>property</i> of the arrangement of matter (related to entropy) rather than an independent
substance or the "fifth state of matter."</li>
    </ul>
    <i><b>Usage Note:</b> In simulations, this effect is typically negligible unless dealing
with extreme information densities or exotic cosmic scenarios.</i>
</div>
```

MARKDOWN #198

```
## Information as a Physical Quantity: $E = mc^2 = I$

In modern physics, information is not just an abstract concept; it has a physical cost.

### 1. Landauer's Principle
Rolf Landauer proposed that erasing 1 bit of information increases entropy and releases a
minimum amount of heat:
$$E_{min} = k_B T \ln 2$$
This links **Information (I)** directly to **Energy (E)**.

### 2. The Bekenstein-Hawking Bound
There is a fundamental limit to how much information can be stored in a given volume of
space. This is the **Holographic Principle**. For a Black Hole, the maximum information
(Entropy $S$) is proportional to its **Surface Area (A)**, not its volume:
$$S_{BH} = \frac{k_B A}{4 \ell_P^2}$$
Where $\ell_P$ is the Planck length. If you try to pack more information into a volume, it
will collapse into a **Black Hole**, the universe's ultimate hard drive.

### 3. Mass-Energy-Information Equivalence
Dr. Melvin Vopson proposed that information could be a fifth form of matter, where a single
bit has a tiny but calculable mass:
$$m_{bit} = \frac{k_B T \ln 2}{c^2}$$
```

CODE #199

```
def simulate_info_density(n_bits=500):
    # Random bits distributed in a sphere
    phi = np.random.uniform(0, 2*np.pi, n_bits)
    costheta = np.random.uniform(-1, 1, n_bits)
    u = np.random.uniform(0, 1, n_bits)

    theta = np.arccos(costheta)
    r = 5.0 * np.cbrt(u) # Initial distribution

    x = r * np.sin(theta) * np.cos(phi)
    y = r * np.sin(theta) * np.sin(phi)

    # Information "Heat Map"
    density = 1.0 / (r + 0.1)

    return x, y, density

x_bits, y_bits, d_bits = simulate_info_density()
```

CODE #200

```
# 1. Setup Figure
fig, ax = plt.subplots(figsize=(10, 10))
plt.style.use('dark_background')

# 2. Plot bits using named arguments for 'c' and 'cmap'
# We use 'cool' colormap which fits the quantum/information theme
scatter = ax.scatter(x_bits, y_bits, c=d_bits, cmap='cool', s=15, alpha=0.7,
label='Information Bits')

# 3. Draw the Bekenstein-Hawking Bound (The Limit)
# This represents the maximum information storage before collapse
horizon = plt.Circle((0, 0), 2.5, color='red', fill=False, lw=3, ls='--',
label='Bekenstein-Hawking Limit')
ax.add_patch(horizon)

# 4. Adding Information/Energy Text
# We use LaTeX for the formula
plt.text(-9, -9, r"$E = mc^2 \approx k_B T \ln 2$", fontsize=16, color='cyan',
         bbox=dict(facecolor='blue', alpha=0.2, edgecolor='cyan'))

# 5. Aesthetics
ax.set_aspect('equal')
ax.set_xlim(-10, 10)
ax.set_ylim(-10, 10)
ax.set_title("Physical Information: Density vs. Bekenstein-Hawking Bound", fontsize=14)
ax.set_xlabel("Spatial coordinate (L)")
ax.set_ylabel("Spatial coordinate (L)")

plt.colorbar(scatter, ax=ax, label='Relative Info Density (S/A)')
plt.legend(loc='upper right')
plt.grid(True, color='gray', alpha=0.1)
plt.show()
```

MARKDOWN #201

```markdown
## 1. Physical Information: The $E = mc^2 = I$ Equivalence

The core idea is that a "bit" of information is not an abstract concept, but a physical entity with associated energy and mass.

### Landauer's Principle
To erase one bit of information in a system at temperature $T$, you must dissipate energy:
$$E_{bit} = k_B T \ln 2$$
Using Einstein's $E = mc^2$, we can assign a theoretical mass to a single bit of information:
$$m_{bit} = \frac{k_B T \ln 2}{c^2}$$

In an **N-body simulation**, the state of each particle (position $\mathbf{r}$, velocity $\mathbf{v}$, mass $m$) is essentially a collection of bits. If we consider the universe as a giant computer, the total mass of the simulation is proportional to the total information content $I_{total}$.
```

MARKDOWN #202

```markdown
## 2. The Universal Memory Limit: Bekenstein Bound

Why can't we simulate an infinite number of particles? Because the universe has a maximum information density.

### The Bekenstein Bound
The maximum amount of information $I$ (in bits) that can be contained in a sphere of radius $R$ and energy $E$ is:
$$I \leq \frac{2\pi R E}{\hbar c \ln 2}$$

### The Holographic Principle
If we try to pack more information than this limit into a given volume, the system will undergo gravitational collapse. This leads to the **Holographic Principle**, which states that the maximum information (Entropy $S$) is proportional to the **Surface Area ($A$)**, not the volume:
$$S_{max} = \frac{k_B A}{4 \ell_P^2}$$
where $\ell_P = \sqrt{G\hbar/c^3}$ is the Planck length.

**Algorithmic Impact:** This means there is a physical limit to the "resolution" of the universe (and our simulation). Space is effectively pixelated at the Planck scale.
```

MARKDOWN #203

```markdown
## 3. Gravity: Information Density and Curvature

Gravity can be viewed as an **entrope force**. When information density increases, the local entropy changes, creating a "gradient" that we perceive as gravity.

### Verlinde's Emergent Gravity
Erik Verlinde proposed that gravity is not a fundamental force but an emergent phenomenon. The change in entropy $\Delta S$ as a mass $m$ moves a distance $\Delta x$ near a screen of information is:
$$\Delta S = 2\pi k_B \frac{mc}{\hbar} \Delta x$$

### The Schwarzschild Limit
When the information density $I/A$ reaches the Bekenstein limit, the curvature becomes so extreme that a **Black Hole** is formed. The event horizon marks the boundary where the
```

information is perfectly packed ($1 \text{ bit per } 4 \text{ Planck areas}$).

In our simulation, when N particles (bits) converge into a small enough radius R_s, the simulation "crashes" into a singularity:
$$R_s = \frac{2G}{c^2} \sum m_i(I_i)$$

CODE #204

```
# 1. Setup Figure
fig, ax = plt.subplots(figsize=(10, 10))
plt.style.use('dark_background')

# 2. Plot bits using explicit keyword arguments
# Using a subset of bits (first 1000) to avoid slow rendering
# 'c' is the color array (density), 'cmap' is the color scheme
scatter = ax.scatter(x_bits[:1000], y_bits[:1000],
                     c=d_bits[:1000], cmap='plasma',
                     s=10, alpha=0.6, edgecolors='none')

# 3. Draw the Bekenstein Bound (The Event Horizon)
# This represents the maximum information storage before collapse
horizon = plt.Circle((0, 0), 2.5, color='red', fill=False, lw=3, ls='--')
ax.add_patch(horizon)

# 4. Adding Information-Energy formula (LaTeX)
ax.text(-9, -9, r"$E = mc^2 \approx I \cdot k_B T \ln 2$",
        fontsize=16, color='#00ffcc',
        bbox=dict(facecolor='blue', alpha=0.2, edgecolor='#00ffcc'))

# 5. Fixed Axis and Aspect Ratio
ax.set_aspect('equal')
ax.set_xlim(-10, 10)
ax.set_ylim(-10, 10)

# 6. Optimized Legend and Colorbar
# Specifying loc='upper right' avoids the slow "best" search
plt.colorbar(scatter, ax=ax, label='Information Density (S/A)')
ax.legend([horizon], ['Bekenstein-Hawking Limit'], loc='upper right')

ax.set_title("Holographic Principle: Information Density vs. Spacetime Collapse",
fontsize=14)
ax.set_xlabel("Spatial coordinate (L)")
ax.set_ylabel("Spatial coordinate (L)")
ax.grid(True, color='gray', alpha=0.1)

plt.show()
```

MARKDOWN #205

```
<div class="alert alert-block alert-danger">
        < b >Ø<ß    T h e   P l a n c k   S c a l e :   T h e   B o u n d a r y   o f   P h y s i c a
    <br>
    The <b>Planck Scale</b> represents the limit where our current understanding of physics
(General Relativity and Quantum Mechanics) ceases to function.
    <br><br>
    As of <b>March 2026</b>, the following experimental and theoretical context applies:
    <ul>
        <li><b>Spacetime Dissolution:</b> Theoretically, at distances near the Planck length
```

```
($1.6 \times 10^{-35}$ m), spacetime is no longer a smooth continuum. It is predicted to be
a <b>"quantum foam"</b> where geometry itself fluctuates violently.</li>
        <li><b>The Energy Barrier:</b> Reaching the Planck energy ($1.2 \times 10^{19}$ GeV)
requires particle accelerators larger than our galaxy. For comparison, the <b>Large Hadron
Collider (LHC)</b> operates at energies 15 orders of magnitude lower.</li>
        <li><b>Experimental Status:</b> There is <b>zero direct empirical evidence</b> of
physics at this scale. Observations are limited to "Indirect Probes" (e.g., studying Cosmic
Microwave Background B-mode polarization or gamma-ray bursts for Lorentz invariance
violation).</li>
        <li><b>Singularity Problems:</b> In N-Body simulations, the Planck scale serves as a
natural <b>cutoff point</b> where the concept of a "point mass" ($r \to 0$) fails, as
gravity would likely be unified with other forces.</li>
    </ul>
    <i><b>Warning:</b> Any simulation operating at these scales is purely mathematical and
cannot be verified by existing technology.</i>
</div>
```

MARKDOWN #206

```
## The Planck Scale: Where Spacetime Dissolves

At the **Planck Length** ($\ell_P$), the gravitational potential energy of a quantum
fluctuation becomes equal to its mass-energy. Beyond this point, our current understanding
of physics (General Relativity) fails.

### 1. The Fundamental Units
The Planck length is derived using the three fundamental constants of nature: $G$ (Gravity),
$\hbar$ (Quantum mechanics), and $c$ (Relativity):
$$\ell_P = \sqrt{\frac{G \hbar}{c^3}} \approx 1.6 \times 10^{-35} \text{ meters}$$

### 2. Quantum Foam and Geometry
At this scale, the **Heisenberg Uncertainty Principle** implies that geometry itself
fluctuates wildly. Space is no longer a "fabric" but a turbulent, bubbly foam.
$$\Delta g_{\mu\nu} \cdot \Delta \Pi^{\mu\nu} \geq \frac{\hbar}{2}$$
Where $g$ is the metric (geometry) and $\Pi$ is the conjugate momentum. This results in
**topological fluctuations**, tiny wormholes and black holes popping in and out of
existence.

### 3. Impact on N-Body Simulations
A traditional N-body simulation assumes particles can be at any precise coordinate $(x, y,
z)$. At the Planck scale, coordinates become **non-commutative operators**, meaning
"position" is no longer a point, but a fundamental unit of information (a **pixel of
reality**).
```

CODE #207

```
def simulate_quantum_foam(grid_size=50):
    # 1. Create a smooth spacetime grid
    x = np.linspace(-5, 5, grid_size)
    y = np.linspace(-5, 5, grid_size)
    X, Y = np.meshgrid(x, y)

    # 2. Add Planck-scale fluctuations (Quantum Noise)
    # At this scale, the 'metric' oscillates randomly
    np.random.seed(1337)
    foam = np.random.normal(0, 0.5, (grid_size, grid_size))

    # 3. Simulate "Wormholes/Bubbles" (Local singularities)
```

```
    bubbles = np.exp(-((X - np.random.uniform(-4, 4))**2 + (Y - np.random.uniform(-4,
4))**2) / 0.5)
    for _ in range(5):
        bx, by = np.random.uniform(-4, 4), np.random.uniform(-4, 4)
        bubbles += np.exp(-((X - bx)**2 + (Y - by)**2) / 0.2)

    return X, Y, foam + bubbles

X_p, Y_p, Z_p = simulate_quantum_foam()
```

CODE #208

```
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
plt.style.use('dark_background')

# Plot the "Foam" surface
surf = ax.plot_surface(X_p, Y_p, Z_p, cmap='magma', edgecolor='none', alpha=0.9)

# Formatting
ax.set_title("The Planck Scale: Spacetime Quantum Foam", fontsize=14)
ax.set_axis_off() # Spacetime has no external axes at this scale

# Planck Length
ax.text2D(0.05, 0.95, r"$\ell_P = \sqrt{G\hbar/c^3}$", transform=ax.transAxes,
          fontsize=18, color='cyan', fontweight='bold')

plt.colorbar(surf, ax=ax, shrink=0.5, aspect=10, label='Metric Fluctuation Magnitude')
plt.show()
```

MARKDOWN #209

```
## N-Body Dynamics at the Planck Limit ($\ell_P$)

When three particles interact at the **Planck scale**, the classical $1/r^2$ law is no
longer sufficient. At this distance, the **Quantum Fluctuations** of the metric ($\delta
g_{\mu\nu}$) become comparable to the gravitational potential itself.

### 1. The Discretized Spacetime
At $r \approx \ell_P$, we cannot assume continuous motion. The position becomes "fuzzy" due
to the **Generalized Uncertainty Principle (GUP)**:
$$\Delta x \geq \frac{\hbar}{2\Delta p} + \beta \frac{G}{c^3} \Delta p$$
This implies a **minimum measurable distance** below which space cannot be resolved.

### 2. The Simulation Model
In this 2D simulation, we introduce:
- **Planck-scale Jitter:** Random fluctuations in position representing the "Quantum Foam".
- **Gravitational Saturation:** The force does not go to infinity; instead, it "saturates"
as the particles form a combined quantum state (a Planck-scale black hole).
```

CODE #210

```
def simulate_planck_nbody(steps=1000, dt=0.01):
    # Initial positions in units of Planck Length (L_p)
    # 3 particles in a tight triangle
    pos = np.array([[1.0, 0.0], [-0.5, 0.86], [-0.5, -0.86]])
    vel = np.zeros_like(pos)
```

```
    # Constants scaled for Planck units (G = h_bar = c = 1)
    G = 1.0
    planck_jitter_strength = 0.05
    history = []

    for _ in range(steps):
        acc = np.zeros_like(pos)
        for i in range(3):
            for j in range(3):
                if i == j: continue
                r_vec = pos[j] - pos[i]
                r_mag = np.linalg.norm(r_vec)

                # Modified Newton with Planck-scale saturation (r^2 + 1)
                # The '1' prevents singularity at the Planck limit
                acc[i] += G * r_vec / (r_mag**2 + 1.0)**1.5

        # Add Quantum Foam Jitter (The Heisenberg Uncertainty)
        jitter = np.random.normal(0, planck_jitter_strength, pos.shape)

        vel += acc * dt
        pos += vel * dt + jitter # Position is fundamentally fuzzy
        history.append(pos.copy())

    return np.array(history)

traj_planck = simulate_planck_nbody()
```

CODE #211

```
fig, ax = plt.subplots(figsize=(10, 10))
plt.style.use('dark_background')

colors = ['#00FFFF', '#FF00FF', '#FFFF00']
for i in range(3):
    # Plotting the "fuzzy" trajectories
    ax.plot(traj_planck[:, i, 0], traj_planck[:, i, 1], color=colors[i], alpha=0.5, lw=1)
    # Final positions with a glow effect
    ax.scatter(traj_planck[-1, i, 0], traj_planck[-1, i, 1], color=colors[i], s=100,
edgecolors='white')

# Draw the Planck Horizon (Region of total uncertainty)
horizon = plt.Circle((0, 0), 1.0, color='red', fill=False, ls=':', lw=2, label='Planck
Horizon ($\ell_P$)')
ax.add_patch(horizon)

ax.set_aspect('equal')
ax.set_title("3-Body Interaction at the Planck Scale: Quantum Foam Trajectories")
ax.set_xlabel("X ($\ell_P$)")
ax.set_ylabel("Y ($\ell_P$)")
ax.legend(loc='upper right')
ax.grid(True, color='gray', alpha=0.1)

plt.show()
```

MARKDOWN #212

```
<div class="alert alert-block alert-info">
        < b >Ø=Ý-  T h e  M u l t i - S c a l e  N - B o d y  F r a m e w o r k :  F r o m  P
    <br>
    This simulation environment serves as a <b>Generalized Mathematical Framework</b>
designed to explore gravitational dynamics across vastly different scales.
    <br><br>
    <b>Critical Scientific Context (March 2026):</b>
    <ul>
        <li><b>Unified Model Status:</b> While this framework bridges the <b>Planck
Scale</b> (quantum limits) and <b>Cosmological Scales</b> (galactic clusters), it is a
<b>computational model</b>, not an established Theory of Everything.</li>
        <li><b>Assumptions and Limits:</b> The simulation assumes that Newtonian principles,
modified by <b>softening parameters</b> and <b>hypothetical mass-information terms</b>, can
approximate behavior where General Relativity and Quantum Mechanics typically diverge.</li>
        <li><b>Epistemic Uncertainty:</b> As we move toward the Planck scale, the
<b>"Continuum Assumption"</b> of spacetime begins to fail. Conversely, at galactic scales,
effects like <b>Dark Matter</b> and <b>Dark Energy</b> are modeled as external parameters
rather than emergent properties.</li>
        <li><b>Predictive vs. Descriptive:</b> This framework is a powerful <b>tool for
"What-If" scenarios</b> and pedagogical exploration. It should not be treated as an absolute
representation of physical reality in regimes where empirical data is currently
unattainable.</li>
    </ul>
    <i><b>Status:</b> Research Tool / Theoretical Prototype. All results must be interpreted
within the limits of numerical integration and the chosen physical constants.</i>
</div>
```

MARKDOWN #213

```
# The Multi-Scale N-Body Framework: From Planck to Cosmos

To simulate the entire hierarchy of the universe, we define a **Generalized Potential
($\Phi_{total}$)** that transitions between different physical regimes based on the
interaction scale $r$.

### 1. The Unified Potential Equation
The total potential energy of the system is the sum of terms that dominate at specific
scales:

$$\Phi_{total}(r) = \Phi_{Planck} + \Phi_{Higgs} + \Phi_{Newton} + \Phi_{GR} +
\Phi_{\Lambda}$$

### 2. Scale-Specific Physics:


| Scale | Regime | Mathematical Driver | Dominant Term |
| :--- | :--- | :--- | :--- |
| **$10^{-35}$ m** | **Planck Scale** | Quantum Gravity / GUP | $\delta g_{\mu\nu}$ (Metric
Jitter) |
| **$10^{-18}$ m** | **Higgs Scale** | Particle Physics | $m = g \cdot \phi$ (Mass
Acquisition) |
| **$10^{11}$ m** | **Planetary** | Newtonian Mechanics | $1/r^2$ (Stable Orbits) |
| **$10^{21}$ m** | **Galactic** | Dark Matter / FMM | $O(N)$ Tree Structures (Cosmic Web) |
| **Scale-less** | **Inflation** | Scalar Fields | $a(t) \propto e^{Ht}$ (Exponential
Stretch) |
```

3. The Unified Equation of Motion
For any particle i, the evolution is governed by:
$$\mathbf{a}_i = \underbrace{\frac{\mathbf{F}_{cl}}{m(\phi)}}_{\text{Higgs-Newton}} + \underbrace{\mathbf{a}_{GR}}_{\text{Einstein}} + \underbrace{\mathbf{a}_{\Lambda}}_{\text{Dark Energy}} + \underbrace{\xi(\ell_P)}_{\text{Planck Noise}}$$

- $m(\phi)$: Mass is a function of the local **Higgs field**.
- $\mathbf{a}_{GR}$: Relativistic corrections ($1/c^2$).
- $\mathbf{a}_{\Lambda}$: Cosmological expansion.
- $\xi(\ell_P)$: Stochastic quantum fluctuations at the **Planck length**.

MARKDOWN #214

The End of Dynamics: Maximal Entropy (S_{max})

In the far future, the Universe reaches a state of **Maximal Entropy**. Mathematically, this is governed by the **Second Law of Thermodynamics** and the **Holographic Bound**.

1. Thermodynamic Equilibrium
Entropy (S) is a measure of disorder. At S_{max}, the temperature T is uniform across the entire universe (near absolute zero), and no more useful work (like star formation) can be performed.
$$\frac{dS}{dt} \geq 0 \implies S \to S_{max}$$

2. The Decay of Information
According to the **Hawking-Penrose theorems**, all matter eventually ends up in Black Holes, which then evaporate via **Hawking Radiation**. The final state is a dilute gas of photons and leptons, separated by distances larger than the observable horizon.

3. The Dominance of Dark Energy (Λ)
In this regime, the N-body acceleration equation is dominated entirely by the repulsive term:
$$\mathbf{a}_i \approx \frac{\Lambda c^2}{3} \mathbf{r}_i$$
Gravity $\sum \frac{Gm}{r^2}$ becomes effectively zero because $r \to \infty$. The "Cosmic Web" dissolves into a featureless, cold void.

MARKDOWN #215

Statistical Fingerprints of the Heat Death

To mathematically prove the state of **Maximal Entropy**, we track two key statistical indicators:

1. **Velocity Dispersion (σ_v^2):** In the early universe, gravity creates high-velocity "clusters". At S_{max}, velocities become uniform and extremely low as the system reaches thermal equilibrium.
2. **Structural Information (I):** We measure the **Correlation Function** of the particles. As $t \to \infty$, the correlation drops to zero, meaning the position of one particle tells you nothing about the others, a state of perfect disorder.
3. **The Expansion Dominance Ratio:**
$$\mathcal{R} = \frac{|\mathbf{a}_{Dark Energy}|}{|\mathbf{a}_{Gravity}|}$$
When $\mathcal{R} \gg 1$, structure formation is mathematically impossible.

CODE #216

```
def simulate_entropy_stats(steps=1000, dt=0.2):
    n_particles = 300
```

```
    # Start from a high-density state
    pos = np.random.normal(0, 1, (n_particles, 2))
    vel = np.random.normal(0, 0.5, (n_particles, 2))

    # Physics constants for the end of time
    G = 0.05
    Lambda = 0.02

    velocity_dispersion = []
    ratio_de_g = [] # Dark Energy vs Gravity ratio

    for s in range(steps):
        # 1. Gravity (Newtonian)
        diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
        dist_sq = np.sum(diff**2, axis=-1) + 1.0
        acc_g = np.sum(diff * (dist_sq**(-1.5))[:, :, np.newaxis], axis=1) * G

        # 2. Dark Energy (Repulsive)
        acc_de = Lambda * pos

        # 3. Stats Logging
        mag_g = np.mean(np.linalg.norm(acc_g, axis=1))
        mag_de = np.mean(np.linalg.norm(acc_de, axis=1))
        ratio_de_g.append(mag_de / (mag_g + 1e-9))

        v_disp = np.var(vel)
        velocity_dispersion.append(v_disp)

        # 4. Update
        vel = (vel + (acc_g + acc_de) * dt) * 0.99 # Cooling effect
        pos += vel * dt

    return np.array(velocity_dispersion), np.array(ratio_de_g)

v_stats, r_stats = simulate_entropy_stats()
```

CODE #217

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
plt.style.use('dark_background')

# Plot 1: Velocity Dispersion (Thermal Equilibrium)
ax1.plot(v_stats, color='cyan', lw=2)
ax1.set_title("The Cooling Universe: Decay of Kinetic Energy", fontsize=14)
ax1.set_xlabel("Time (Billions of Years)")
ax1.set_ylabel("Velocity Variance ($\sigma_v^2$)")
ax1.grid(True, alpha=0.1)

# Plot 2: Dark Energy Dominance
ax2.plot(r_stats, color='magenta', lw=2)
ax2.set_yscale('log') # Log scale is better for exponential growth
ax2.axhline(y=1.0, color='white', linestyle='--', label='Balance Point (G = DE)')
ax2.set_title("Expansion Dominance Over Gravity", fontsize=14)
ax2.set_xlabel("Time")
ax2.set_ylabel("Ratio (Repulsion / Attraction)")
ax2.legend()
ax2.grid(True, alpha=0.1)
```

```
plt.tight_layout()
plt.show()
```

MARKDOWN #218

```
# The Cosmic Life Cycle: From Inflation to Heat Death

This framework unifies the three phases of the universe's evolution into a single physical
and statistical model.

### 1. Inflation (The Birth of Order)
Driven by the Inflaton field $\phi$, space expands exponentially, stretching quantum
fluctuations into macroscopic seeds:
$$a(t) \propto e^{Ht}, \quad P(k) \approx A k^{n_s-1}$$
*Metric: Rapid growth of the scale factor $a(t)$.*

### 2. Gravity (The Formation of Structure)
Newtonian and Relativistic forces overcome expansion locally to form the Cosmic Web:
$$\mathbf{a}_i = \sum_{j \neq i} \frac{G m_j \mathbf{r}_{ij}}{(r_{ij}^2 + \epsilon^2)^{3/2}}
- 2H\mathbf{v}_i$$
*Metric: Growth of Density Perturbations $\delta = (\rho - \bar{\rho})/\bar{\rho} > 1$.*

### 3. Maximal Entropy (The Dissolution of Everything)
Dark Energy ($\Lambda$) eventually dominates, and the Second Law of Thermodynamics leads to
$S \to S_{max}$.
The acceleration is dominated by the repulsive term:
$$\mathbf{a}_i \approx \frac{\Lambda c^2}{3} \mathbf{r}_i, \quad \lim_{t \to \infty}
\text{Work}(W) = 0$$
*Metric: Ratio of Repulsion to Attraction $\mathcal{R} = |a_{DE}| / |a_G| \to \infty$.*
```

MARKDOWN #219

```
## Algorithmic Evolution: How the Simulation Changes Through Time

The $N$-body engine is not static; it must adapt its mathematical logic to reflect the
dominant physics of each cosmic era.

### 1. The Inflationary Phase: Stochastic Grid Generation
In this era, the simulation does not track "particles" in the classical sense. Instead, it
operates on a **Scalar Field ($\phi$)**.
- **The Algorithm:** The engine generates Gaussian Random Noise across a grid.
- **The Process:** It applies an exponential scaling factor $a(t)$ to the grid coordinates.
- **Goal:** To create the initial "Seeds" (Initial Conditions) for the $N$-body particles.

### 2. The Gravitational Phase: Tree-Based Hierarchical Collapse
As matter clumps together, the simulation transitions to a **Particle-Based** approach. This
is the era where our optimized algorithms are most active.
- **The Algorithm:** Use of **Barnes-Hut (Quadtrees/Octrees)** or **FMM**.
- **The Process:** The engine solves the Poisson equation to find density peaks ($\delta >
1$).
- **Complexity:** This is the most computationally expensive phase ($O(N \log N)$), as
trillions of interactions form the **Cosmic Web**.

### 3. The Heat Death Phase: Linear Repulsive Drift
As the universe reaches maximal entropy, the "N-Body" nature of the simulation effectively
dissolves.
- **The Algorithm:** Transition from Pairwise Interactions to **Linear Vector Fields**.
```

- **The Process:** Since particles become so distant ($r \to \infty$), the gravitational term $\sum 1/r^2$ is turned off (or becomes negligible).
- **The Result:** The simulation simplifies to a primitive expansion model where each particle's velocity is simply a function of its distance from the origin (Hubble Flow): $\mathbf{v}_i = H \mathbf{r}_i$.

MARKDOWN #220

Unified Framework: Spatiotemporal Scales in N-Body Simulations

To simulate the Universe from its birth to its eventual end, the N-body engine must transition through different **Mathematical Regimes**. This table summarizes how the physics and the simulation logic shift across scales.

1. Spatial Scales (The "Where")

Scale	Domain	Physics Regime	Computational Approach
10^{-35} m	**Planck Scale**	Quantum Gravity	Stochastic Metric Jitter (ξ)
10^{-18} m	**Subatomic**	Higgs Field	Dynamical Mass Acquisition ($m(\phi)$)
10^{11} m	**Planetary**	Newtonian / GR	Symplectic Integration (Verlet / RK4)
10^{21} m	**Galactic**	Dark Matter	Hierarchical Trees (Barnes-Hut / FMM)
10^{26} m	**Cosmological**	Dark Energy	Comoving Coordinates & Hubble Flow

---

2. Temporal Eras (The "When")

Phase A: Inflation (The Seed)
- **Primary Driver:** Inflaton Field (ϕ)
- **Simulation Logic:** **Scalar Field Grid**. The engine generates Gaussian noise and applies an exponential stretch $a(t) \propto e^{Ht}$.
- **Result:** Initial conditions for all matter.

Phase B: Gravitational Era (The Web)
- **Primary Driver:** Dark Matter & Baryonic Gravity
- **Simulation Logic:** **Hierarchical $O(N \log N)$ Summation**. Particles clump into filaments and nodes.
- **Result:** Formation of the **Cosmic Web**, galaxies, and stars.

Phase C: Heat Death (The Void)
- **Primary Driver:** Dark Energy (Λ) & Entropy (S)
- **Simulation Logic:** **Linear Repulsive Drift**. Gravity becomes negligible ($r \to \infty$); particles follow the expansion of space.
- **Result:** Maximal entropy (S_{max}) and the dissolution of structure.

CODE #221

```
# Time axis (Logarithmic to cover Inflation to Heat Death)
time = np.logspace(-2, 10, 500)

# Mathematical Models for Energy/Entropy
# 1. Complexity/Structure (Grows then decays)
structure = 10 * np.exp(-(np.log10(time)-4)**2 / 10)
# 2. Entropy (Always grows)
entropy = 5 * np.log10(time + 1)
# 3. Expansion Factor (Exponential then linear-ish)
```

```
expansion = time**1.5

plt.figure(figsize=(12, 6))
plt.style.use('dark_background')

plt.plot(time, structure, color='cyan', lw=3, label='Structural Complexity (Cosmic Web)')
plt.plot(time, entropy, color='magenta', lw=2, ls='--', label='Total Entropy (S)')
plt.fill_between(time, structure, color='cyan', alpha=0.1)

plt.xscale('log')
plt.title("The Grand Evolution: Structure vs. Entropy", fontsize=15)
plt.xlabel("Cosmic Time (Log Scale)")
plt.ylabel("Magnitude")
plt.axvline(x=10**4, color='white', ls=':', label='Peak Galaxy Formation')
plt.legend(loc='upper left')
plt.grid(True, alpha=0.1)
plt.show()
```

MARKDOWN #222

```
# Conclusion: Insights from N-Body Simulations

This comprehensive study of N-body systems has demonstrated the transition from predictable,
analytical mechanics to the complex world of deterministic chaos. Key takeaways include:

*   **The Complexity Barrier:** We observed that the **Direct Summation** method, while
highly accurate, scales at $O(N^2)$. This creates a computational bottleneck that
necessitates approximation algorithms like **Barnes-Hut** ($O(N \log N)$) and **Fast
Multipole Method** (**$O(N)$**) for large-scale galactic simulations.
*   **The Emergence of Chaos:** While the 1-body and 2-body problems (Keplerian orbits) are
stable and solvable, the introduction of a third body introduces **sensitivity to initial
conditions** (the Butterfly Effect). Even microscopic perturbations can lead to macroscopic
divergence in trajectories over time.
*   **Numerical Stability:** We highlighted the importance of **Symplectic Integrators**
(like Velocity Verlet) for energy conservation and the use of **softening parameters** to
prevent numerical singularities during close encounters.
*   **Hierarchical Stability:** We demonstrated that stability in multi-body systems can be
achieved through **hierarchical configurations** (e.g., a moon orbiting a planet, which in
turn orbits a star), provided the orbital ratios are sufficiently large to minimize tidal
disruptions.

This project serves as a robust foundation for diving into more advanced topics in
computational astrophysics. Beyond basic orbital mechanics, these numerical techniques are
essential for exploring the large-scale structure of the universe through **Dark Matter
simulations**, where billions of particles interact to form cosmic webs. Furthermore, the
principles of hierarchical stability and close encounters are critical for understanding the
**long-term evolution of star clusters** and the complex dynamics of **galactic
collisions**. By mastering these integrators and optimization algorithms, we can bridge the
gap between simple Newtonian physics and the large-scale simulation of celestial phenomena
that define our universe.
```

MARKDOWN #223

```
<div style="padding: 30px; background-color: #f8f9fa; border-radius: 15px; border: 1px solid
#dee2e6; text-align: center; margin-top: 50px;">
    <h1 style="color: #2c3e50; font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;
font-weight: 300;">
                Ø<ß    T h a n k   Y o u !
```

```
    </h1>
    <hr style="width: 50px; border: 1.5px solid #3498db; margin: 20px auto;">
    <p style="font-size: 1.2em; color: #7f8c8d; line-height: 1.6;">
        Thank you for journeying through the <b>Multi-Scale N-Body Framework</b>. <br>
        Your curiosity in exploring the boundaries between <b>classical mechanics</b>,
        <b>information theory</b>, and the <b>quantum limits of spacetime</b> is what
        drives the evolution of this project.
    </p>
    <p style="font-style: italic; color: #bdc3c7; margin-top: 20px;">
        "Somewhere, something incredible is waiting to be known." - Carl Sagan
    </p>
</div>
```

MARKDOWN #224

```
<h2 style="color: #24292e; border-bottom: 1px solid #eaecef; padding-bottom: 10px;
 f o n t - f a m i l y :   s a n s - s e r i f ; " >Ø=ÜÊ   P r o j e c t   S e l f - A s s e s s m e

<h4 style="color: #28a745; background-color: #f3fdf4; padding: 15px; border-left: 6px solid
 # 2 8 a 7 4 5 ;   b o r d e r - r a d i u s :   4 p x ;   f o n t - f a m i l y :   s a n s - s e r
Strengths</h4>
<ul style="color: #155724; line-height: 1.6; font-family: sans-serif; margin-bottom: 25px;">
    <li><b>Multi-Scale Coverage:</b> Successfully bridges Planck-scale limits with
cosmological N-body dynamics.</li>
    <li><b>Numerical Stability:</b> Implementation of the <b>Velocity Verlet</b> integrator
ensures long-term energy conservation.</li>
    <li><b>Interactive Visualization:</b> High-quality 3D Plotly renders allow for real-time
exploration of chaotic paths.</li>
    <li><b>Theoretical Depth:</b> Successfully integrates speculative concepts like
Information-Mass Equivalence ($E=mc^2=I$).</li>
</ul>

<h4 style="color: #dc3545; background-color: #fff5f5; padding: 15px; border-left: 6px solid
 # d c 3 5 4 5 ;   b o r d e r - r a d i u s :   4 p x ;   f o n t - f a m i l y :   s a n s - s e r
Weaknesses</h4>
<ul style="color: #721c24; line-height: 1.6; font-family: sans-serif; margin-bottom: 25px;">
    <li><b>Computational Bottleneck:</b> The direct $O(N^2)$ summation approach strictly
limits the simulation to a small number of bodies.</li>
    <li><b>Chaos Sensitivity:</b> Microscopic numerical rounding errors eventually lead to
trajectory divergence in high-noise scenarios.</li>
    <li><b>Verification Gap:</b> Experimental "What-If" models (Higgs/Information) remain
mathematically consistent but physically unverified.</li>
</ul>

<p style="text-align: right; font-size: 0.85em; color: #6a737d; border-top: 1px solid #eee;
padding-top: 10px;">
    Project Review | Created by <span style="color: #0366d6; font-weight: bold;">Dimo
Dimov</span> | March 2026
</p>
```

MARKDOWN #225

```
### Automated Project Metrics
This cell analyzes the "**N-body Simulations.ipynb**" file to generate structural
statistics.
It calculates the balance between documentation and implementation.
```

CODE #226

```
# 1. Configuration
notebook_filename = 'N-body Simulations.ipynb'

if os.path.exists(notebook_filename):
    # Load the notebook structure
    with open(notebook_filename, 'r', encoding='utf-8') as f:
        nb_content = nbformat.read(f, as_version=4)

    # 2. Metric Initialization
    cell_counts = {'markdown': 0, 'code': 0, 'raw': 0}
    total_code_lines = 0
    function_count = 0

    # 3. Content Scanning
    total_cells = len(nb_content.cells)
    for cell in nb_content.cells:
        cell_type = cell.cell_type
        cell_counts[cell_type] += 1

        if cell_type == 'code':
            lines = cell.source.splitlines()
            total_code_lines += len(lines)

            # Count Python function definitions
            for line in lines:
                clean_line = line.strip()
                if clean_line.startswith('def '):
                    function_count += 1

    # Calculate Percentages
    pct_md = (cell_counts['markdown'] / total_cells) * 100
    pct_code = (cell_counts['code'] / total_cells) * 100

    # 4. Console Summary Report
    print("=" * 50)
    print(f" PROJECT METRICS: {notebook_filename}")
    print("=" * 50)
    print(f" Total Cells in Project:   {total_cells}")
    print(f" Documentation (Markdown): {cell_counts['markdown']} cells ({pct_md:.1f}%)")
    print(f" Implementation (Python):  {cell_counts['code']} cells ({pct_code:.1f}%)")
    print(f" Total Lines of Code:      {total_code_lines}")
    print(f" Total Defined Functions:  {function_count}")
    print("=" * 50)

    # 5. Visual Representation
    plt.style.use('ggplot')
    labels = ['Documentation', 'Code Logic']
    values = [cell_counts['markdown'], cell_counts['code']]
    percentages = [pct_md, pct_code]
    colors = ['#3498db', '#2ecc71']

    plt.figure(figsize=(10, 6))
    bars = plt.bar(labels, values, color=colors, edgecolor='black', alpha=0.7)

    # Add value and percentage labels on top of bars
```

```
    for i, bar in enumerate(bars):
        yval = bar.get_height()
        label_text = f"{int(yval)} cells\n({percentages[i]:.1f}%)"
        plt.text(bar.get_x() + bar.get_width()/2, yval + 2.2,
                 label_text, ha='center', va='bottom',
                 fontsize=11, fontweight='bold', color='#333333')

    plt.title(f"Structural Composition of {notebook_filename}", fontsize=14, pad=25)
    plt.ylabel("Number of Cells", fontsize=12)
    plt.ylim(0, max(values) * 1.15) # Extra space for labels
    plt.grid(axis='y', linestyle='--', alpha=0.5)
    plt.tight_layout()
    plt.show()

else:
    print(f"Error: '{notebook_filename}' not found. Please verify the file path.")
```

MARKDOWN #227

```
# References
```

MARKDOWN #228

```
## 1. Mathematical Foundations & Chaos
```


1. Chierchia, L., Kaloshin, V., Mather, J., & Terracini, S. New Perspectives on the N-body Problem.
2. Simon, S., & Morales-Ruiz, J. (2009). On the meromorphic non-integrability of some $ N $-body problems. Discrete and Continuous Dynamical Systems, 24(4), 1225-1273.
3. Meyer, K. R. (1999). Periodic solutions of the N-body problem (Vol. 1719). Springer Science & Business Media.
4. Lovett, E. O. (1909). The problem of several bodies: recent progress in its solution. Science, 29(733), 81-91.
5. Shaw, B. Three bodies and no solution.
6. Hevia, D. F., & Ranada, A. F. (1996). Chaos in the three-body problem: the Sitnikov case. European Journal of Physics, 17(5), 295-302.
7. Stone, N. C., & Leigh, N. W. (2019). A statistical solution to the chaotic, non-hierarchical three-body problem. Nature, 576(7787), 406-410.
8. Poincaré, H. (1890). On the three-body problem and the equations of dynamics. Acta Math, 13(1), 144203.
9. Bulgac, A., Abdurrahman, I., & WlazBowski, G. (2
conditions of the Time-Dependent Density Functional Theory. arXiv preprint arXiv:2108.10858.
10. Ennabati, A. (2025). A Relational Stability Reformulation of the N-Body Problem Through the" E Field of the Relative Cosmic Equilibrium (R
11. Quinn, T. (2019). N-body simulations of the Solar System, planet formation, and galaxy clusters. In The Restless Universe Applications of Gravitational N-Body Dynamics to Planetary Stellar and Galactic Systems (pp. 1-20). CRC Press.
12. Przybylska, M., & Maciejewski, A. J. (2025). Non-integrability of charged three-body problem. Celestial Mechanics and Dynamical Astronomy, 137(1), 8.
13. Yagasaki, K. (2024). Non-integrability of the restricted three-body problem. Ergodic Theory and Dynamical Systems, 44(10), 3012-3040.
14. Cipriani, P., & Pettini, M. (2003). Strong chaos in N-body problem and microcanonical thermodynamics of collisionless self gravitating systems. Astrophysics and space science, 283(3), 347-368.
15. Maciejewski, A. J., & Przybylska, M. (2003). Non-integrability of the problem of a rigid satellite in gravitational and magnetic fields. Celestial Mechanics and Dynamical Astronomy, 87(4), 317-351.
16. Pinzari, G. (2017). Two–centre and three–body problems: renormalizable integrability and

instabilities.

MARKDOWN #229

```markdown
## 2. Numerical Integration & Symplectic Methods

1. Markiewicz, D. W. (1999). Survey on symplectic integrators. Preprint Univ. California at Berkeley, Spring.
2. Hernandez, D. M., & Bertschinger, E. (2018). Time-symmetric integration in astrophysics. Monthly Notices of the Royal Astronomical Society, 475(4), 5570-5584.
3. Hernandez, D. M. (2019). Should N-body integrators be symplectic everywhere in phase space?. Monthly Notices of the Royal Astronomical Society, 486(4), 5231-5238.
4. Tu, X., Wang, Q., & Tang, Y. (2022). Symplectic Integrators in Corotating Coordinates. arXiv preprint arXiv:2202.03779.
5. Xin, W., & Tian-yi, H. (2005). Constraints and numerical integration. Chinese Astronomy and Astrophysics, 29(1), 81-91.
6. Kuppa, K. (2016). Long-term Orbit Propagation Using Symplectic Integration Algorithms.
7. Scherer, P. O. (2017). Equations of motion. In Computational Physics: Simulation of Classical and Quantum Systems (pp. 289-321). Cham: Springer International Publishing.
8. Bolatti, D. A., & de Ruiter, A. H. PRACTICAL GALERKIN VARIATIONAL INTEGRATORS FOR ORBITAL DYNAMICS ABOUT ASTEROIDS.
9. Bolatti, D. A., & de Ruiter, A. H. (2019). Galerkin variational integrators for orbit propagation with applications to small bodies. Journal of Guidance, Control, and Dynamics, 42(2), 347-363.
10. French, A., Cullerne, J. P., & Kanchanasakdichai, O. (2019). Numerical methods as an introduction to calculus. Physics Education, 54(4), 045009.
11. Parkkulainen, E. (2019). Real-time Physics Simulation.
12. Iserles, A., & Quispel, G. R. W. (2018). Why geometric numerical integration?. In Discrete Mechanics, Geometric Integration and Lie–Butcher Series: DMGILBS, Madrid, May 2015 (pp. 1-28). Cham: Springer International Publishing.
13. Blanes, S., & Casas, F. (2025). A concise introduction to geometric numerical integration. CRC press.
14. Hairer, E. (2014). Challenges in geometric numerical integration. Trends in contemporary mathematics, 125-135.
15. Gonçalves Ferrari, G., Boekholt, T., & Portegies Zwart, S. F. (2014). A Keplerian-based Hamiltonian splitting for gravitational N-body simulations. Monthly Notices of the Royal Astronomical Society, 440(1), 719-730.
16. Blanes, S., & Casas, F. (2022). Preface for the special issue “Geometric numerical integration, twenty-five years later”. International Journal of Computer Mathematics, 99(1), 1-3.
17. Boekholt, T. (2015). Chaotic dynamics in N-body systems. Ph. D. Thesis.
18. Quinlan, G. D., & Tremaine, S. (1992). On the reliability of gravitational N-body integrations. Monthly Notices of the Royal Astronomical Society, 259(3), 505-518.
19. El-Zant, A. A., Everitt, M. J., & Kassem, S. M. (2019). Errors, chaos, and the collisionless limit. Monthly Notices of the Royal Astronomical Society, 484(2), 1456-1474.
20. Aarseth, S. J. (1988). Integration methods for small N-body systems. In The Few Body Problem: Proceedings of the 96th Colloquium of the International Astronomical Union Held in Turku, Finland, June 14–19, 1987 (pp. 287-307). Dordrecht: Springer Netherlands.
21. Goodman, J., Heggie, D. C., & Hut, P. (1993). On the exponential instability of N-body systems. Astrophysical Journal v. 415, p. 715, 415, 715.
```

MARKDOWN #230

```markdown
## 3. Algorithmic Optimization (The $O(N^2)$ Barrier)

1. Zhuo, Z., Zhang, X., Niu, W., Yang, G., & Zhang, J. (2016). Improving data field hierarchical clustering using Barnes–Hut algorithm. Pattern Recognition Letters, 80, 113-120.
```

2. Singh, J. P., Holt, C., Totsuka, T., Gupta, A., & Hennessy, J. (1995). Load balancing and data locality in adaptive hierarchical N-body methods: Barnes-Hut, fast multipole, and radiosity. Journal of Parallel and Distributed Computing, 27(2), 118-141.
3. Grama, A., Kumar, V., & Sameh, A. (1998). Scalable parallel formulations of the Barnes–Hut method for n-body simulations. Parallel Computing, 24(5-6), 797-822.
4. Balarezo, J. G., & Pineda, I. (2025, October). Numerical Performance Analysis of the Direct and Barnes-Hut Algorithm for Solving the N-Body Problem. In Conference on Information and Communication Technologies of Ecuador (pp. 365-376). Cham: Springer Nature Switzerland.
5. Aronsson, J., Butt, K., Jeffrey, I., & Okhmatovski, V. I. (2010). The Barnes–Hut hierarchical center-of-charge approximation for fast capacitance extraction in multilayered media. IEEE transactions on microwave theory and techniques, 58(5), 1175-1188.
6. Wang, Q. (2021). A hybrid fast multipole method for cosmological n-body simulations. Research in Astronomy and Astrophysics, 21(1), 003.
7. Mukherjee, D., Zhu, Q., Trac, H., & Rodriguez, C. L. (2021). Fast multipole methods for N-body simulations of collisional star systems. The Astrophysical Journal, 916(1), 9.
8. Koumoutsakos, P. (1995). Fast multipole methods for three-dimensional N-body problems. Center for Turbulence Research Annual Research Briefs: 1995.
9. Yokota, R., & Barba, L. A. (2011). Treecode and fast multipole method for N-body simulation with CUDA. In GPU Computing Gems Emerald Edition (pp. 113-132). Morgan Kaufmann.
10. Dehnen, W. (2014). A fast multipole method for stellar dynamics. Computational Astrophysics and Cosmology, 1(1), 1.
11. Kuzmin, N. M., Sirotin, D. S., & Khoperskov, A. V. (2024). Efficiency of parallel c o m p u t a t i o n s o f g r a v i t a t i o n a l f o r c e s b y T r e e C o d e m e D 8 7 8 : 0 8 : > < ? L N B 5 @ = > 5 < > 4 5 ; 8 @ > 2 0 = 8 5 , 2 7 (4) , 3 9 - 5 5 .
12. Huang, S. Y., Spurzem, R., & Berczik, P. (2016). Performance analysis of parallel gravitational N-body codes on large GPU clusters. Research in Astronomy and Astrophysics, 16(1), 011.
13. Klypin, A. (2018). Cosmological N-Body Simulations. In The Encyclopedia of Cosmology: Volume 2: Numerical Simulations in Cosmology (pp. 27-55).
14. Dehnen, W., & Read, J. I. (2011). N-body simulations of gravitational dynamics. The European Physical Journal Plus, 126(5), 55.
15. Zwart, S. P., & Bédorf, J. (2014). Computational gravitational dynamics with modern numerical accelerators. arXiv preprint arXiv:1409.5474.
16. Mukherjee, D. (2025). Accelerating the Next Frontier of Gravitational Waves: Advanced Computational Methods & Simulations (Doctoral dissertation, Carnegie Mellon University).
17. Liu, P., & Bhatt, S. N. (1994, August). Experiences with parallel N-body simulation. In Proceedings of the sixth annual ACM symposium on Parallel algorithms and architectures (pp. 122-131).
18. Blelloch, G., & Narlikar, G. (1997). A practical comparison of n-body algorithms. Parallel Algorithms, 30, 81-96.
19. Teng, S. H. (1998). Provably good partitioning and load balancing algorithms for parallel adaptive N-body simulation. SIAM Journal on Scientific Computing, 19(2), 635-656.
20. Warren, M. S., & Salmon, J. K. (1993, December). A parallel hashed oct-tree n-body algorithm. In Proceedings of the 1993 ACM/IEEE conference on Supercomputing (pp. 12-21).
21. Liu, P. (1994). The parallel implementation of N-body algorithms (Doctoral dissertation, Yale University).
22. Aghababaie Beni, L., & Chandramowlishwaran, A. (2017, August). PASCAL: A Parallel Algorithmic SCALable Framework for N-body Problems. In European Conference on Parallel Processing (pp. 482-496). Cham: Springer International Publishing.
23. Gualandris, A., Zwart, S. P., & Tirado-Ramos, A. (2007). Performance analysis of direct n-body algorithms for astrophysical simulations on distributed systems. Parallel Computing, 33(3), 159-173.

MARKDOWN #231

4. Specific Solutions & Phenomena (Figure-Eight & Butterfly Effect)

1. Chenciner, A., & Montgomery, R. (2000). A remarkable periodic solution of the three-body problem in the case of equal masses. Annals of Mathematics, 881-901.
2. Montgomery, R. (2001). A new solution to the three-body problem. differential equations, 1001, d2xi.
3. Montgomery, R. (2002). Action spectrum and collisions in the planar three-body problem. Contemporary Mathematics, 292(173-184), 1.
4. Pina, E., & Lonngi, P. (2003). On the figure eight orbit of the three-body problem. Revista mexicana de física, 49(5), 439-444.
5. Montgomery, R., & Buzelli, C. (2019). The three-body problem. Scientific American, 321(2), 66-75.
6. Meyer, K. R. (1981). Periodic solutions of the N-body problem. Journal of differential equations, 39(1), 2-38.
7. Chen, K. C. (2003). Variational methods on periodic and quasi-periodic solutions for the N-body problem. Ergodic Theory and Dynamical Systems, 23(6), 1691-1715.
8. Zelati, V. C. (1989). A class of periodic solutions of the N-body problem. Celestial Mechanics and Dynamical Astronomy, 46(2), 177-186.
9. Meyer, K. R. (1999). Periodic solutions of the N-body problem (Vol. 1719). Springer Science & Business Media.
10. Simó, C. (2019). Periodic orbits of the planar N-body problem with equal masses and all bodies on the same path. In The Restless Universe Applications of Gravitational N-Body Dynamics to Planetary Stellar and Galactic Systems (pp. 265-284). CRC Press.
11. Liao, S., & Li, X. (2019). On the periodic solutions of the three-body problem. National Science Review, 6(6), 1070-1071.
12. Shen, B. W., Pielke Sr, R. A., & Zeng, X. (2023). The 50th anniversary of the metaphorical butterfly effect since Lorenz (1972): Multistability, multiscale predictability, and sensitivity in numerical models. Atmosphere, 14(8), 1279.
13. Introna, M. (2026). Mathematical and Computational Methods for the n-Body Problem.
14. Bo-Wen, S., PielkeSr, R. A., & Zeng, X. (2023). The 50th Anniversary of the Metaphorical Butterfly Effect since Lorenz (1972): Multistability, Multiscale Predictability, and Sensitivity in Numerical Models. Atmosphere, 14(8), 1279.
15. Liao, S. (2013). Chaotic motion of three-body problem: an origin of macroscopic randomness of the universe. arXiv preprint arXiv:1304.2089.
16. Boekholt, T. (2015). Chaotic dynamics in N-body systems. Ph. D. Thesis.
17. Georgakarakos, N. (2013). The dependence of the stability of hierarchical triple systems on the orbital inclination. New Astronomy, 23, 41-48.
18. Conway, L. (2025). Higher-Order Effects in the Dynamics of Hierarchical Triple Systems (Doctoral dissertation, University of Florida).
19. Haim, N., & Katz, B. (2018). Extreme close approaches in hierarchical triple systems with comparable masses. Monthly Notices of the Royal Astronomical Society, 479(3), 3155-3166.
20. Hamers, A. S. (2012). The evolution of coeval stellar hierarchical triple systems (Master's thesis).
21. Walker, I. W. (1980). Empirical stability criteria for hierarchical many-body dynamical systems in celestial mechanics. University of Glasgow (United Kingdom).
22. Ceppi, S., Cuello, N., Lodato, G., Clarke, C., Toci, C., & Price, D. J. (2022). Accretion rates in hierarchical triple systems with discs. Monthly Notices of the Royal Astronomical Society, 514(1), 906-919.
23. Bailyn, C. D. (1989). The formation of hierarchical triple systems by single-star-binary encounters. Astrophysical Journal, Part 1 (ISSN 0004-637X), vol. 341, June 1, 1989, p. 175-185., 341, 175-185.
24. Voyatzis, G., & Mourtetzikoglou, A. (2019). Periodic motion and stability of gravitational planar triple systems. Frontiers in Astronomy and Space Sciences, 5, 49.
25. Lu, C. X., & Naoz, S. (2019). Supernovae kicks in hierarchical triple systems. Monthly Notices of the Royal Astronomical Society, 484(2), 1506-1525.
26. Eggleton, P., & Kiseleva, L. (1995). An empirical condition for stability of hierarchical triple systems. Astrophysical Journal v. 455, p. 640, 455, 640.

MARKDOWN #232

```
## 5. Advanced & Speculative Physics (Theoretical Frontiers)

1. Lairez, D. (2025). Energy and Information: a Chronicle of Hesitations on the Role of the Observer in Physics. arXiv preprint arXiv:2509.06957.
2. Aryal, M., & Everett, A. E. (1986). Gravitational effects of global strings. Physical Review D, 33(2), 333.
3. Cline, J. M., Joyce, M., & Kainulainen, K. (2000). Supersymmetric electroweak baryogenesis. Journal of High Energy Physics, 2000(07), 018-018.
4. Demaria, A., & Volkas, R. R. (2005). Kink-induced symmetry breaking patterns in brane-world SU (3) 3 trinification models. Physical Review D—Particles, Fields, Gravitation, and Cosmology, 71(10), 105011.
5. Huber, S. J., & Schmidt, M. G. (1999). SUSY variants of the electroweak phase transition. The European Physical Journal C-Particles and Fields, 10(3), 473-481.
6. Shin, E. M., & Volkas, R. R. (2004). O (10) kinks: Clash of symmetries on the brane and the gauge hierarchy problem. Physical Review D, 69(4), 045010.
7. Cline, J. M., & Kainulainen, K. (2020). Electroweak baryogenesis at high bubble wall velocities. Physical Review D, 101(6), 063525.
8. Basler, P., Biermann, L., Mühlleitner, M., & Müller, J. (2023). Electroweak baryogenesis in the CP-violating two-Higgs doublet model. The European Physical Journal C, 83(1), 57.
9. Salucci, P., Esposito, G., Lambiase, G., Battista, E., Benetti, M., Bini, D., ... & Yunge, A. A. (2021). Einstein, Planck and Vera Rubin: relevant encounters between the Cosmological and the Quantum Worlds. Frontiers in Physics, 8, 603190.
10. Demir, D. (2017). Naturalizing gravity of the quantum fields, and the hierarchy problem. arXiv preprint arXiv:1703.05733.
11. Salopek, D. S. (1994). Searching for quantum gravity effects in cosmological data. International Journal of Modern Physics D, 3(01), 257-263.
12. DATA, C. (1995). SEARCHING FOR QUANTUM GRAVITY EFFECTS IN. Journees Relativistes' 93, 257.
13. Song, Y., Hu, B., Ruan, C. Z., Moretti, C., & Monaco, P. (2024). An implementation of nDGP gravity in Pinocchio. Journal of Cosmology and Astroparticle Physics, 2024(07), 093.
14. Petrovski, J. (2026). Causal Memory Gravity.
15. Brax, P. (2013). Screening mechanisms in modified gravity. Classical and Quantum Gravity, 30(21), 214005.
16. Carlesi, E., Knebe, A., Yepes, G., Gottlöber, S., Jiménez, J. B., & Maroto, A. L. (2012). N-body simulations with a cosmic vector for dark energy. Monthly Notices of the Royal Astronomical Society, 424(1), 699-715.
17. Neukart, F., Marx, E., & Vinokur, V. (2025). Quantum Memory Matrix framework applied to cosmological structure formation and dark matter phenomenology. Preprints, 2025042379.
18. Srichan, C. (2025). Entanglement-Weighted Correlation Gravity: A Conservative Effective Field Theory Framework.
19. Youvan, D. C. (2025). Heavenly Minds: Applying Informafional Tunneling to Stars, Galaxies, and Divergent Intelligences.
20. Muhlert, M. (2025). The Creative Principle From the Nature of Time to a Universal Dynamic of Emergence. Available at SSRN 5404472.
21. Calzetta, E. (1994). Homoclinic chaos in relativistic cosmology. NATO ASI SERIES B PHYSICS, 332, 203-203.
22. Levich, E. (2014). Theory of Order in Cosmos. Journal" Scientific Israel-Technological Advantages, 16(1).
23. Calzetta[1], E. (1994). IAFE, cc 67, suc 28 (1428), Buenos Aires, Argentina. Deterministic Chaos in General Relativity, 332, 203.
```

MARKDOWN #233

```
<div style="border: 2px solid #0366d6; padding: 40px; border-radius: 15px; background-color: #ffffff; margin: 20px auto; max-width: 850px; font-family: sans-serif;">
<h2 style="color: #0366d6; text-align: center; font-weight: 300; margin-top: 0; border:
```

```
none; background: none;">The Silent Symphony of Gravity</h2>
<p style="font-size: 1.3em; line-height: 1.6; color: #444d56; text-align: center; margin:
30px 0;">In the eternal dance of N-bodies, we find a beautiful paradox where chaos and
mathematical perfection become one.</p>
<hr style="border: 0; border-top: 1px solid #f1f8ff; margin-top: 30px;">
<p style="text-align: right; font-size: 1em; color: #6a737d; margin-bottom: 0;">Created by
<span style="color: #0366d6; font-weight: bold;">Dimo Dimov</span> | 2026</p>
</div>
```